

Neural Networks

Learning the network: Part 1

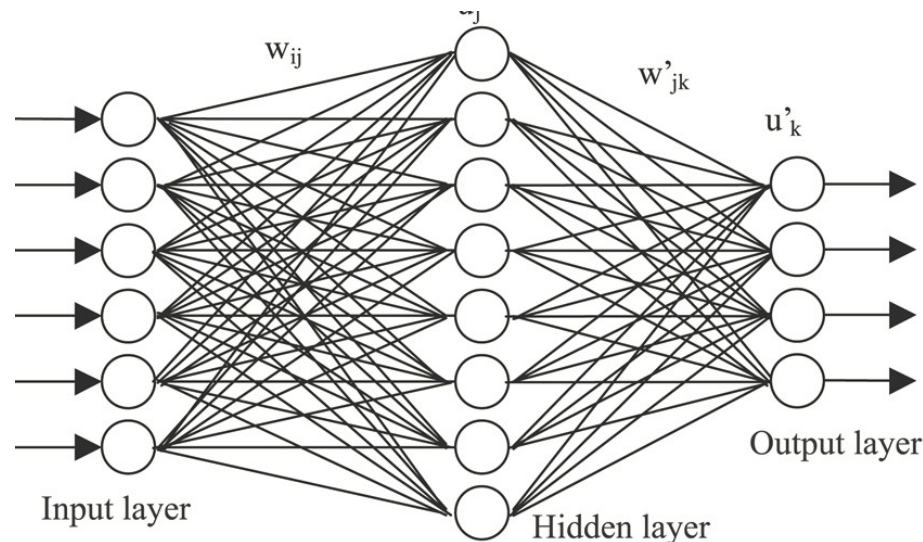
11-785, Fall 2026

Lecture 3

Topics for the day

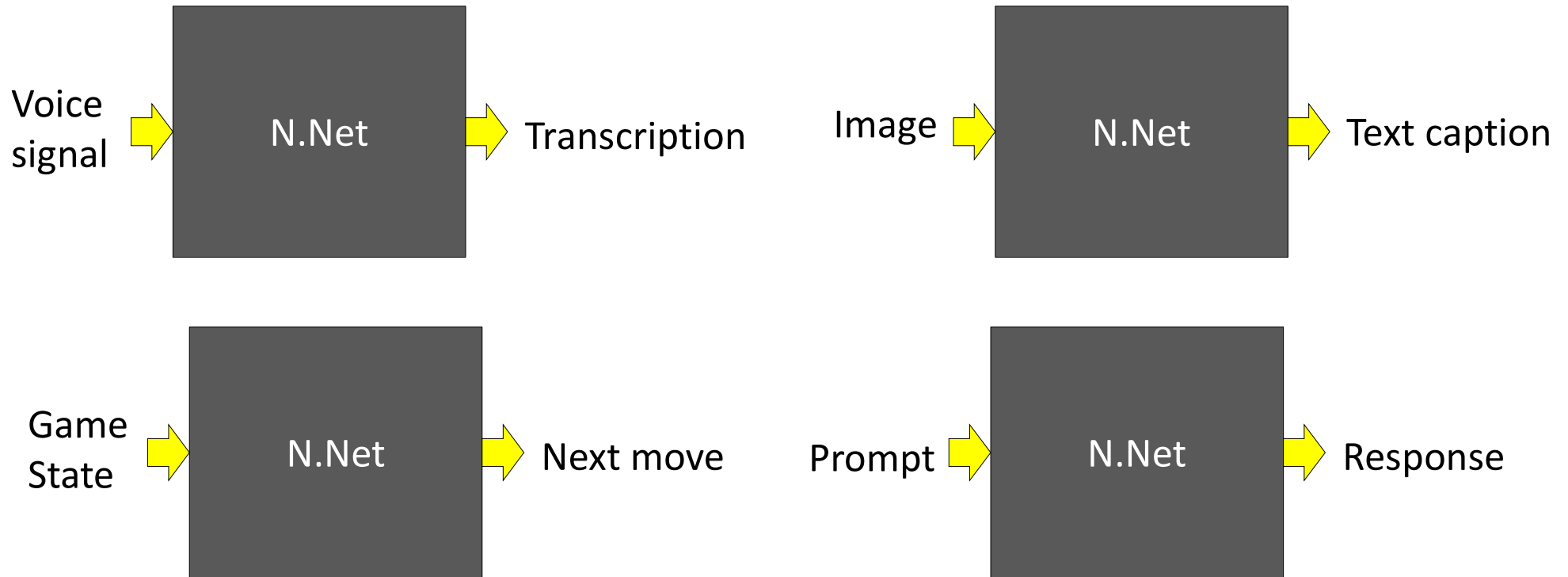
- The problem of learning
- The perceptron rule for learning individual perceptrons
 - And its inapplicability to multi-layer perceptrons
- Greedy solutions for classification networks: ADALINE and MADALINE
- Learning through Empirical Risk Minimization
- Intro to function optimization and gradient descent

Recap



- **Neural networks are universal function approximators**
 - Can model any Boolean function
 - Can model any classification boundary
 - Can model any continuous valued function
- *Provided the network satisfies minimal architecture constraints*
 - Networks with fewer than the required number of parameters can be very poor approximators

These boxes are functions



- Take an input
- Produce an output
- Can be modeled by a neural network!

Questions



- Preliminaries:
 - How do we represent the input?
 - How do we represent the output?
- How do we compose the network that performs the requisite function?

Questions



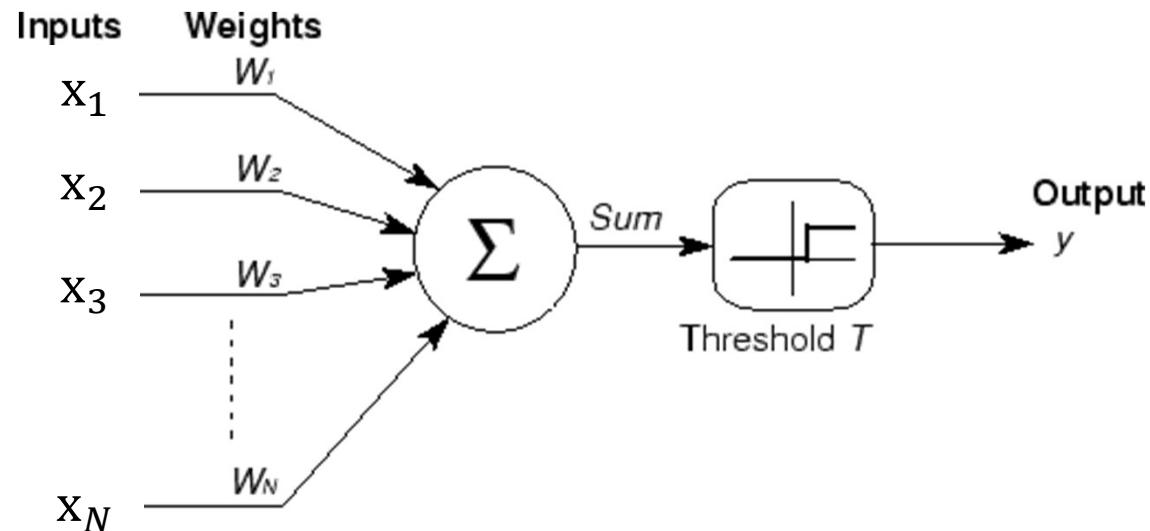
- Preliminaries:

- How do we represent the input?
- How do we represent the output?

A bit later in the program

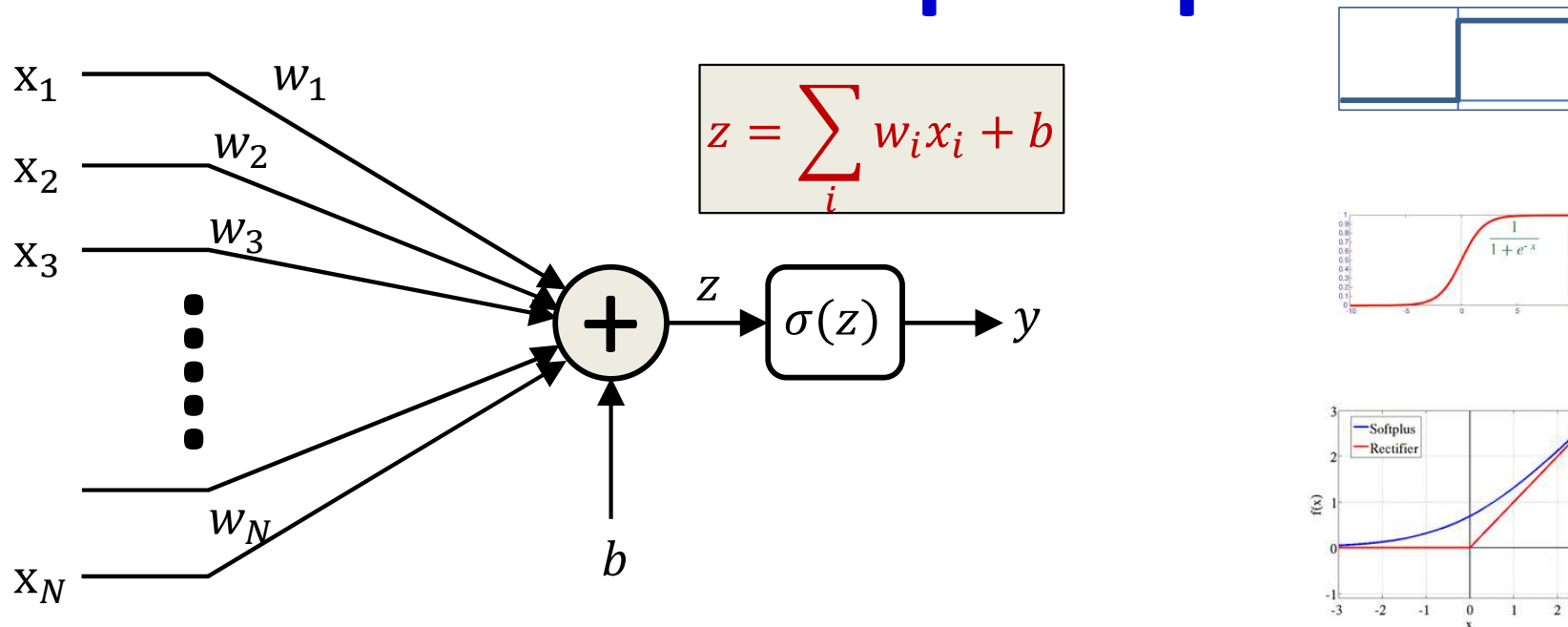
- *How do we compose the network that performs the requisite function?* ←

The original perceptron



- Simple threshold unit
 - Unit comprises a set of weights and a threshold

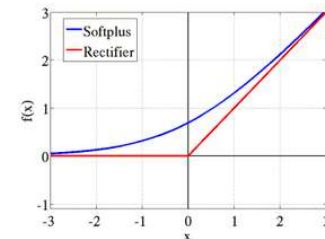
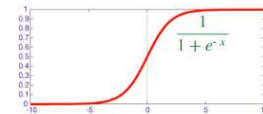
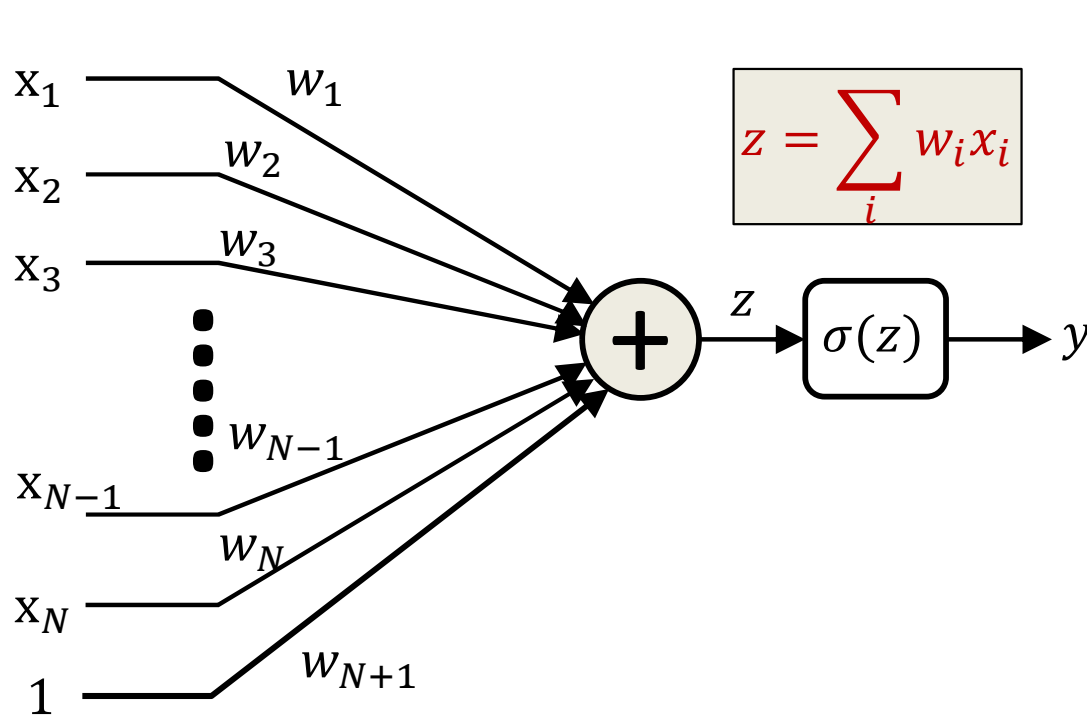
Preliminaries: The units in the network – the perceptron



Activation functions $\sigma(z)$

- Perceptron
 - General setting, inputs are real-valued
 - A *bias* b representing a threshold to trigger the perceptron
 - Activation functions are not necessarily threshold functions
- The parameters of the perceptron (which determine how it behaves) are its weights and bias

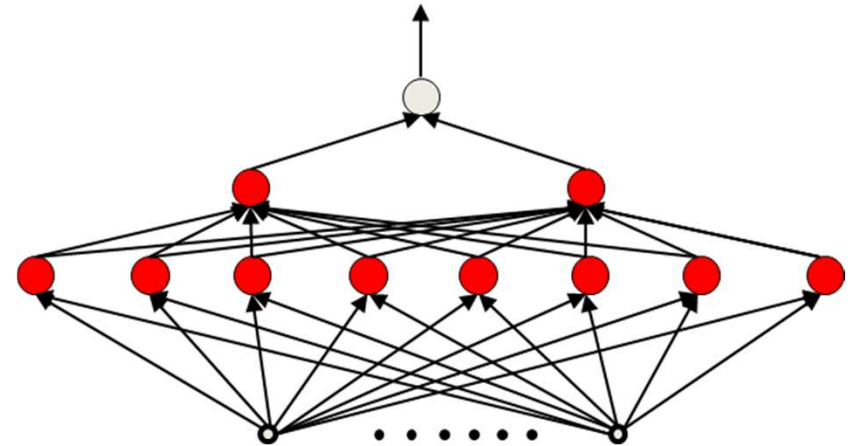
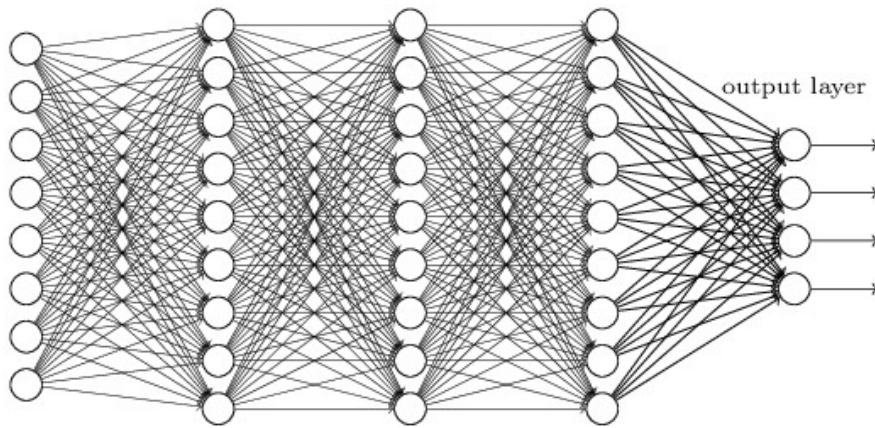
Preliminaries: Redrawing the neuron



Activation functions $\sigma(z)$

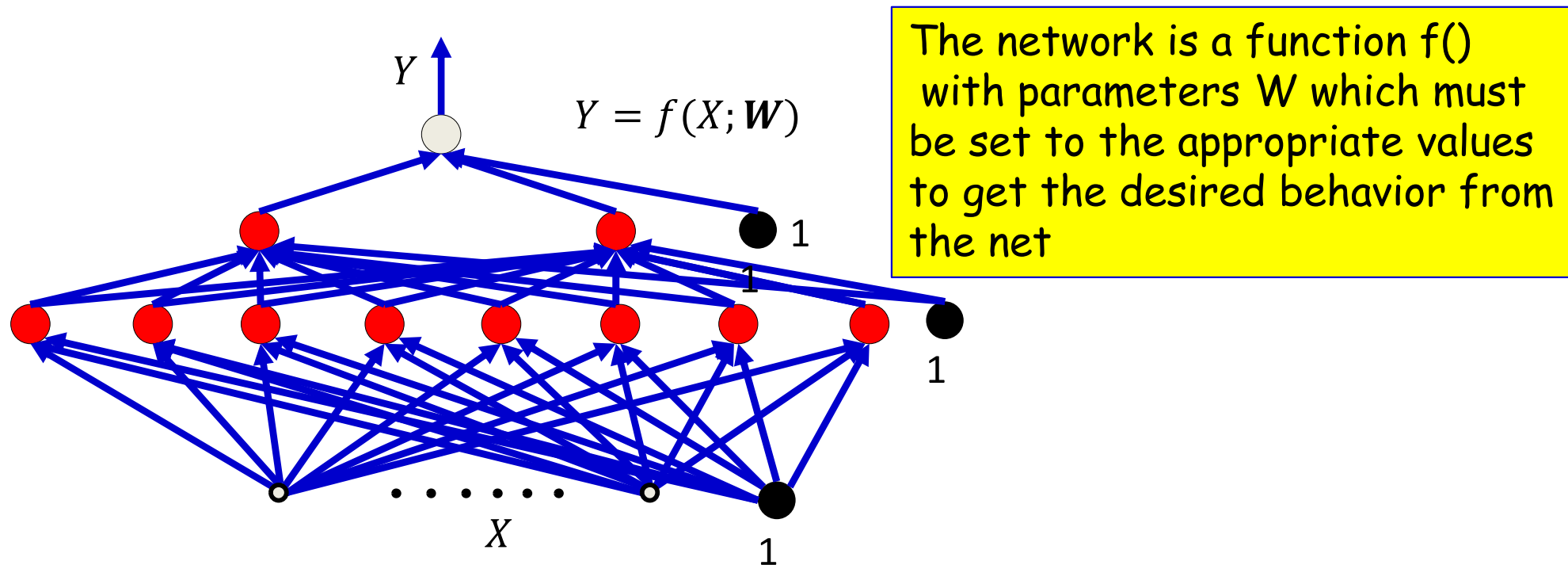
- The bias can also be viewed as the weight of another input component that is always set to 1
 - If the bias is not explicitly mentioned, we will implicitly be assuming that every perceptron has an additional input that is always fixed at 1

First: the structure of the network



- We will assume a *feed-forward* network
 - No loops: Neuron outputs do not feed back to their inputs directly or indirectly
 - Loopy networks are a future topic
- **Part of the design of a network: The architecture**
 - How many layers/neurons, which neuron connects to which and how, etc.
- For now, assume the architecture of the network is capable of representing the needed function

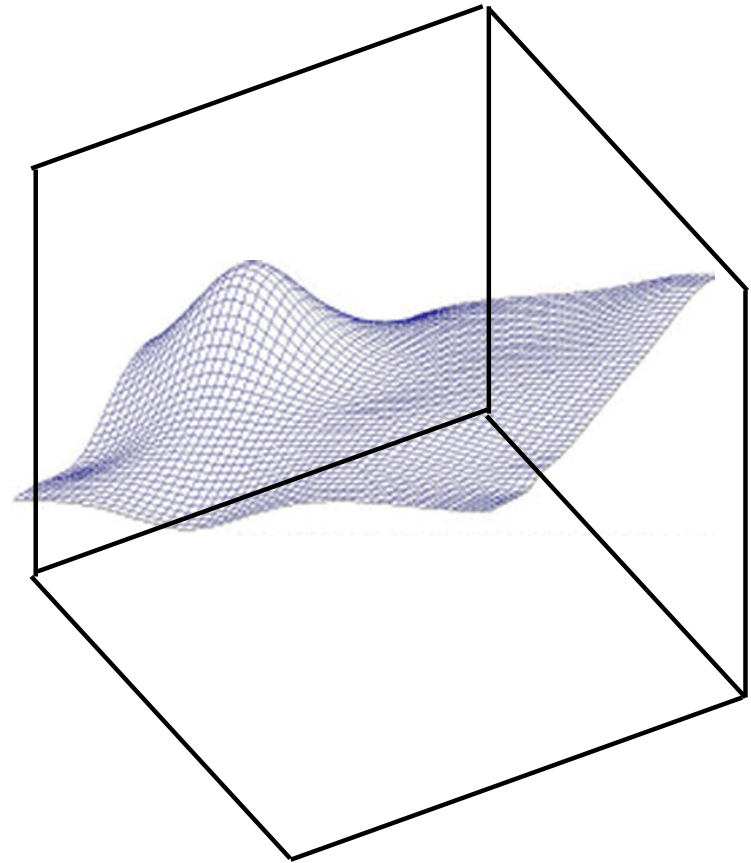
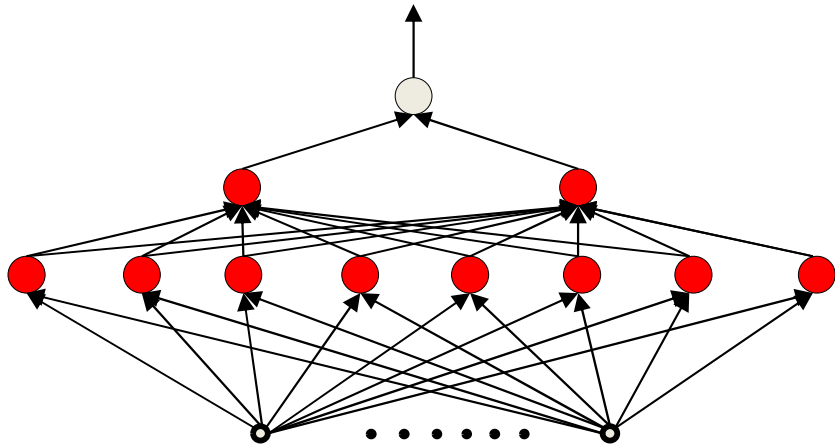
What we learn: The parameters of the network



- **Given:** the architecture of the network
- **The parameters of the network:** The weights and biases
 - The weights associated with the blue arrows in the picture
- **Learning the network :** Determining the values of these parameters such that the network computes the desired function

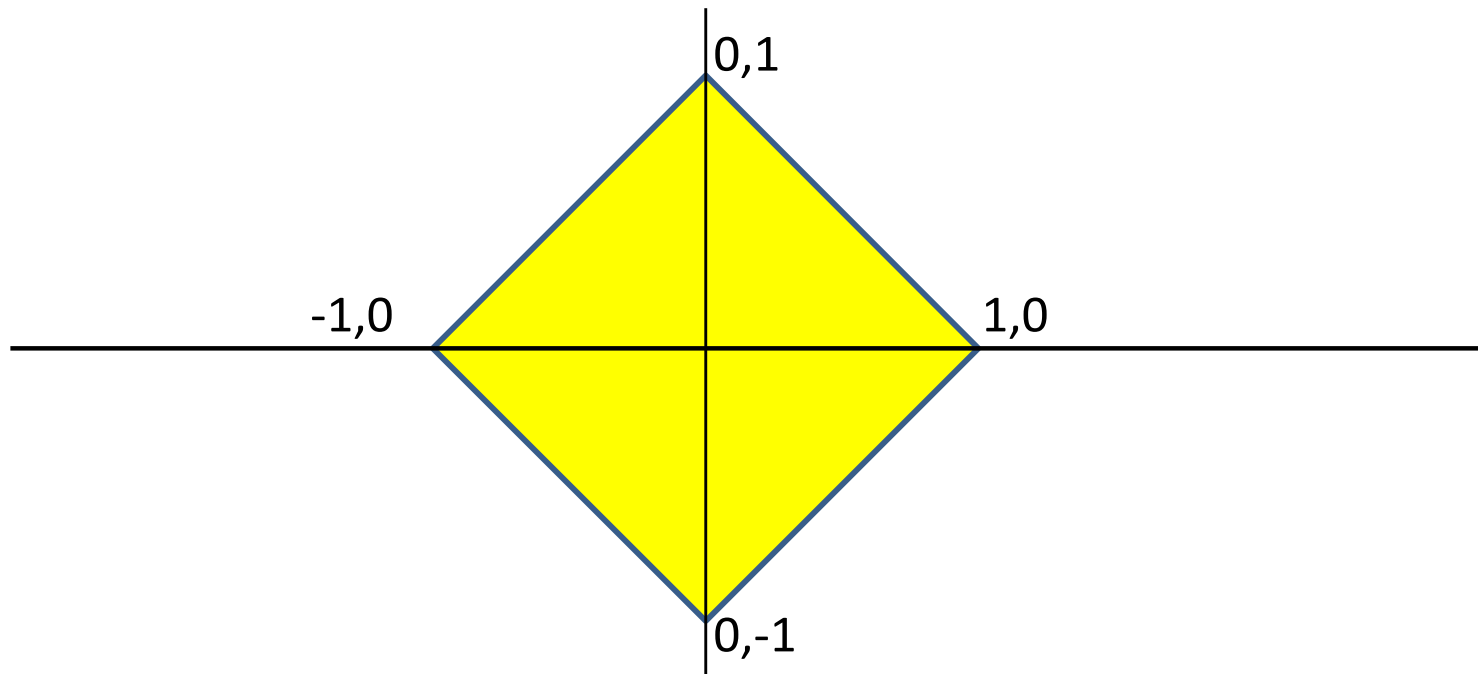
- Moving on..

The MLP *can* represent anything



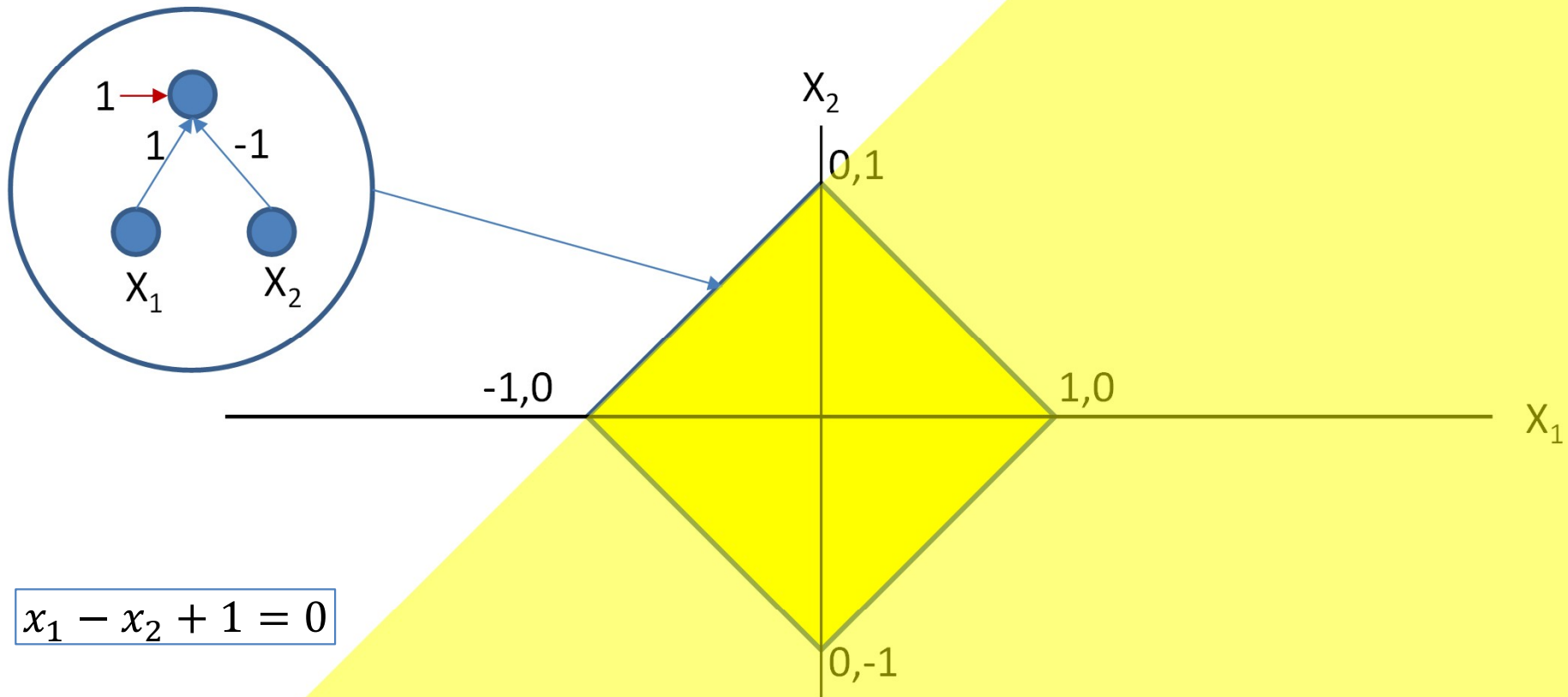
- The MLP *can be constructed* to represent anything
- But *how* do we construct it?

Option 1: Construct by hand



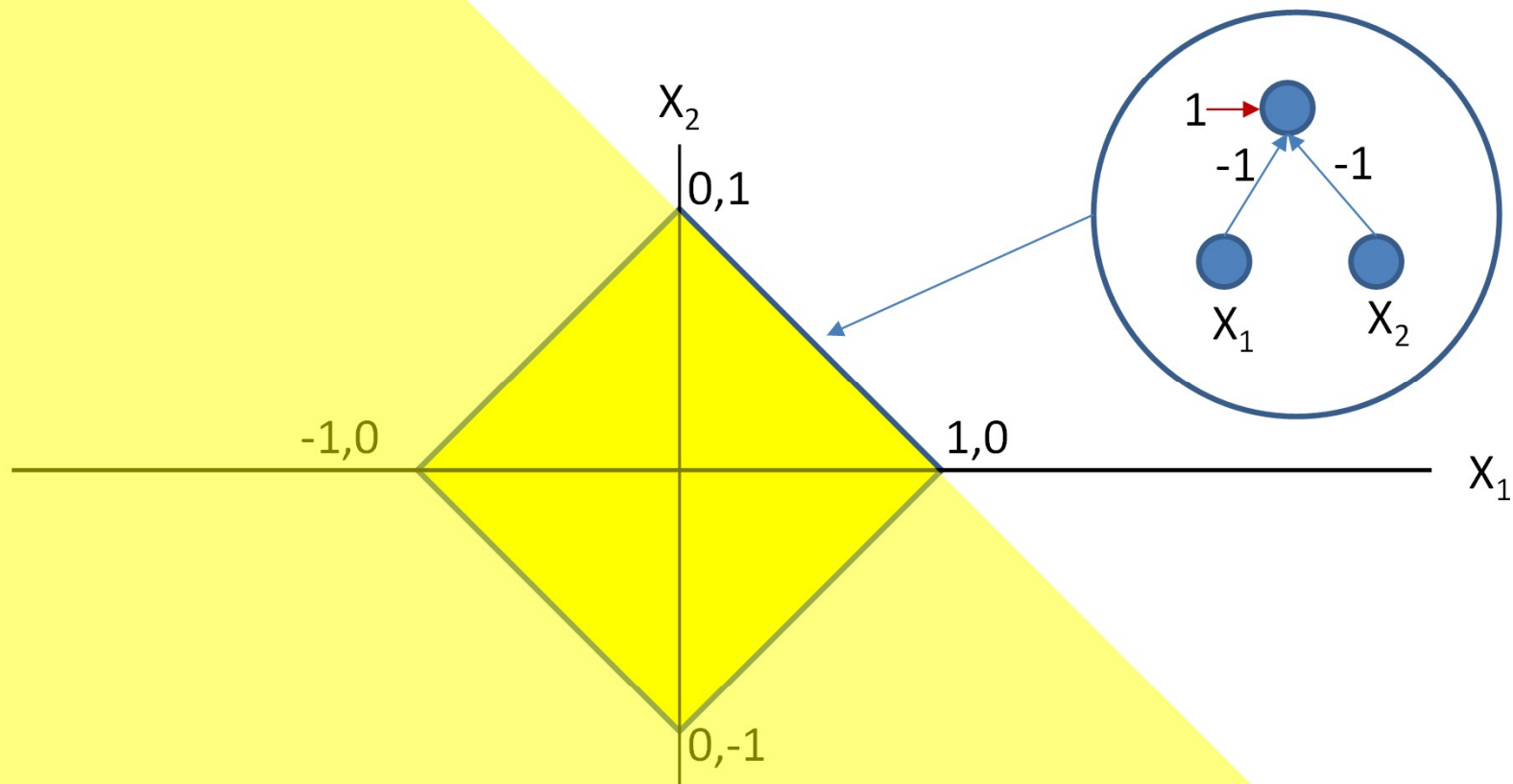
- Given a function, *handcraft* a network to satisfy it
- E.g.: Build an MLP to classify this decision boundary

Option 1: Construct by hand



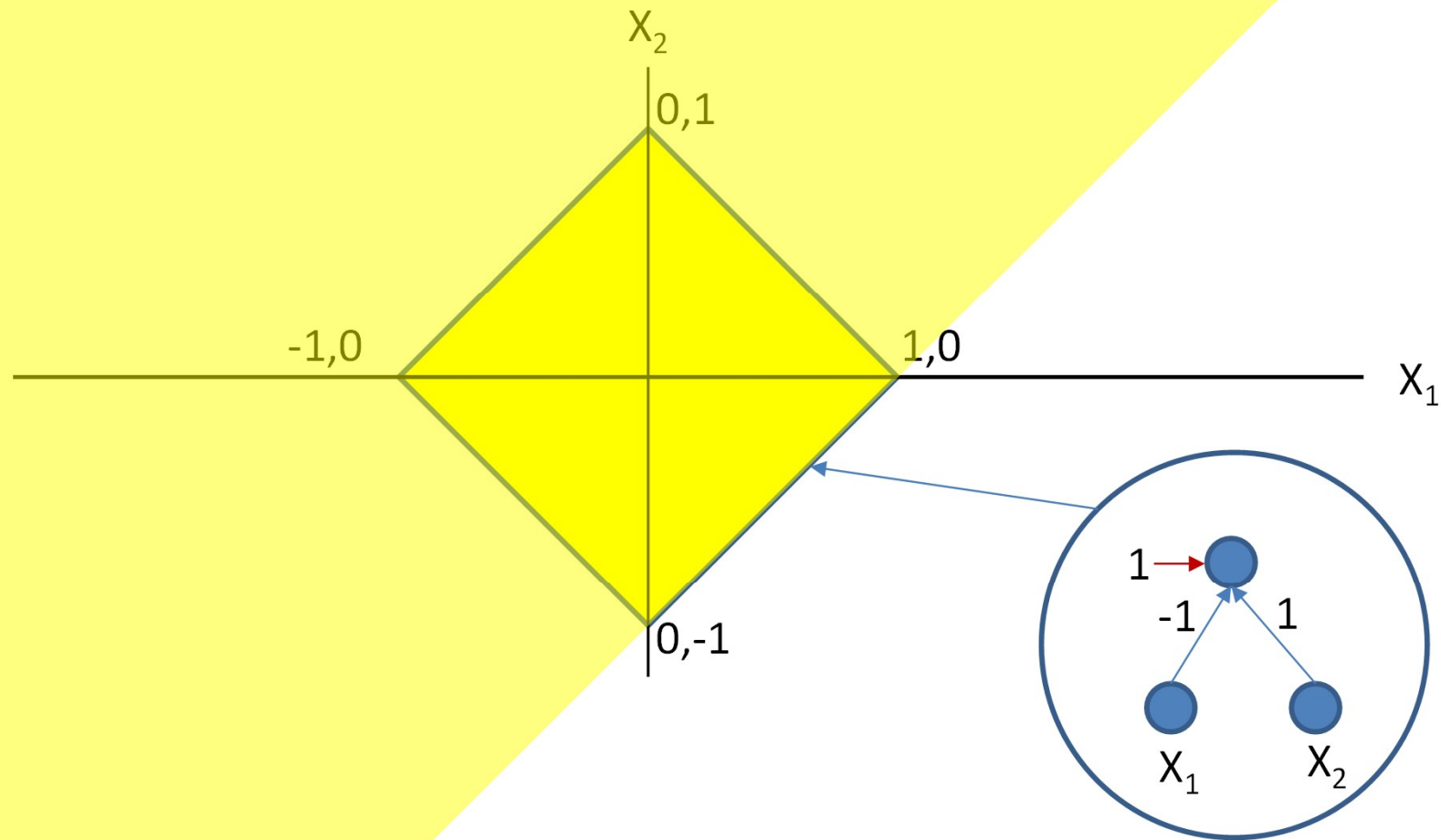
Assuming simple perceptrons:
output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



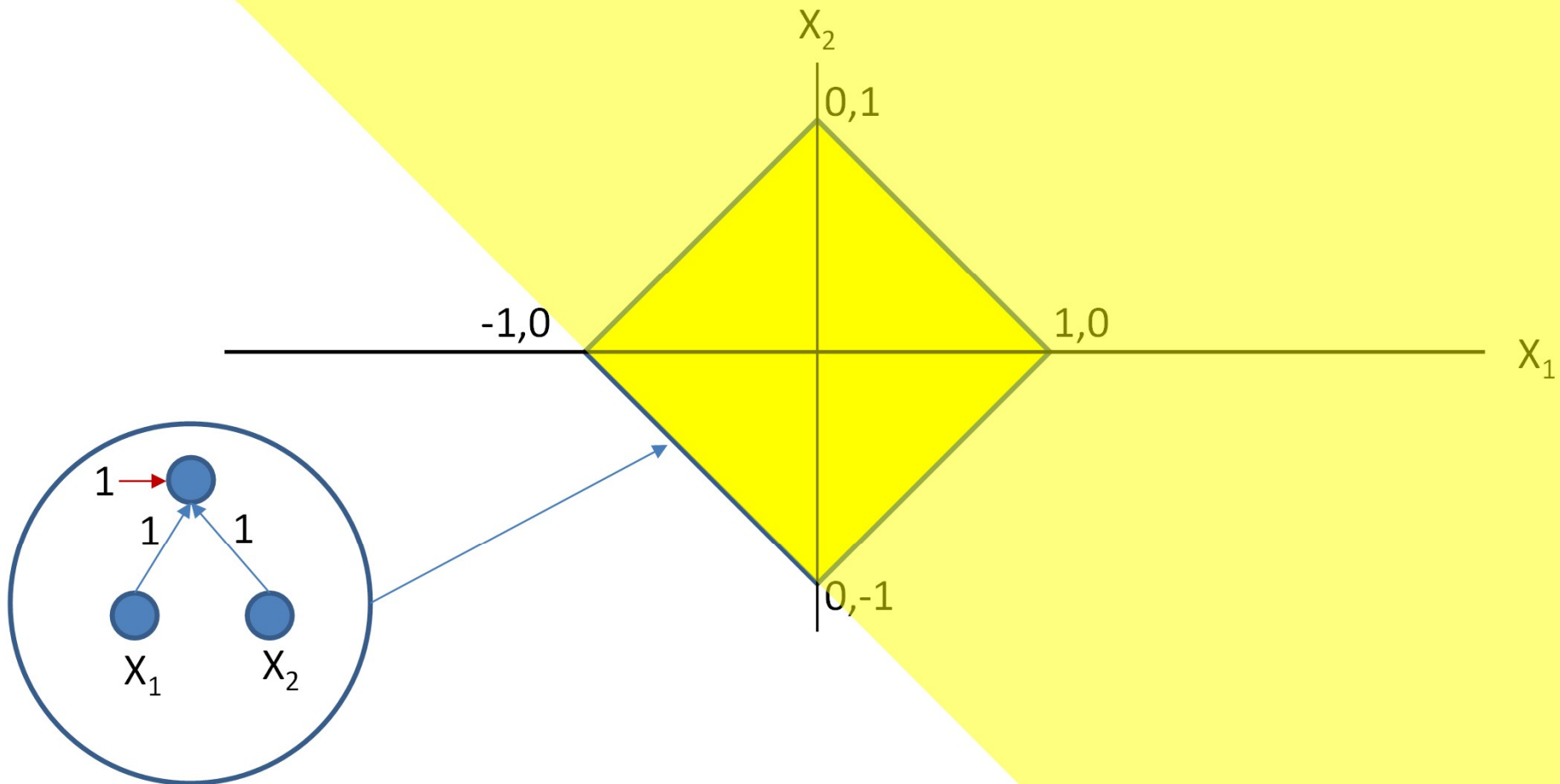
Assuming simple perceptrons:
output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



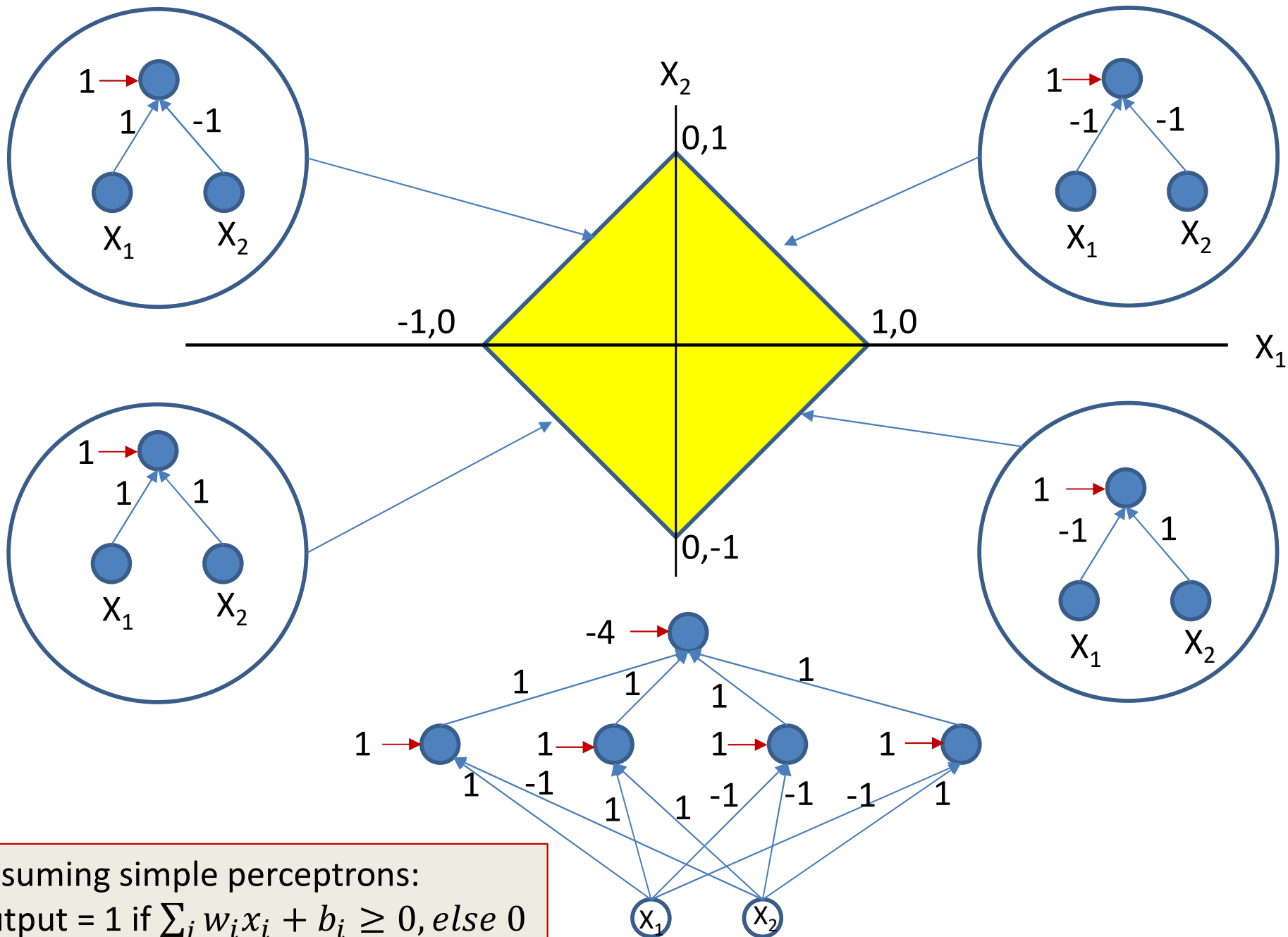
Assuming simple perceptrons:
output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



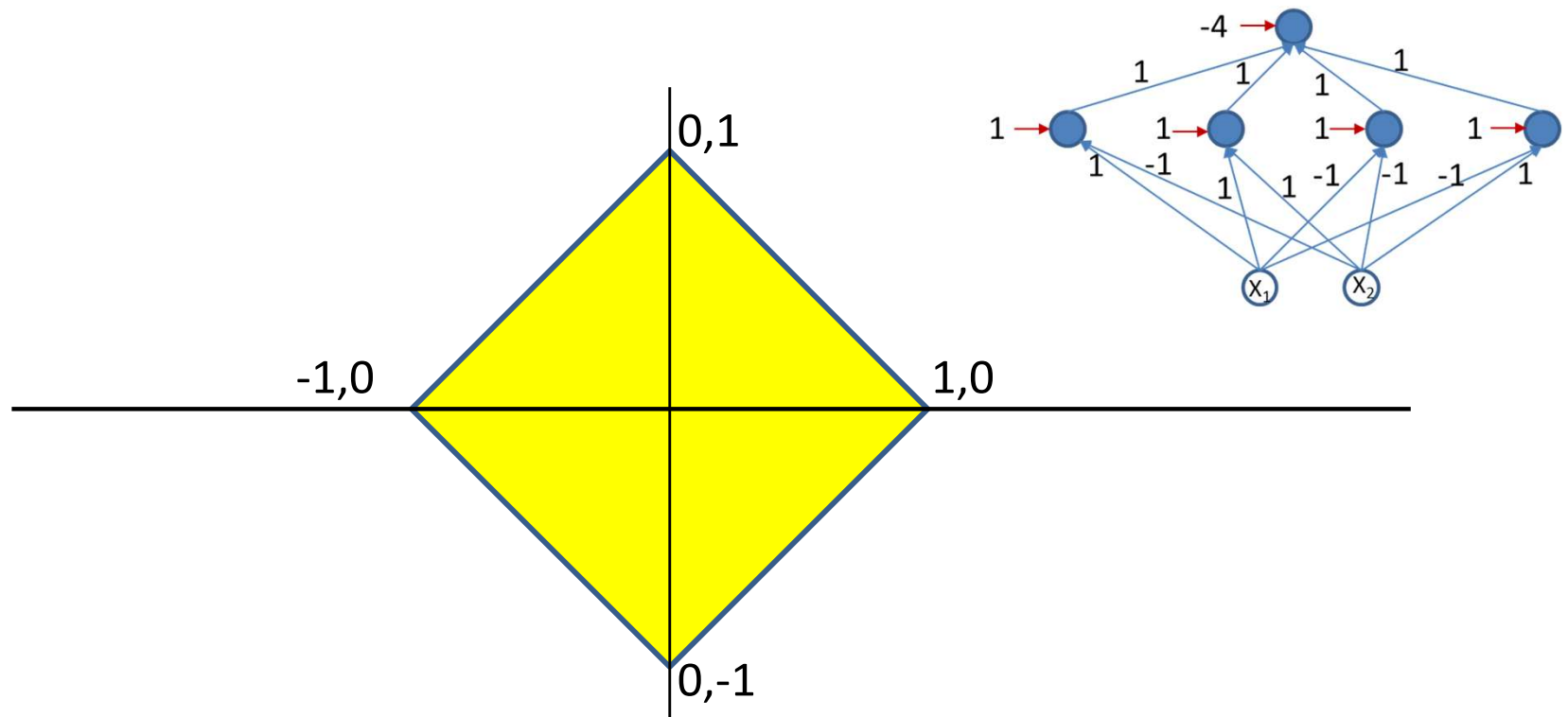
Assuming simple perceptrons:
output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



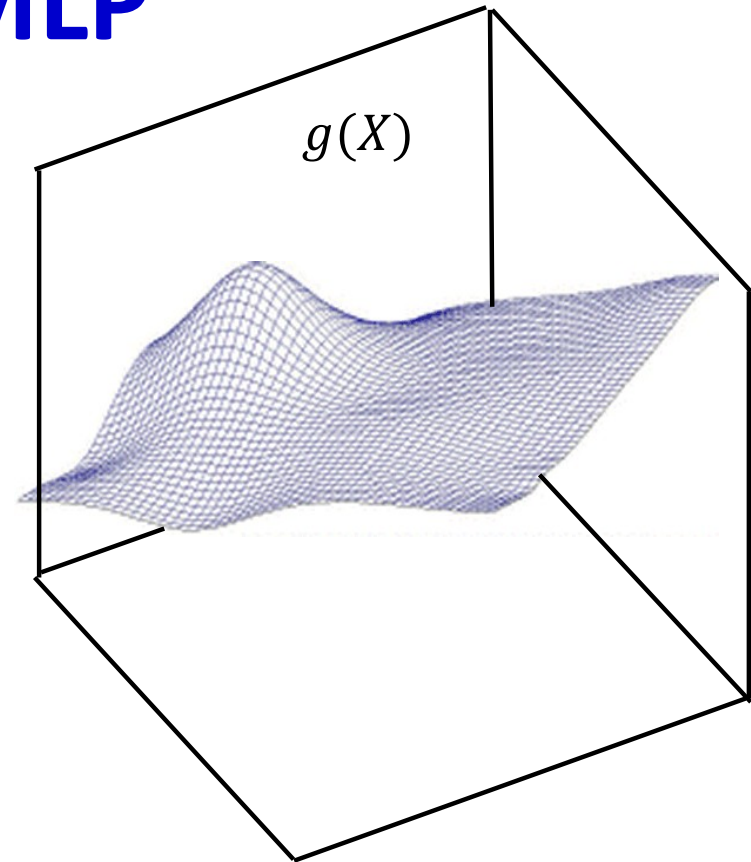
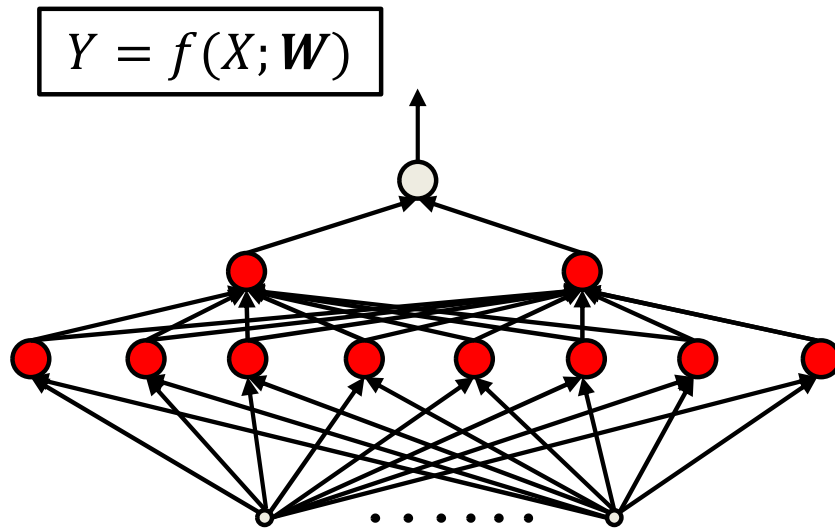
Assuming simple perceptrons:
 output = 1 if $\sum_i w_i x_i + b_i \geq 0$, else 0

Option 1: Construct by hand



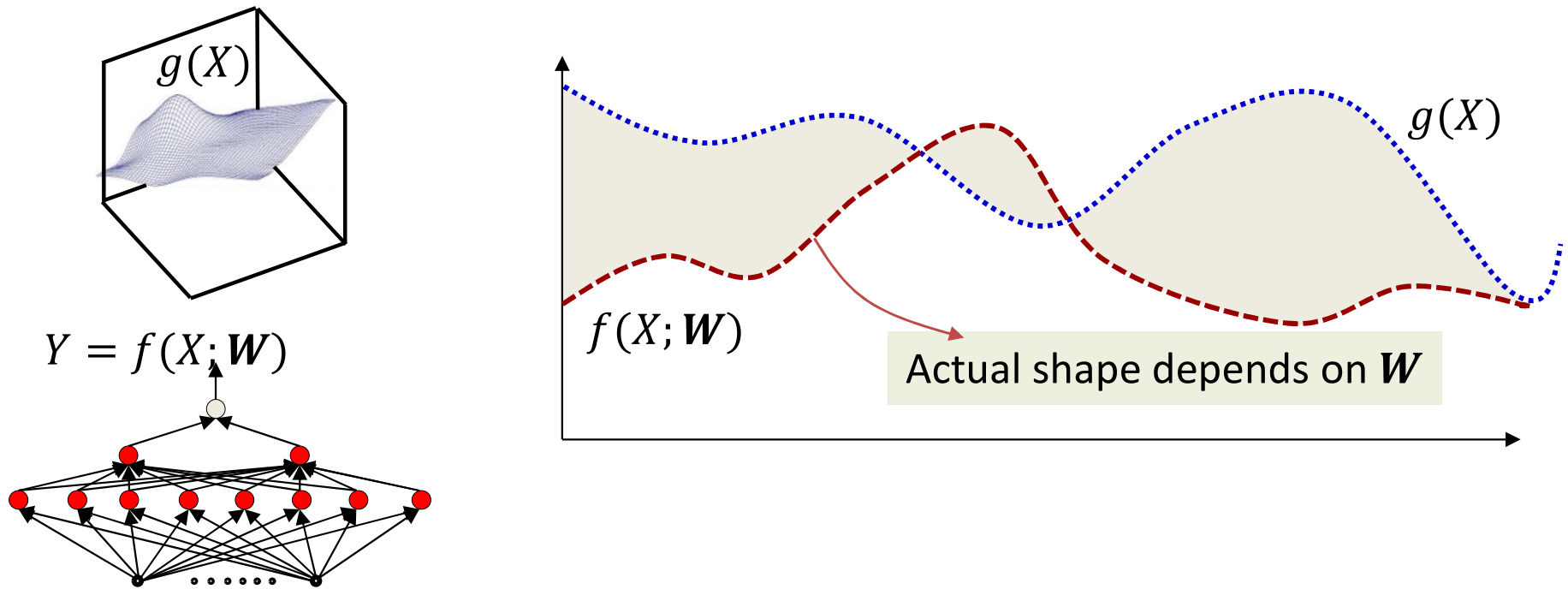
- Given a function, *handcraft* a network to satisfy it
- E.g.: Build an MLP to classify this decision boundary
- Not possible for all but the simplest problems..

Option 2: Automatic estimation of an MLP



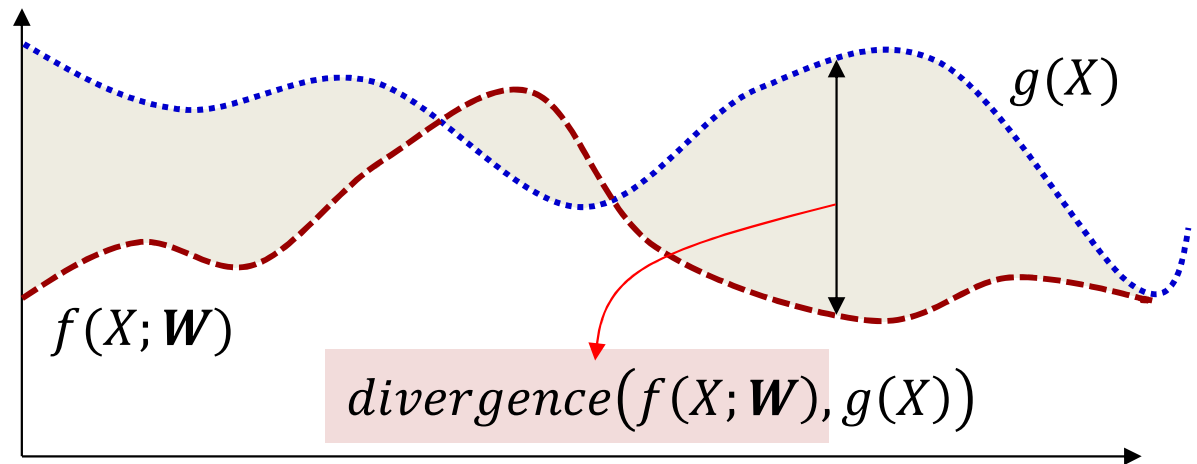
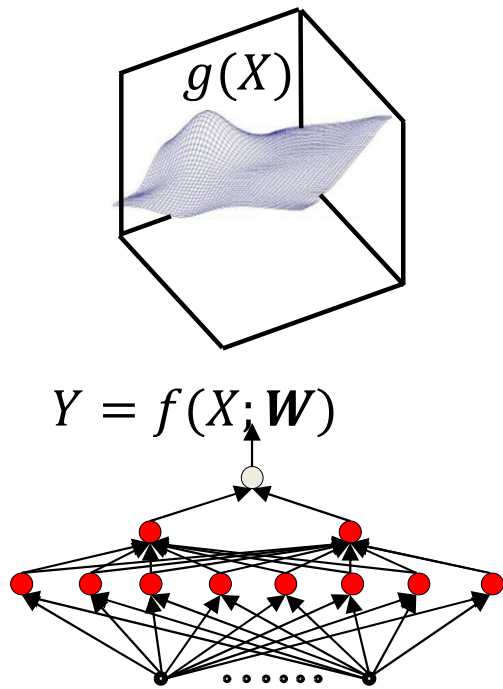
- More generally, *given* the function $g(X)$ to model, we can *derive* the parameters of the network to model it, through computation

How to learn a network?



- Solution: Estimate parameters to minimize the error between the target function $g(X)$ and the network function $f(X; W)$
 - Find the parameter W that minimizes the shaded area

How to learn a network?



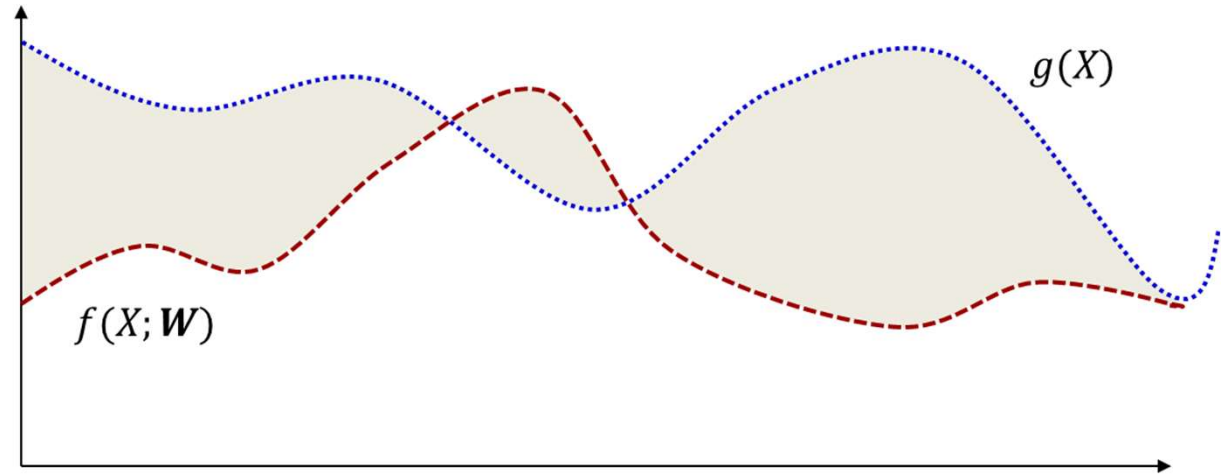
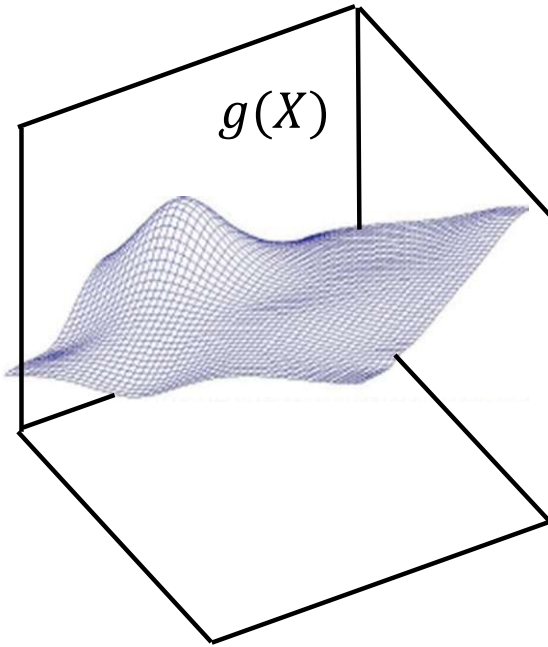
- The shaded area

$$totalerr(\mathbf{W}) = \int_{-\infty}^{\infty} divergence(f(X; \mathbf{W}), g(X)) dX$$

- The optimal $\mathbf{W}$

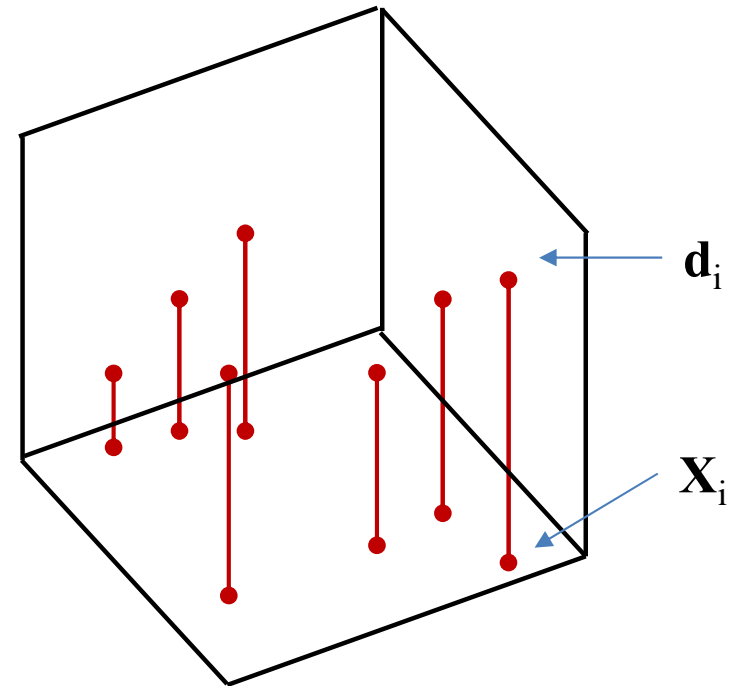
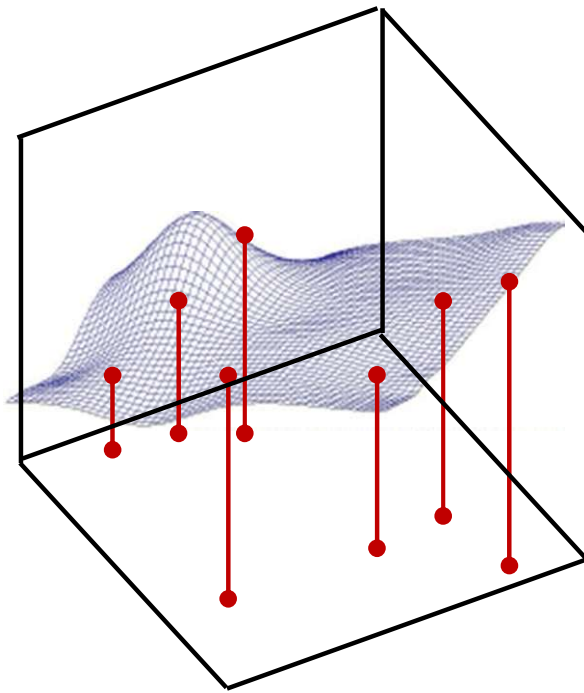
$$\widehat{\mathbf{W}} = \operatorname{argmin}_{\mathbf{W}} totalerr(\mathbf{W})$$

Problem: $g(X)$ is unknown



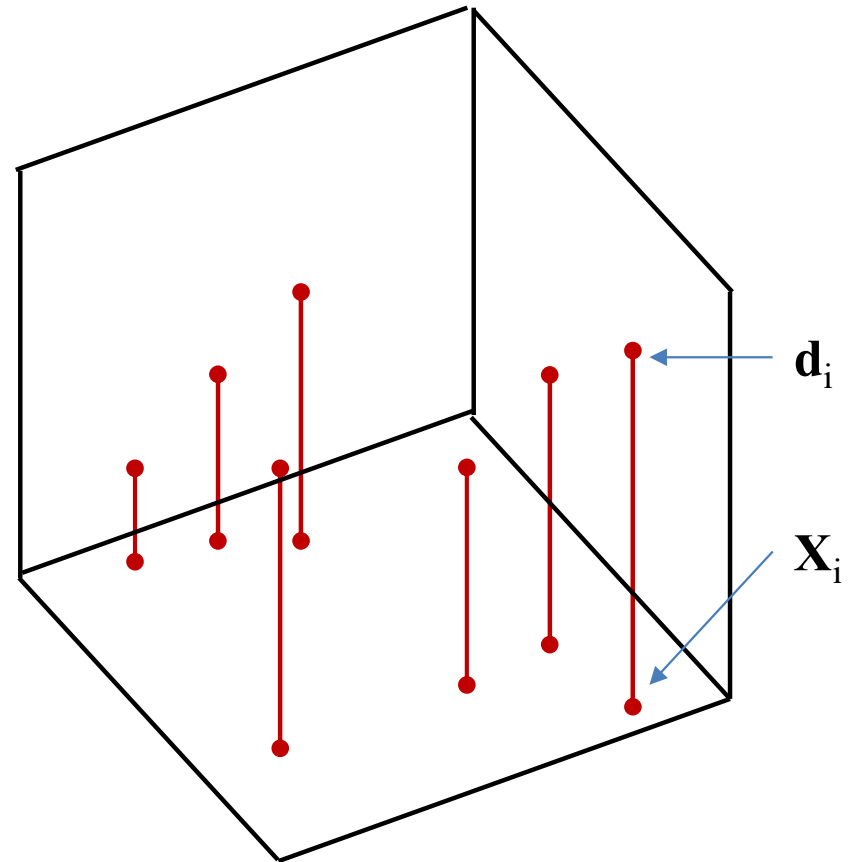
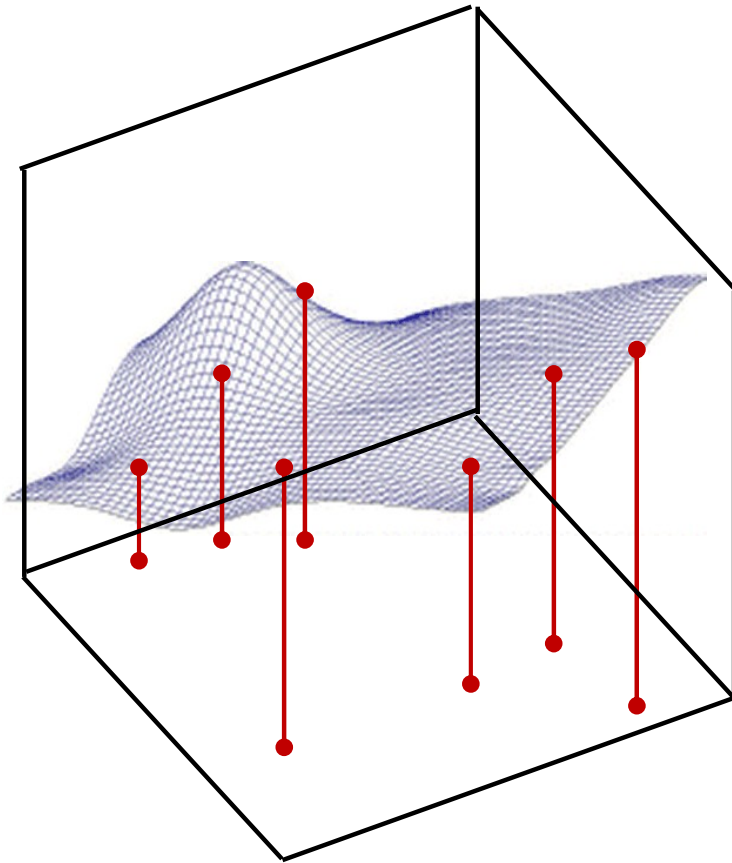
- Function $g(X)$ must be fully specified in order to compute
$$\int_{-\infty}^{\infty} \text{divergence}(f(X; \mathbf{W}), g(X)) dX$$
 - Known *everywhere*, i.e. for *every* input X
- **In practice we will not have such specification**

Sampling the function



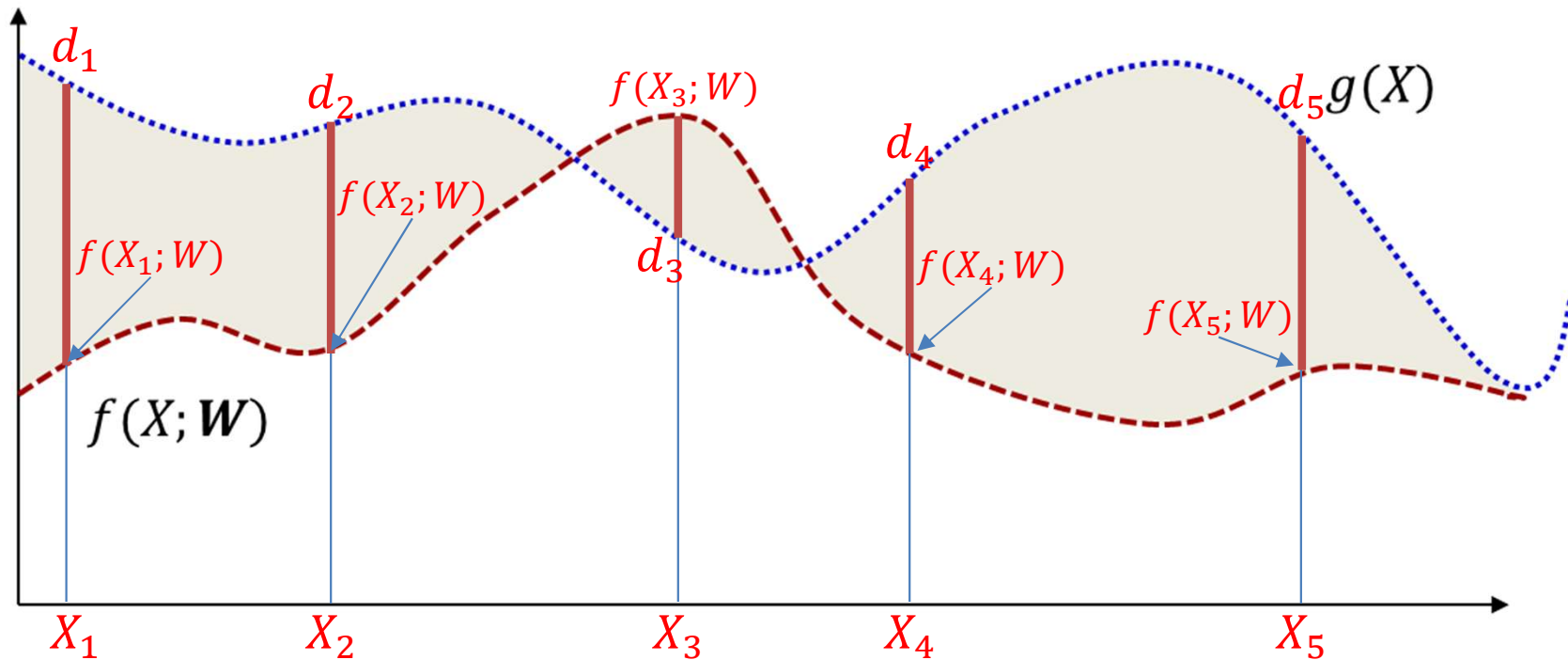
- *Sample $g(X)$*
 - Basically, get input-output pairs for a number of samples of input X_i
 - Many samples (X_i, d_i) , where $d_i = g(X_i) + \text{noise}$
- Very easy to do in most problems: just gather training data
 - E.g. set of images and their class labels
 - E.g. speech recordings and their transcription

Drawing samples



- We must *learn* the *entire* function from these few examples
 - The “training” samples

The *Empirical* error



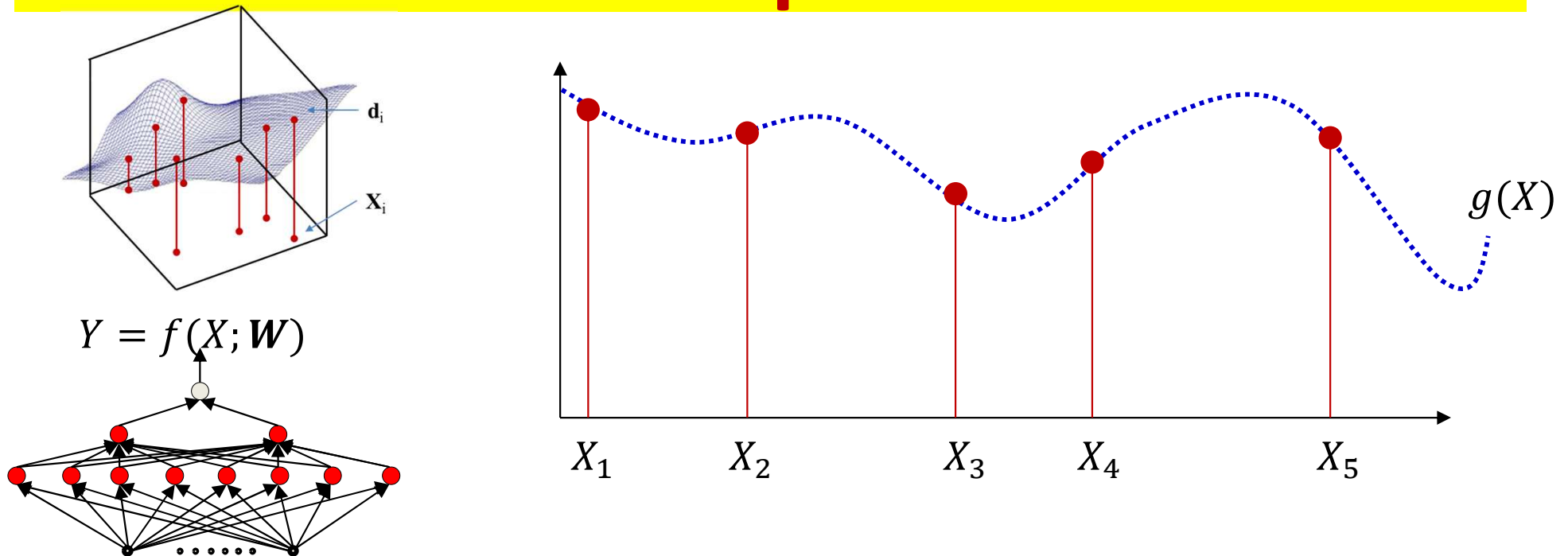
- The *empirical estimate* of the error is the *average* error over the training samples

$$\text{EmpiricalError}(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^N \text{divergence}(f(X_i; \mathbf{W}), d_i)$$

- Estimate network parameters to minimize this average error instead

$$\widehat{\mathbf{W}} = \underset{\mathbf{W}}{\operatorname{argmin}} \text{EmpiricalError}(\mathbf{W})$$

Learning the function from training samples



- Aim: Find the network parameters that “fit” the training points exactly
 $W: EmpiricalError(W) = 0$
 - Assuming network architecture is sufficient for such a fit
 - Assuming unique output d at any X
- And hopefully the resulting function is also correct where we *don't* have training samples

Story so far

- “Learning” a neural network == determining the parameters of the network (weights and biases) required for it to model a desired function
 - The network must have sufficient capacity to model the function
- Ideally, we would like to optimize the network to represent the desired function everywhere
- However, this requires knowledge of the function everywhere
- Instead, we draw “input-output” *training* instances from the function and estimate network parameters to “fit” the input-output relation at these instances
 - And hope it fits the function elsewhere as well

Poll 1

- Since neural networks are universal approximators, any network of any architecture can approximate any function to arbitrary precision (True or False):
 - True
 - False
- Which of the following are true regarding how to compose a network to approximate a given function?
 - The network architecture must have sufficient capacity to model the function
 - The network is actually a parametric function, whose parameters are its weights and biases
 - The parameters must be learned to best approximate the target function
 - The parameters can be perfectly learned from just a few training samples of the target function, even if the actual target function is unknown.

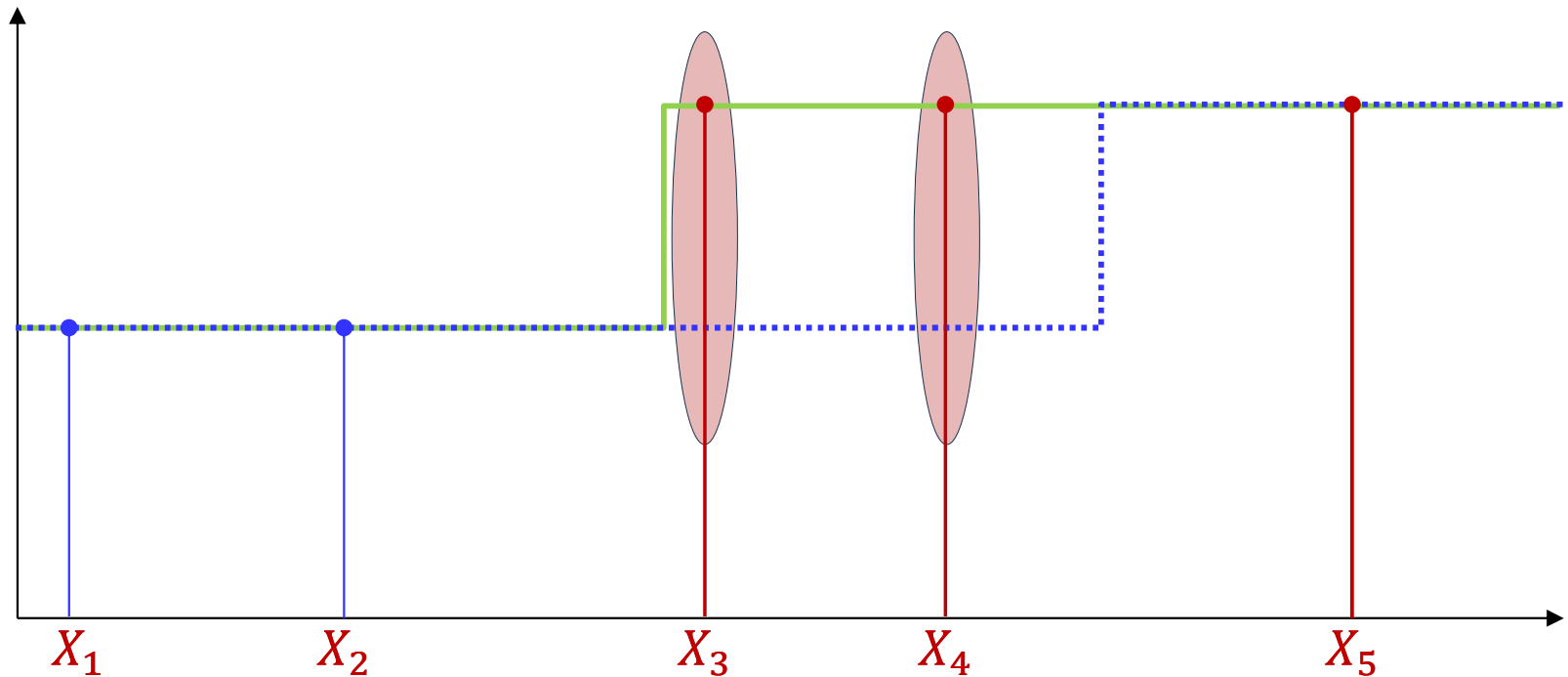
Poll 1

- Since neural networks are universal approximators, any network of any architecture can approximate any function to arbitrary precision (True or False):
 - True
 - **False**
- Which of the following are true regarding how to compose a network to approximate a given function?
 - **The network architecture must have sufficient capacity to model the function**
 - **The network is actually a parametric function, whose parameters are its weights and biases**
 - **The parameters must be learned to best approximate the target function**
 - The parameters can be perfectly learned from just a few training samples of the target function, even if the actual target function is unknown.

Let's begin with a simple task

- Learning a *classifier*
 - Simpler than regressions
- This was among the earliest problems addressed using MLPs
- Specifically, consider *binary* classification
 - Generalizes to multi-class

The *Empirical Classification* error

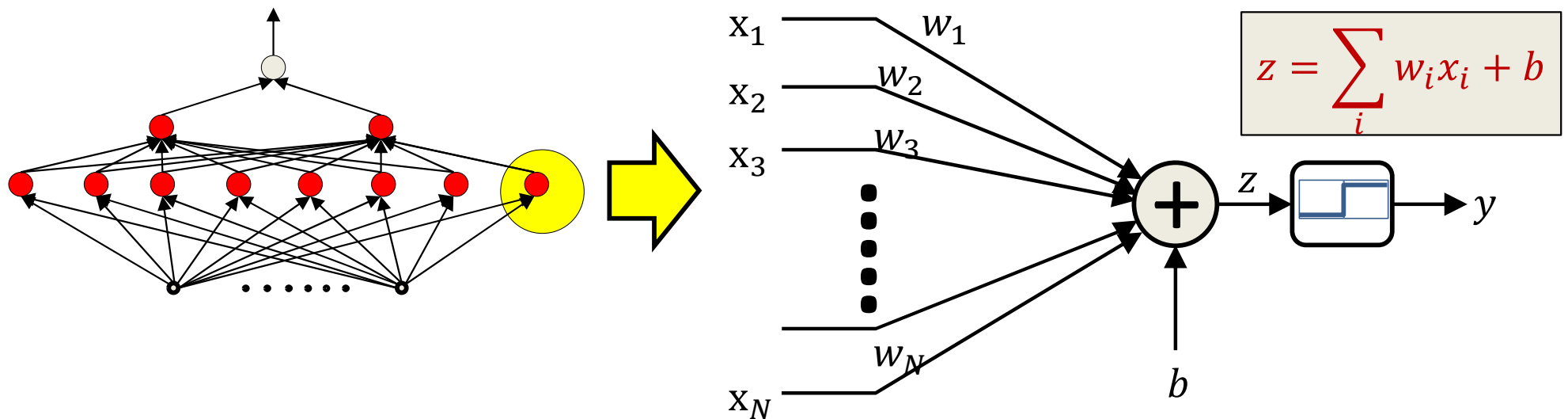


- The obvious error metric in a classifier is binary
 - The classifier is either right (error=0) or wrong (error=1)
 - Either $f(X; \mathbf{W}) = d$, or $f(X; \mathbf{W}) \neq d$

$$\text{EmpiricalError}(\mathbf{W}) = \frac{1}{N} \sum_{i=1}^N 1(f(X_i; \mathbf{W}) \neq d_i)$$

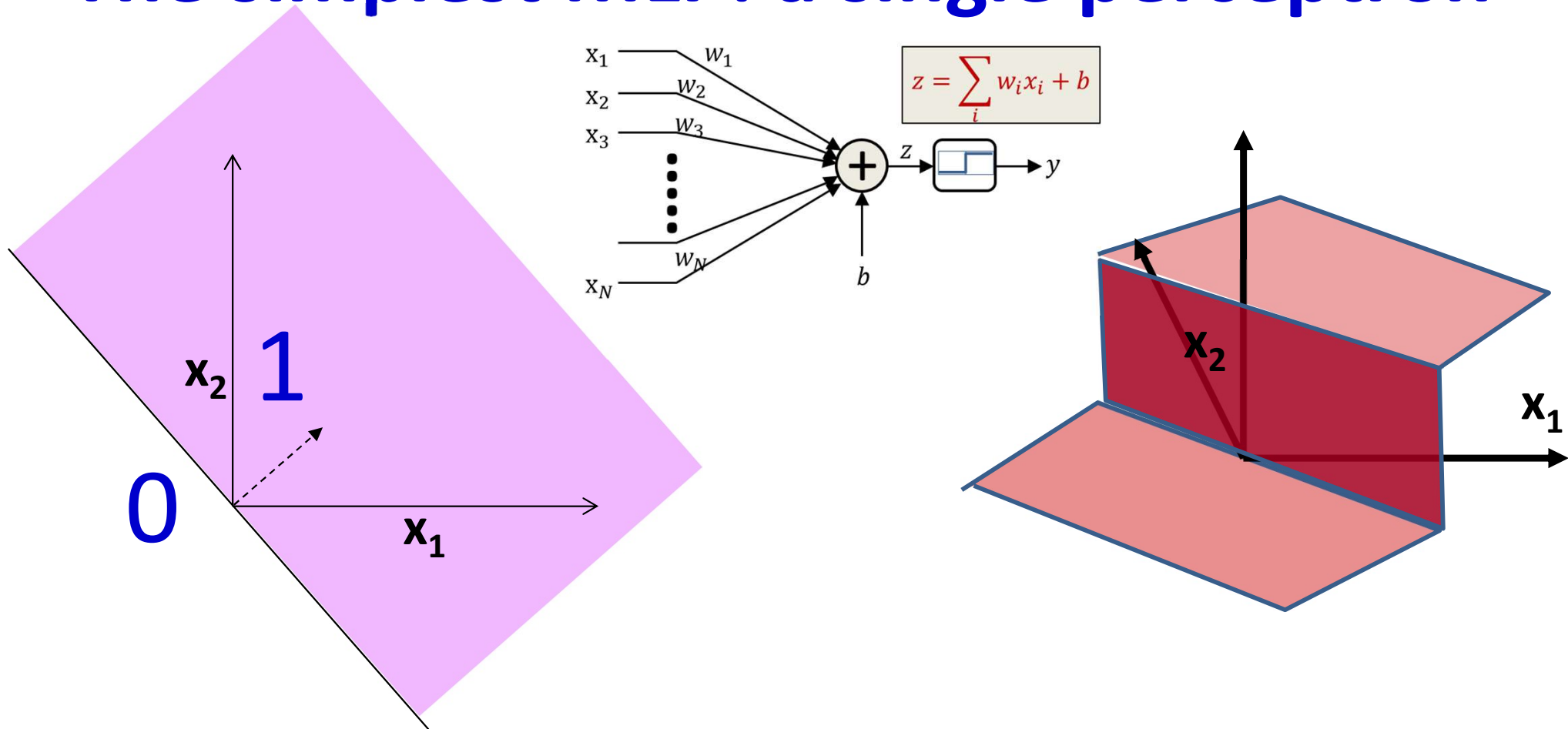
- **Learning the classifier:** Minimizing the count of misclassifications

History: The original MLP



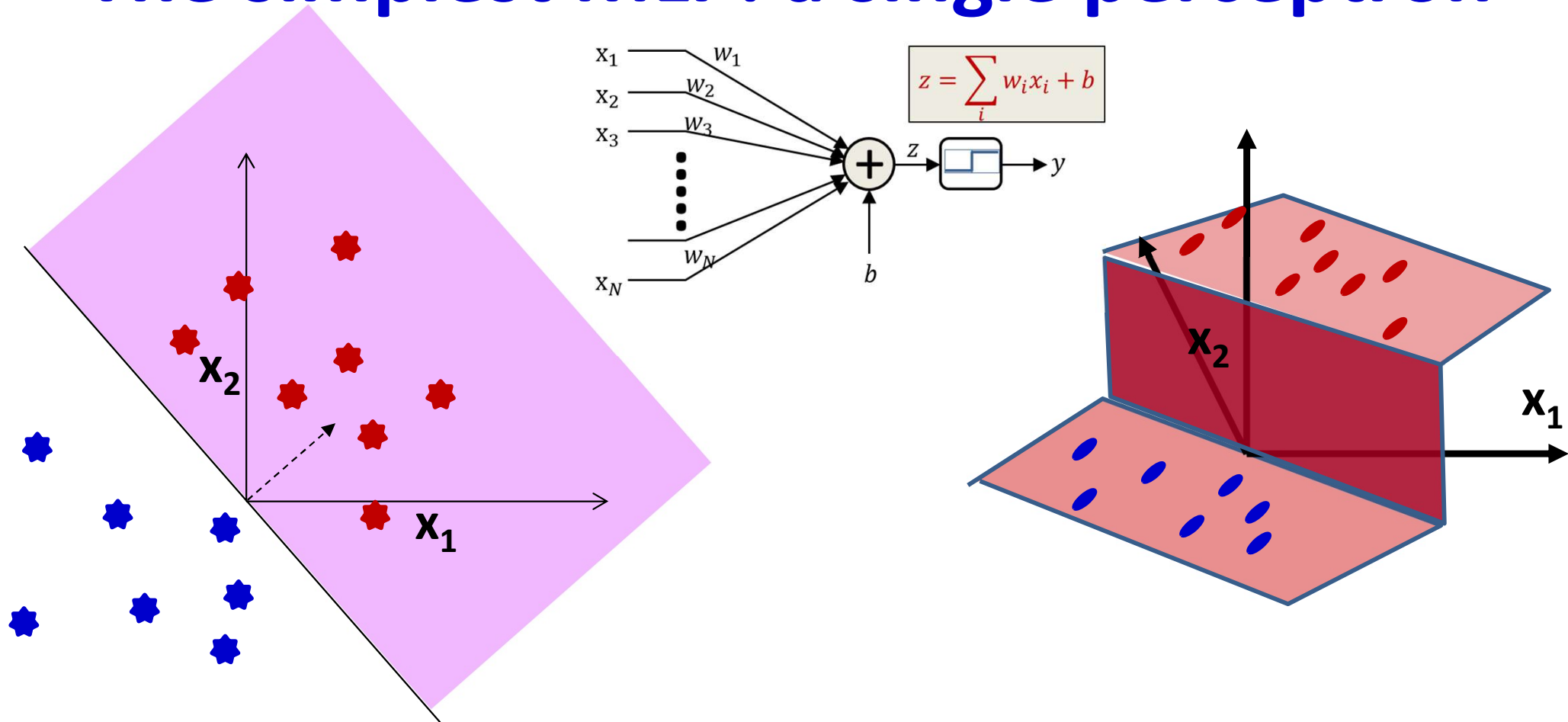
- The original MLP as proposed by Rosenblatt: a network of threshold units
 - But how do you train it?
 - Given only “training” instances of input-output pairs

The simplest MLP: a single perceptron



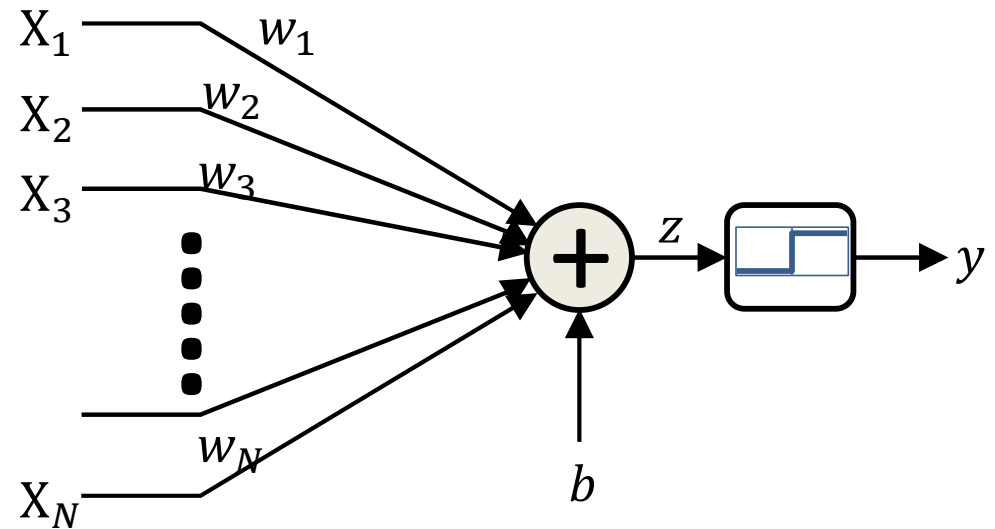
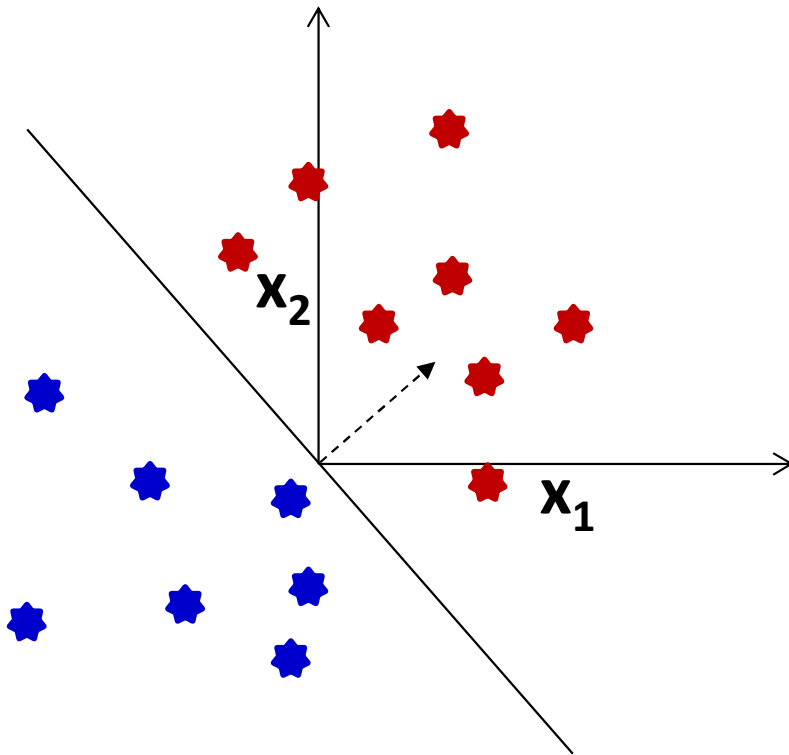
- Learn this function
 - **A step function across a hyperplane**

The simplest MLP: a single perceptron



- Learn this function
 - A step function across a hyperplane
 - Given only samples from it

Learning the perceptron



- Given a number of input output pairs, learn the weights and bias

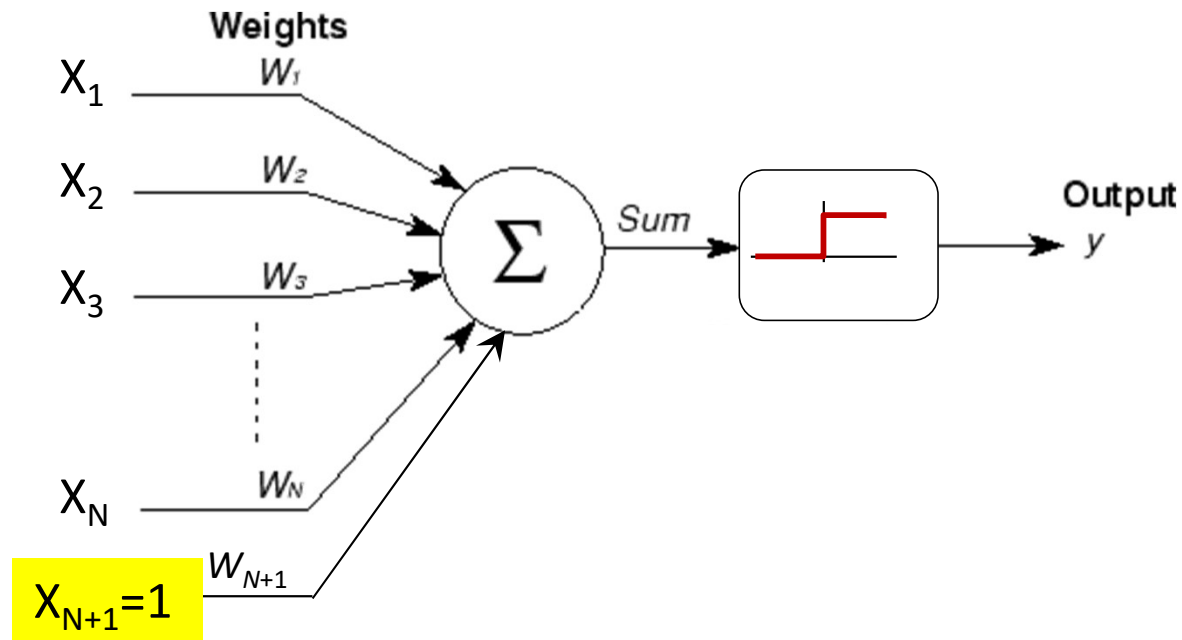
$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^N w_i X_i + b \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

$$\text{Boundary: } \sum_{i=1}^N w_i X_i + b = 0$$

- Learn $W = [w_1 \dots w_N]^T$ and b , given several (X, y) pairs

- $X = [X_1 \dots X_N]^T$

Restating the perceptron



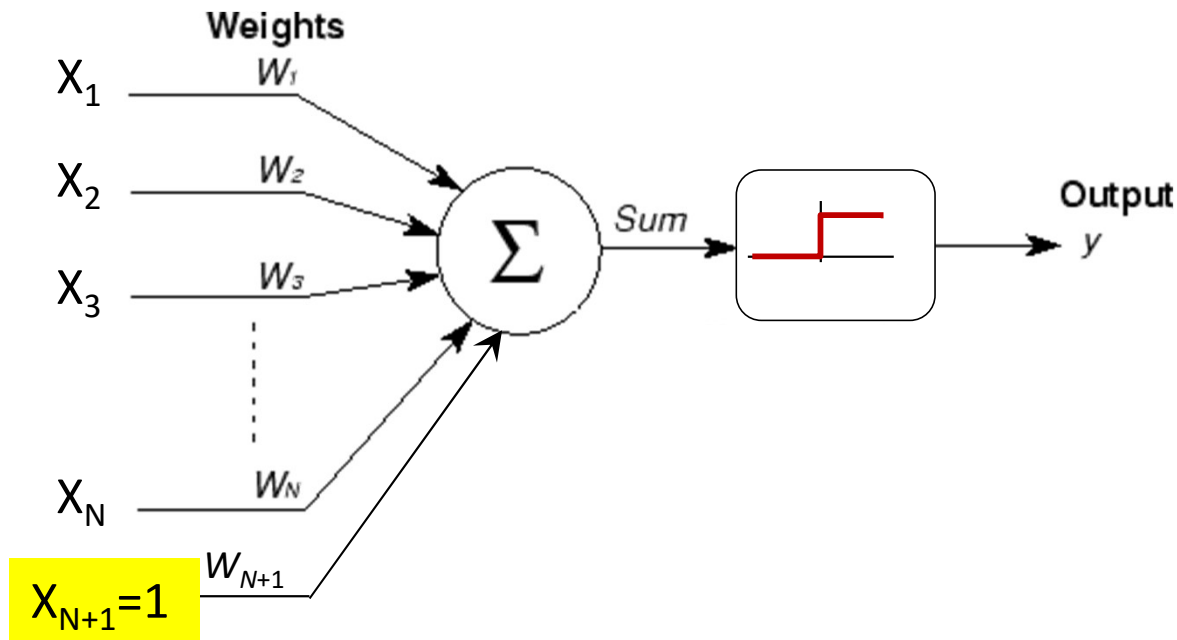
- Restating the perceptron equation by adding another dimension to X

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{N+1} w_i X_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

where $X_{N+1} = 1$

- Note that the boundary $\sum_{i=1}^{N+1} w_i X_i = 0$ is now a hyperplane through origin

Restating the perceptron



- Restating the perceptron equation by adding another dimension to X

$$y = \begin{cases} 1 & \text{if } \sum_{i=1}^{N+1} w_i X_i \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

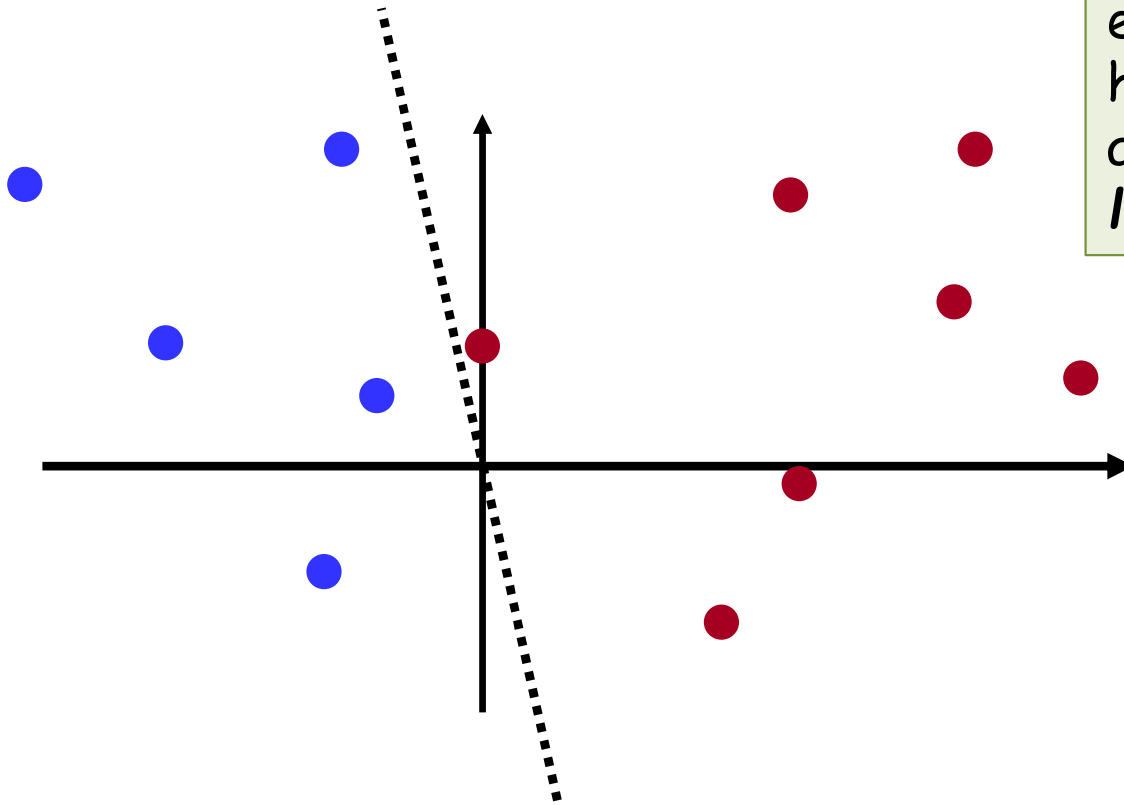
where $X_{N+1} = 1$

How did we make a hyperplane that did not pass through the origin suddenly go through the origin by adding a dimension?

- Note that the boundary $\sum_{i=1}^{N+1} w_i X_i = 0$ is now a hyperplane through origin

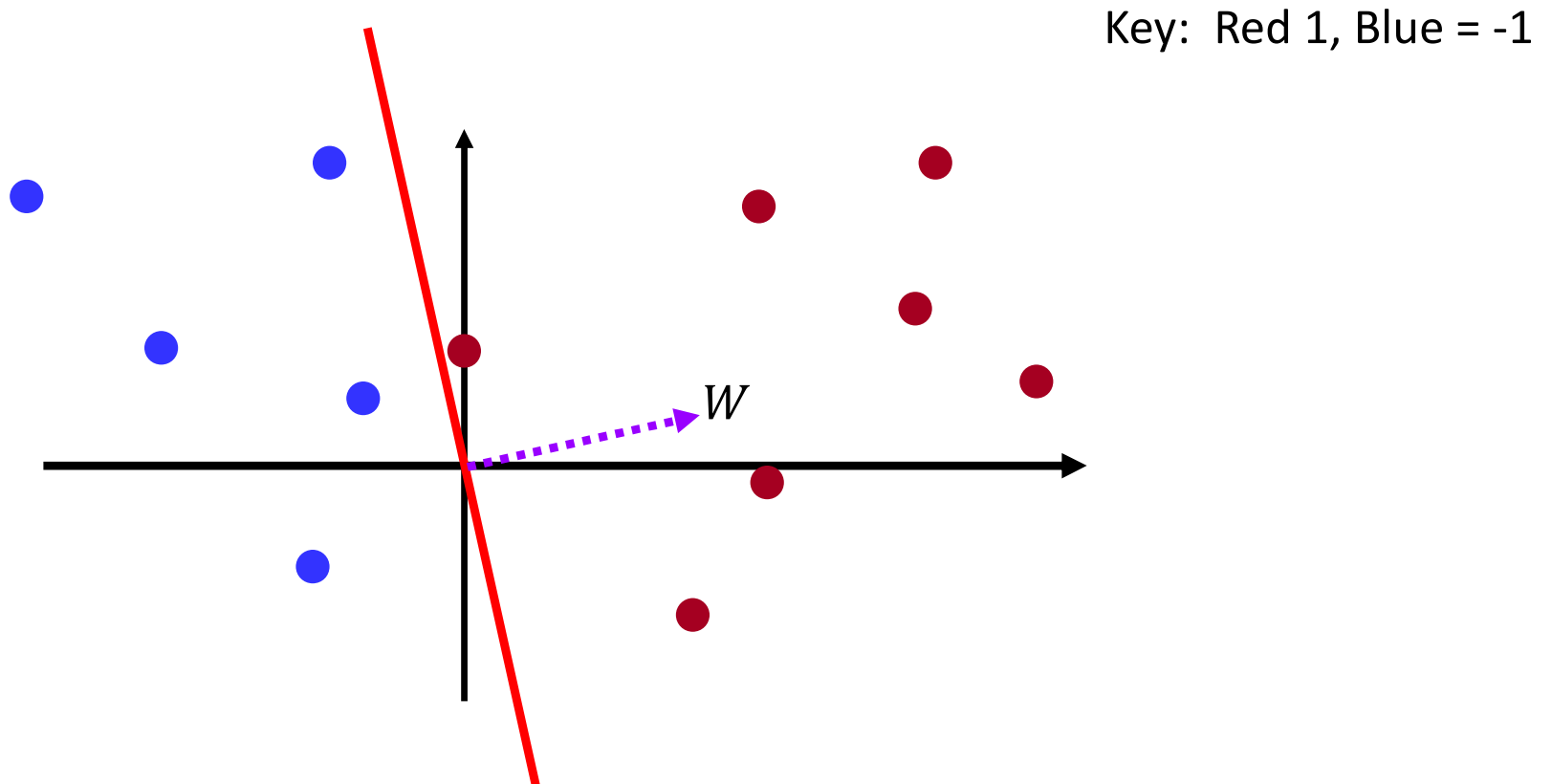
The Perceptron Problem

If the classes can be exactly separated by a hyperplane, the classes are said to be *linearly separable*



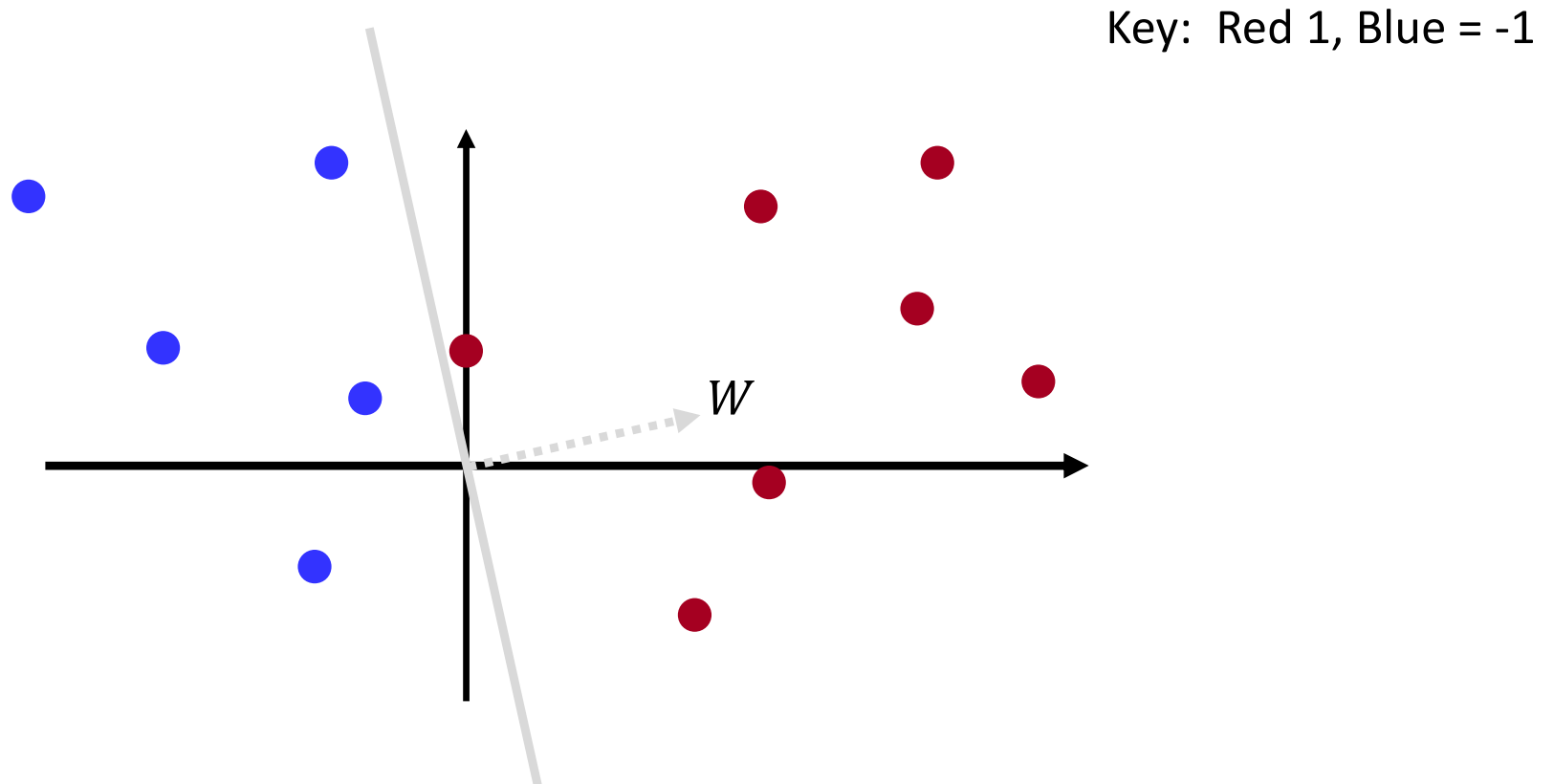
- Find the hyperplane $\sum_{i=1}^{N+1} w_i X_i = 0$ that perfectly separates the two groups of points

The Perceptron Problem



- Find the hyperplane $\sum_{i=1}^{N+1} w_i X_i = 0$ that perfectly separates the two groups of points
 - Let vector $W = [w_1, w_2, \dots, w_{N+1}]^T$ and vector $X = [x_1, x_2, \dots, x_N, 1]^T$
 - $\sum_{i=1}^{N+1} w_i X_i = W^T X$ is an inner product
 - $W^T X = 0$ is the hyperplane comprising all X s orthogonal to vector W
 - Learning the perceptron = finding the weight vector W for the separating hyperplane
 - W points in the direction of the positive class

The Perceptron Problem



- Learning the perceptron: Find the weights vector W such that the plane described by $W^T X = 0$ perfectly separates the classes
 - $W^T X$ is positive ($W^T X > 0$) for all red dots
 - The angle between W and positive-class vectors is less than 90
 - $W^T X$ is negative ($W^T X < 0$) for all blue dots
 - The angle between W and negative-class vectors is greater than 90

The *online* perceptron solution

- The popular solution, originally proposed by Rosenblatt is an *online* algorithm
 - The famous “perceptron” algorithm
- Initializes W and incrementally updates it each time we encounter an instance that is incorrectly classified
 - Guaranteed to find the correct solution for linearly separable data

Perceptron Algorithm: Summary

- Cycle through the training instances
- Only update W on misclassified instances
- If instance misclassified:
 - If instance is positive class (positive misclassified as negative)

$$W = W + X_i$$

- If instance is negative class (negative misclassified as positive)

$$W = W - X_i$$

Perceptron Learning Algorithm

- Given N training instances $(X_1, y_1), (X_2, y_2), \dots, (X_N, y_N)$
 - $y_i = +1$ or -1

Using a +1/-1 representation for classes to simplify notation

- Initialize W
- Cycle through the training instances:
- do

- For $i = 1 \dots N_{train}$

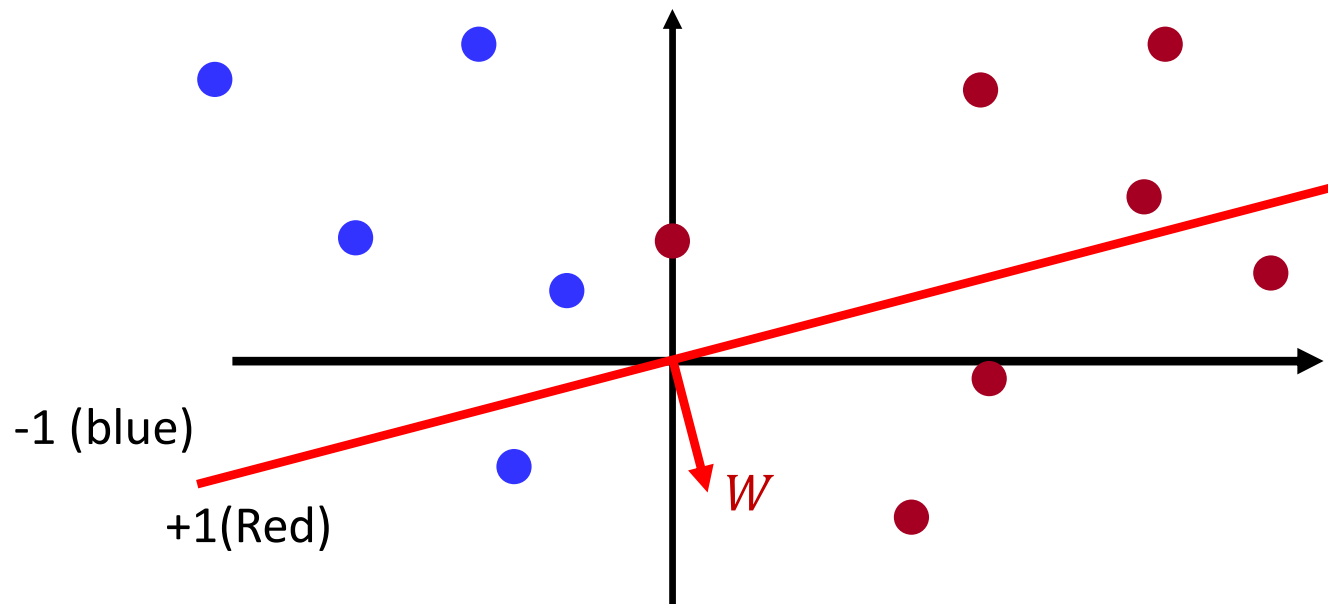
$$O(X_i) = \text{sign}(W^T X_i)$$

- If $O(X_i) \neq y_i$

$$W = W + y_i X_i$$

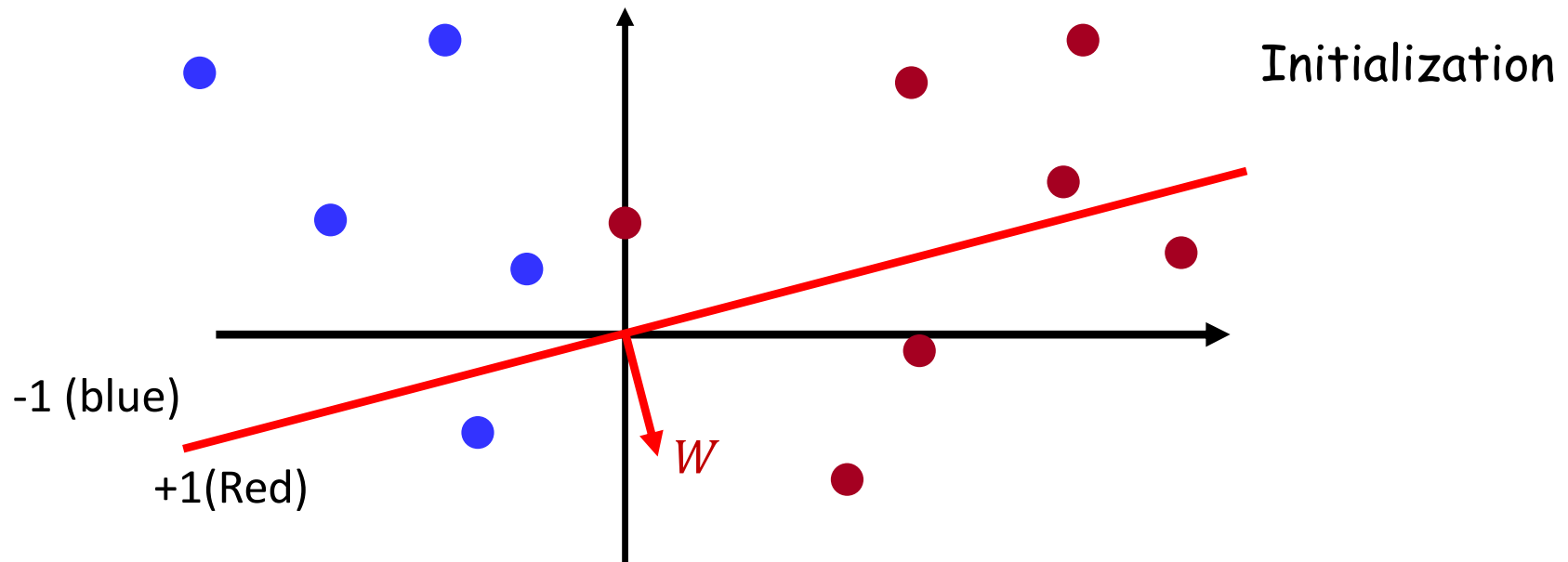
- until no more classification errors

A Simple Method: The Perceptron Algorithm

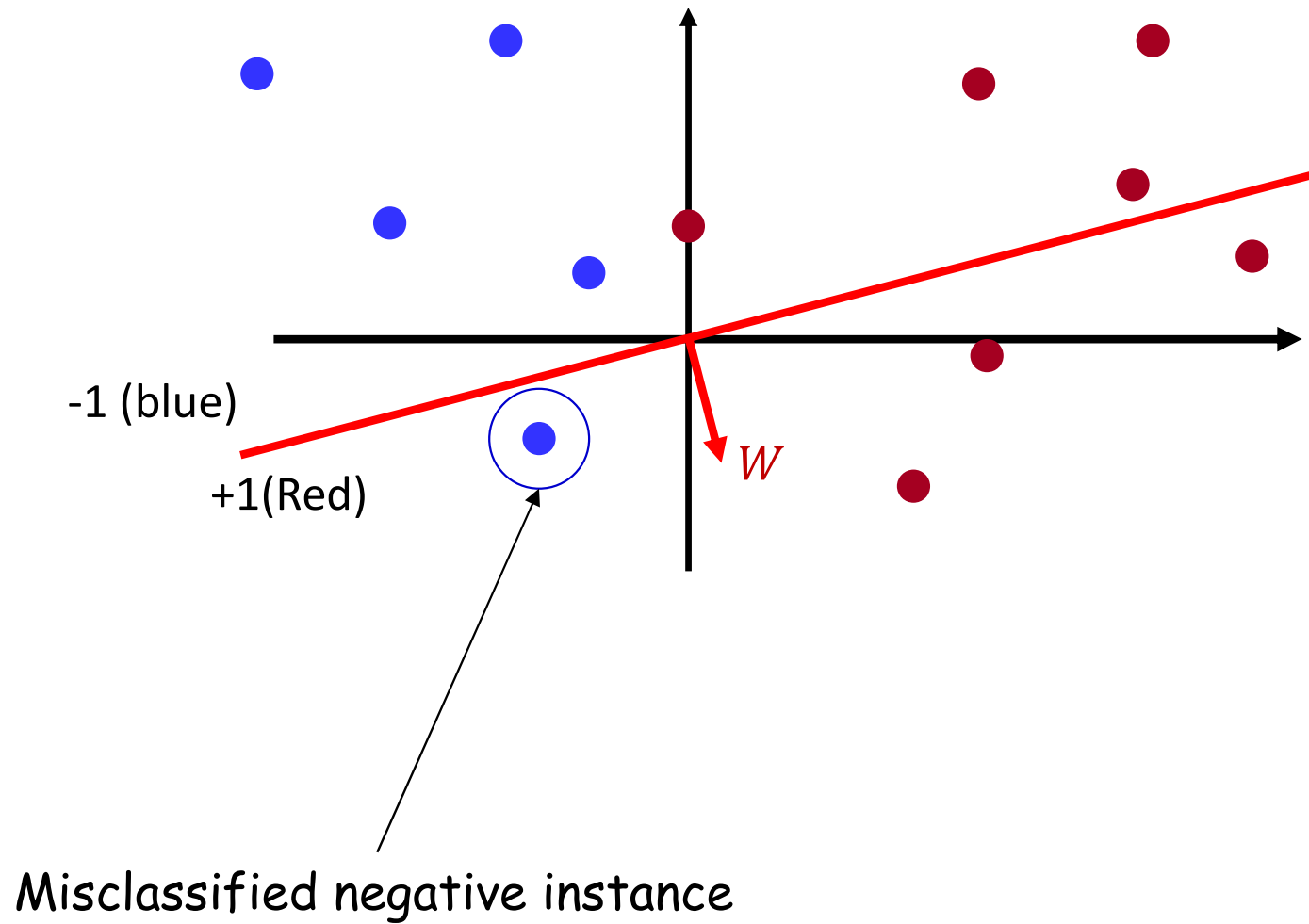


- **Initialize:** Randomly initialize the hyperplane
 - I.e. randomly initialize the normal vector W
- **Classification rule** $\text{sign}(W^T X)$
 - Vectors on the same side of the hyperplane as W will be assigned +1 class, and those on the other side will be assigned -1
- The random initial plane will make mistakes

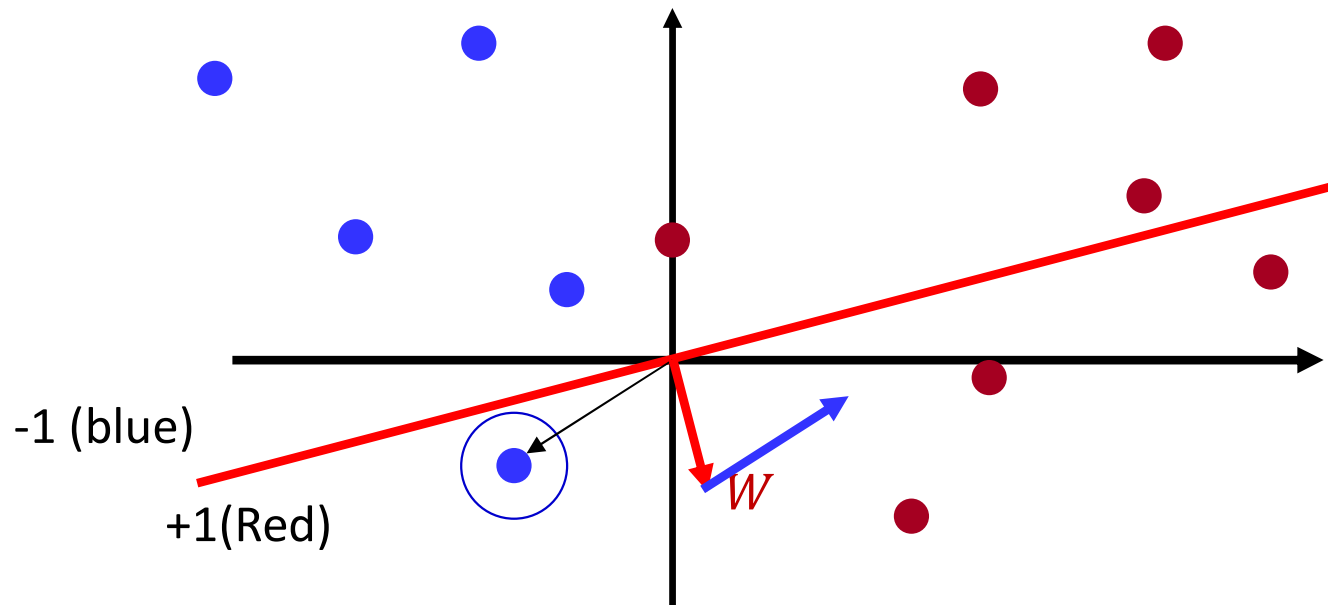
Perceptron Algorithm



Perceptron Algorithm

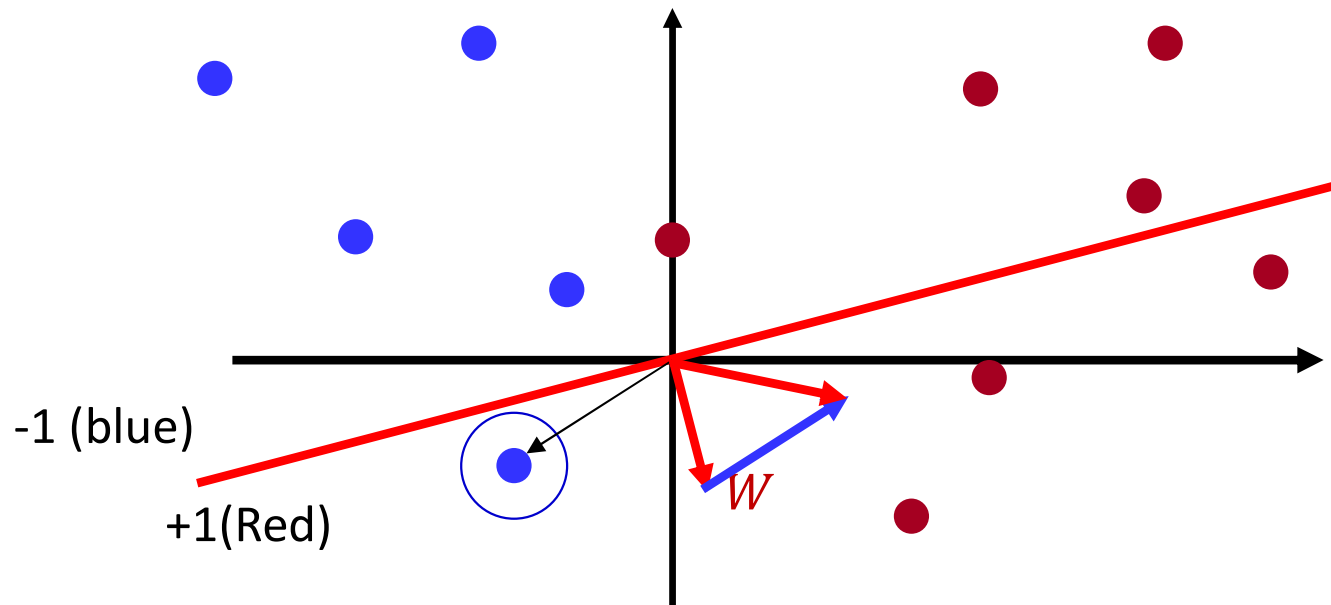


Perceptron Algorithm



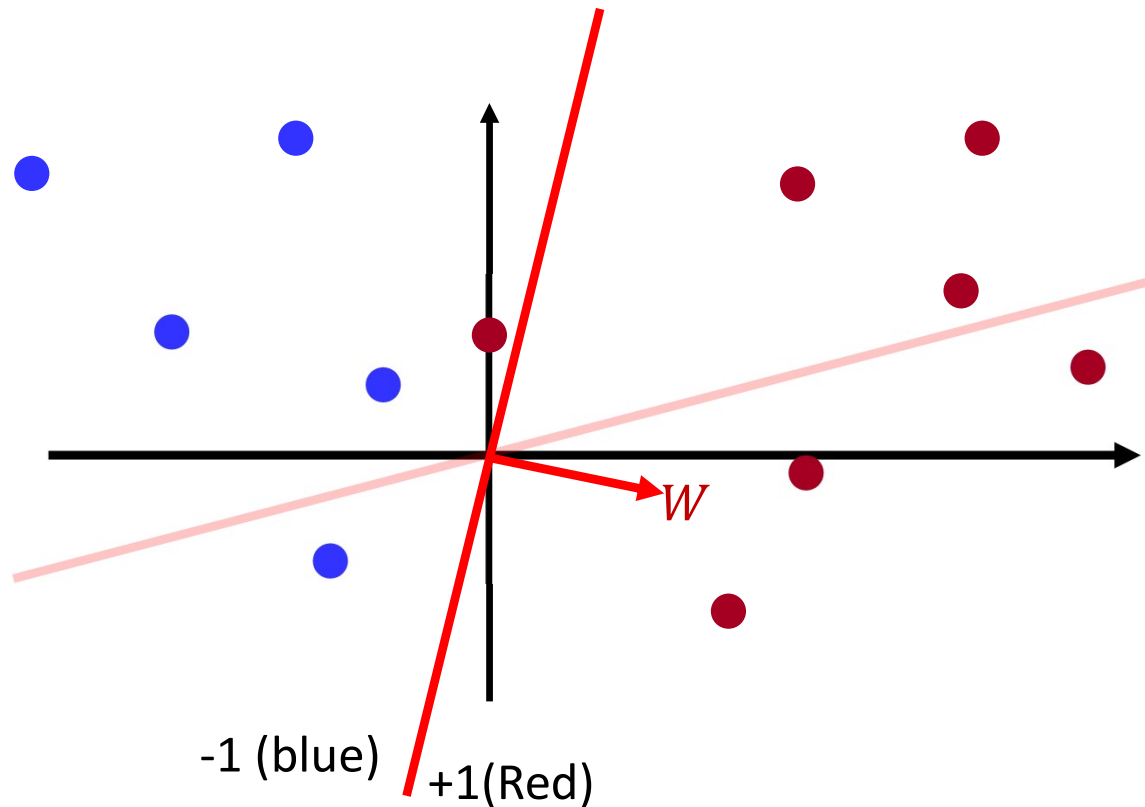
Misclassified *negative* instance, *subtract* it from W

Perceptron Algorithm



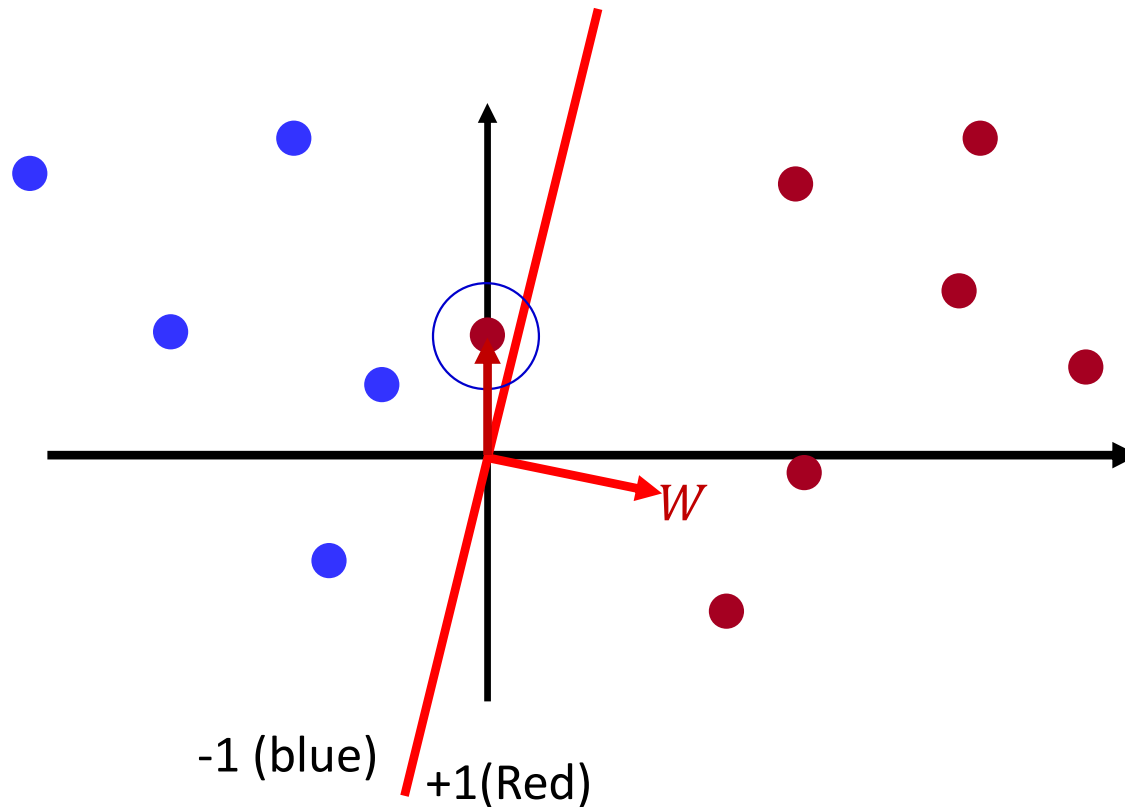
The new weight

Perceptron Algorithm



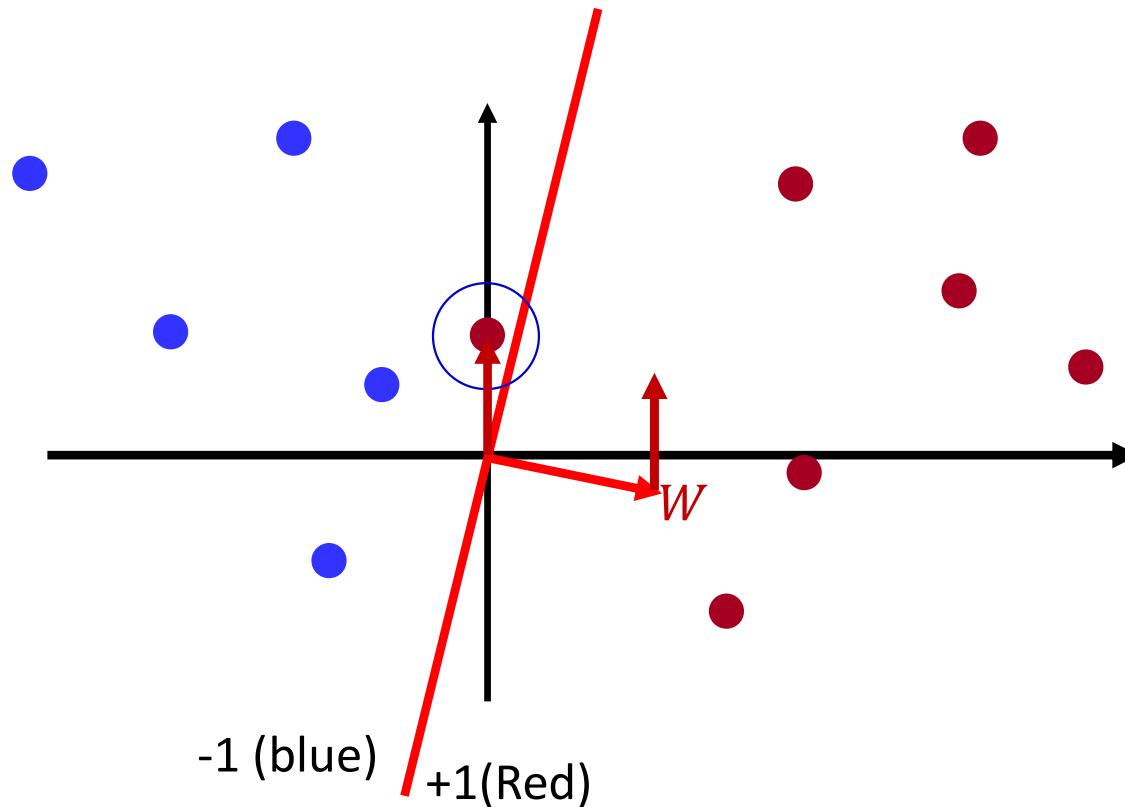
The new weight (and boundary)

Perceptron Algorithm



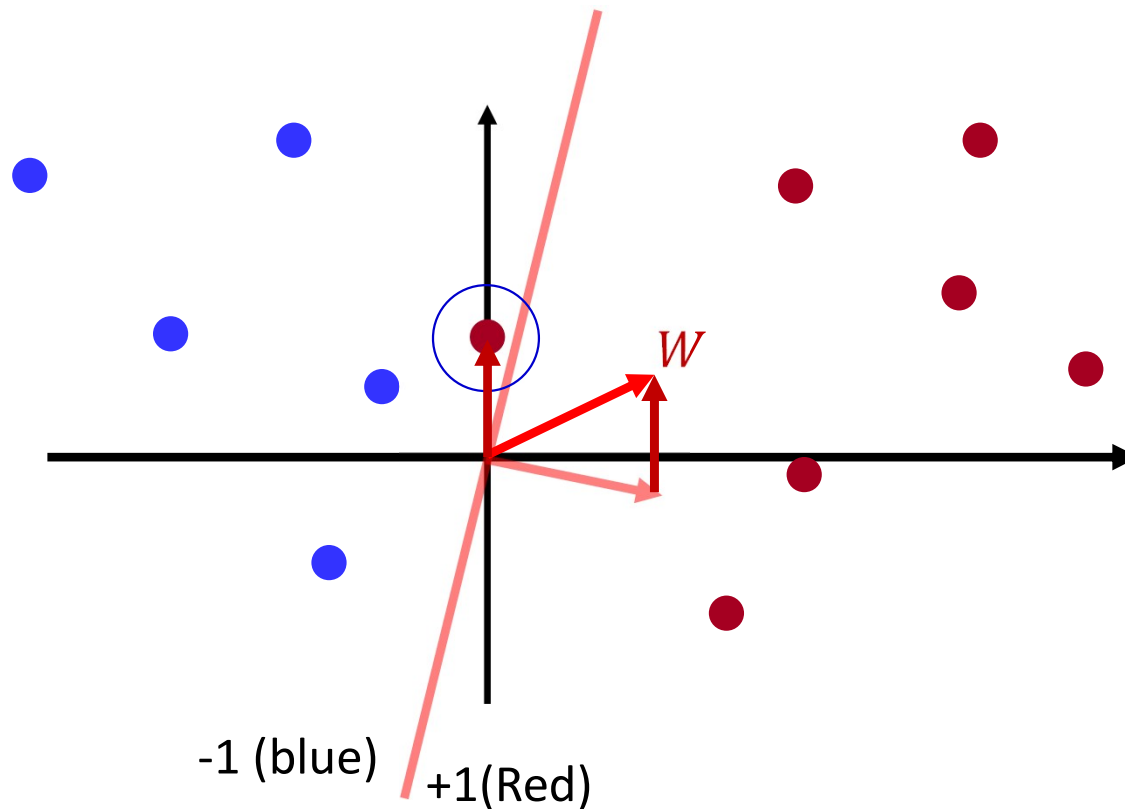
Misclassified *positive* instance

Perceptron Algorithm



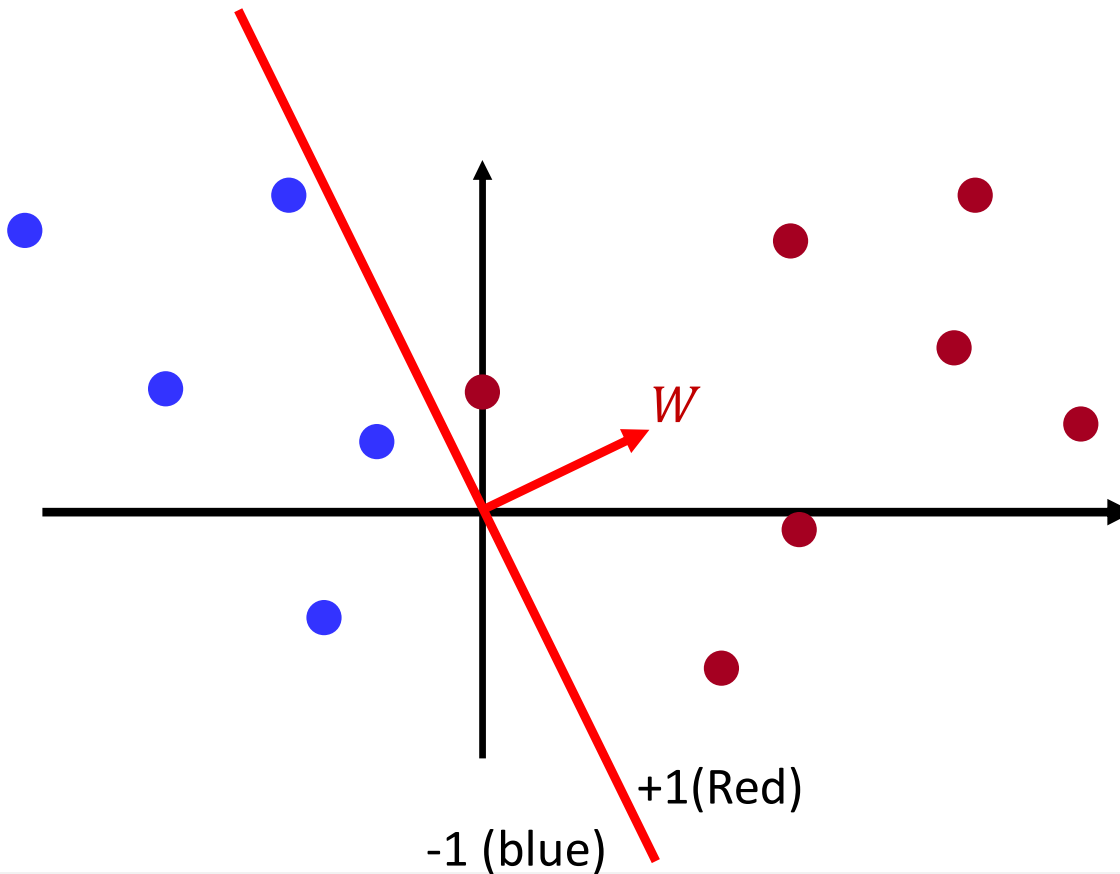
Misclassified *positive* instance, *add* it to W

Perceptron Algorithm



The new weight vector

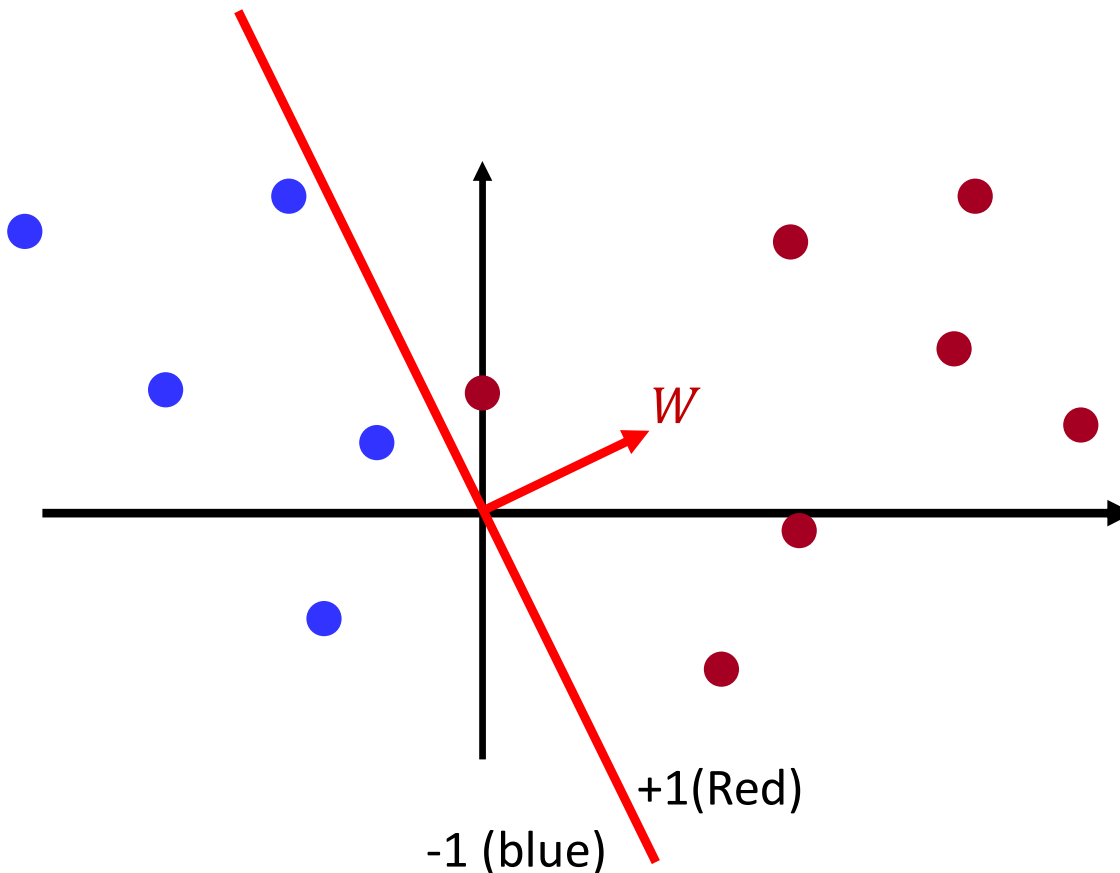
Perceptron Algorithm



The new decision boundary

Perfect classification, no more updates, we are done

Perceptron Algorithm



The new decision boundary

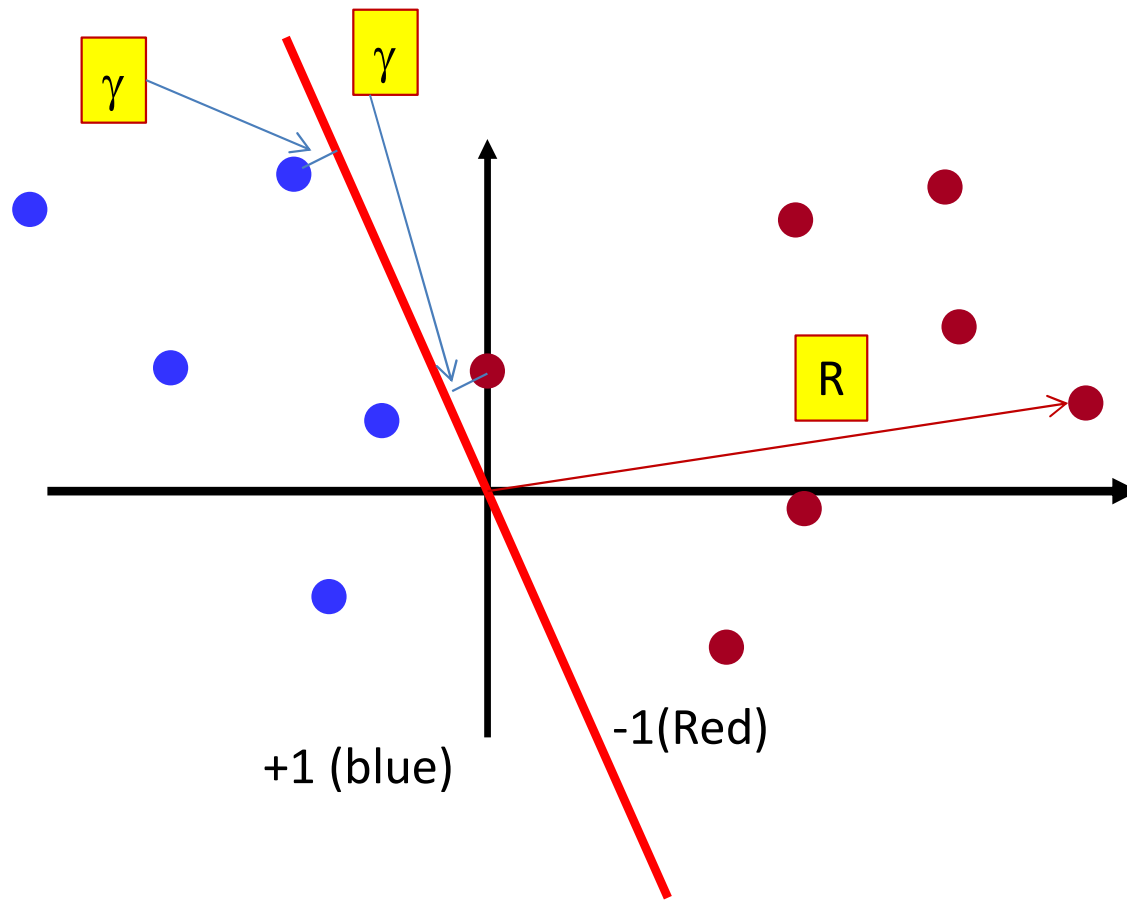
Perfect classification, no more updates, we are done

If the classes are linearly separable, guaranteed to converge in a finite number of steps

Convergence of Perceptron Algorithm

- Guaranteed to converge if classes are linearly separable
 - After no more than $\left(\frac{R}{\gamma}\right)^2$ misclassifications
 - Specifically when W is initialized to 0
 - R is length of longest training point
 - γ is the *best case* closest distance of a training point from the classifier
 - Same as the margin in an SVM
 - Intuitively – takes many increments of size γ to undo an error resulting from a step of size R

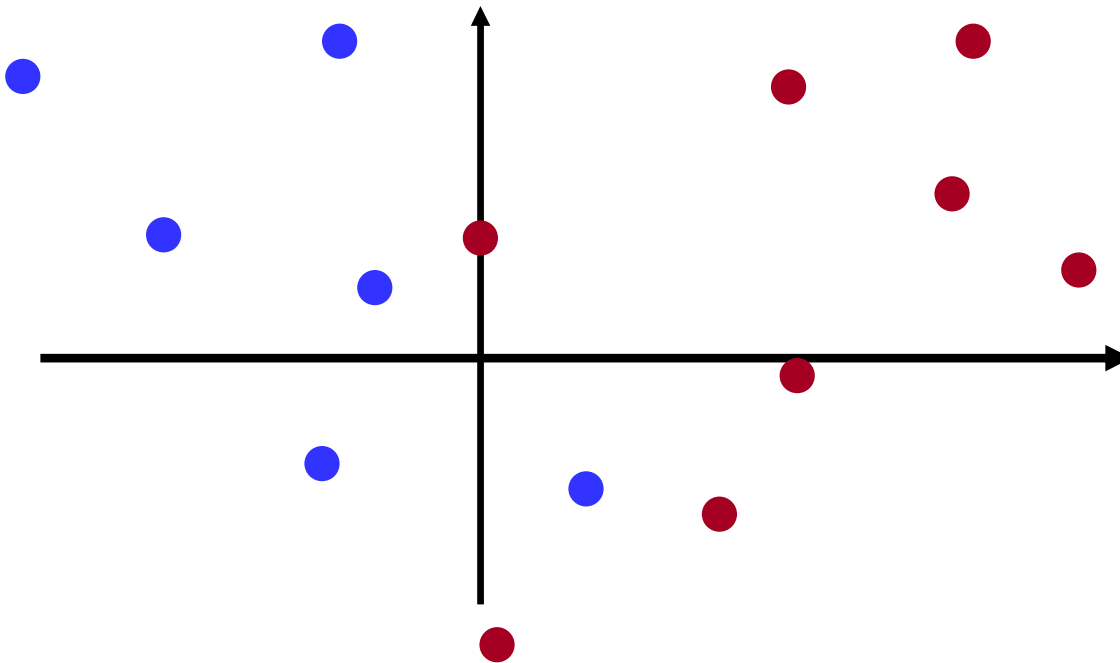
Perceptron Algorithm



γ is the best-case margin
 R is the length of the longest vector

The Perceptron Solution: when classes are not linearly separable

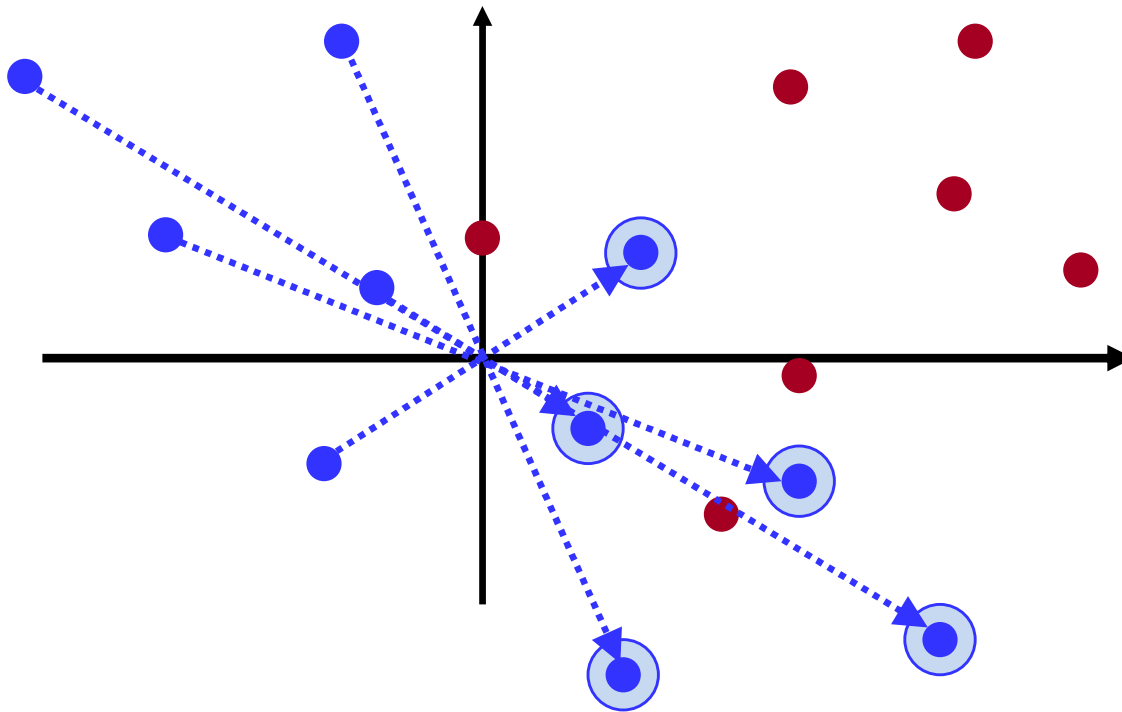
Key: Red 1, Blue = -1



- When classes are not linearly separable, not possible to find a separating hyperplane
 - No “support” plane for reflected data
 - Some points will always lie on the other side
- Model does not support perfect classification of this data
- **Perceptron algorithm will never converge**

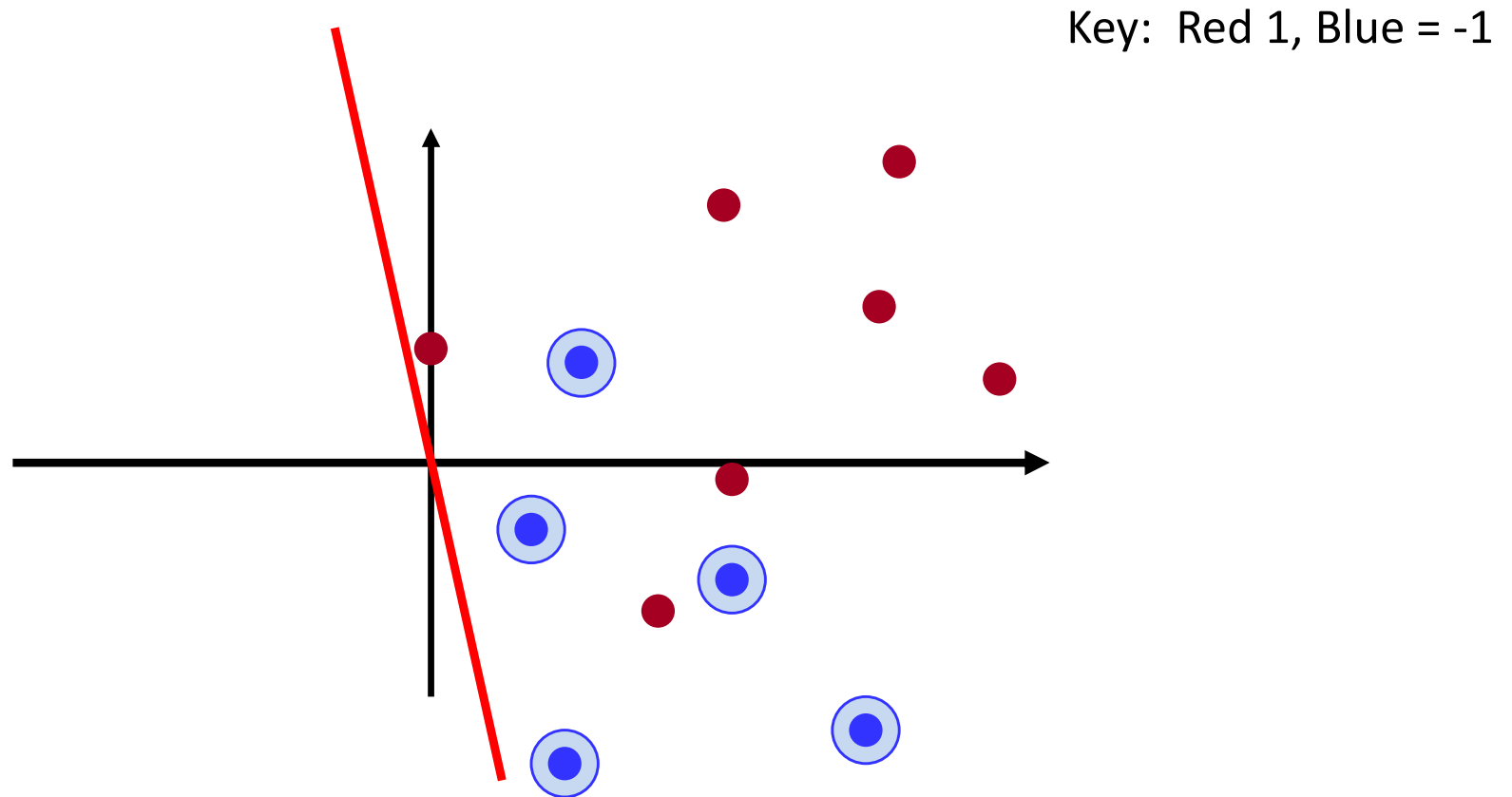
A simpler solution

Key: Red 1, Blue = -1



- *Reflect* all the negative instances across the origin
 - Negate every component of vector X
- If we use class $y \in \{+1, -1\}$ notation for the labels (instead of $y \in \{0,1\}$), we can simply write the “reflected” values as $X' = yX$
 - Will retain the features X for the positive class, but reflect/negate them for the negative class

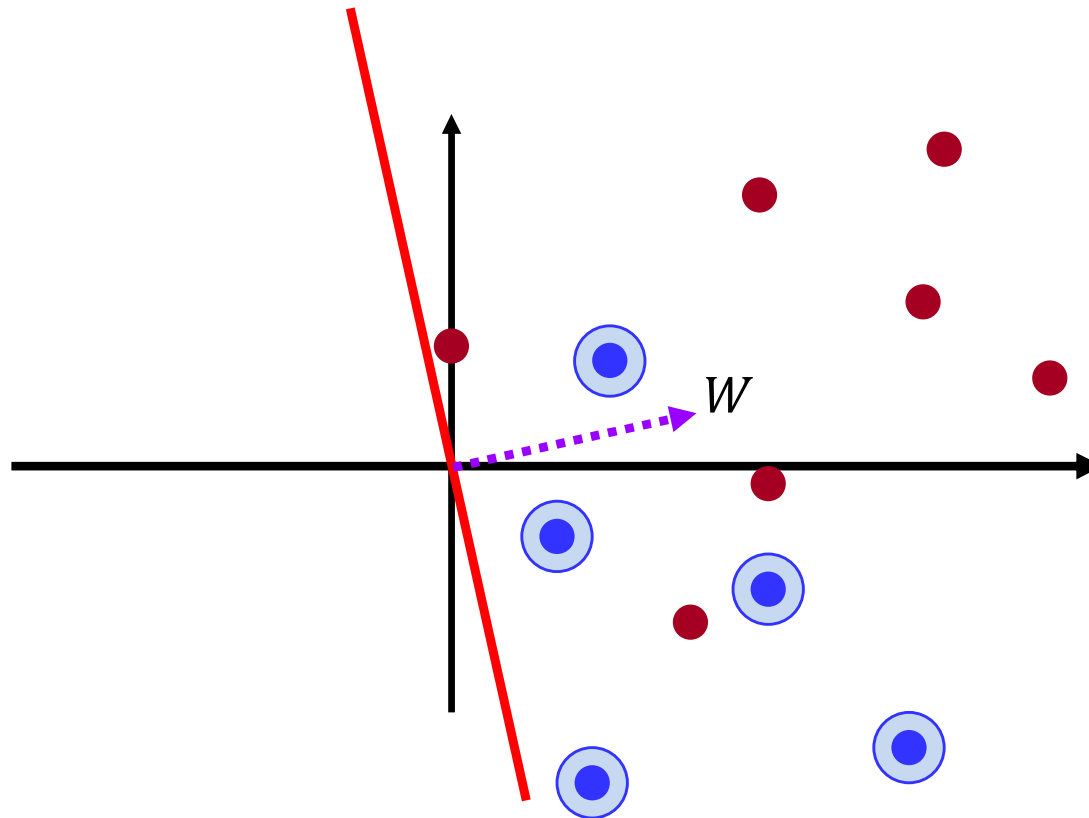
The Perceptron Solution



- Learning the perceptron: Find a plane such that all the modified (X') features lie on one side of the plane
 - Such a plane can always be found if the classes are linearly separable

The Perceptron Solution: Linearly separable case

Key: Red 1, Blue = -1



- When classes are linearly separable: a trivial solution

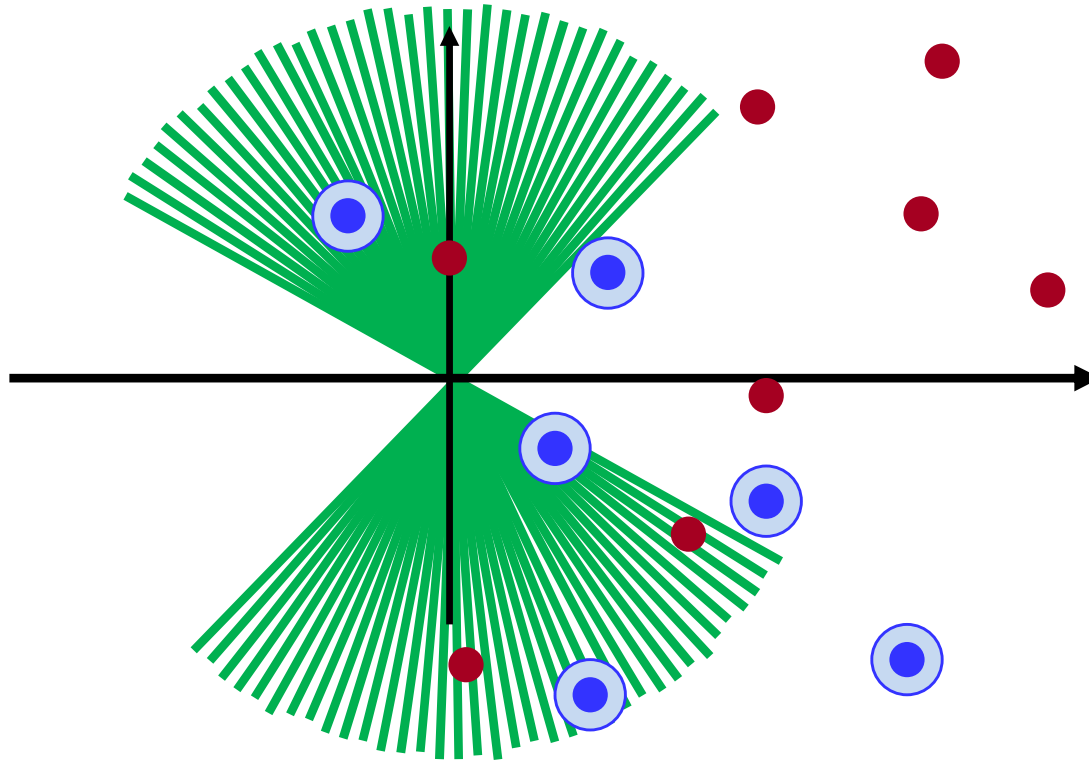
$$W = \frac{1}{N} \sum_i X'_i = \frac{1}{N} \sum_i y_i X_i$$

- Other solutions are also possible, e.g. max-margin solution

The Perceptron Solution:

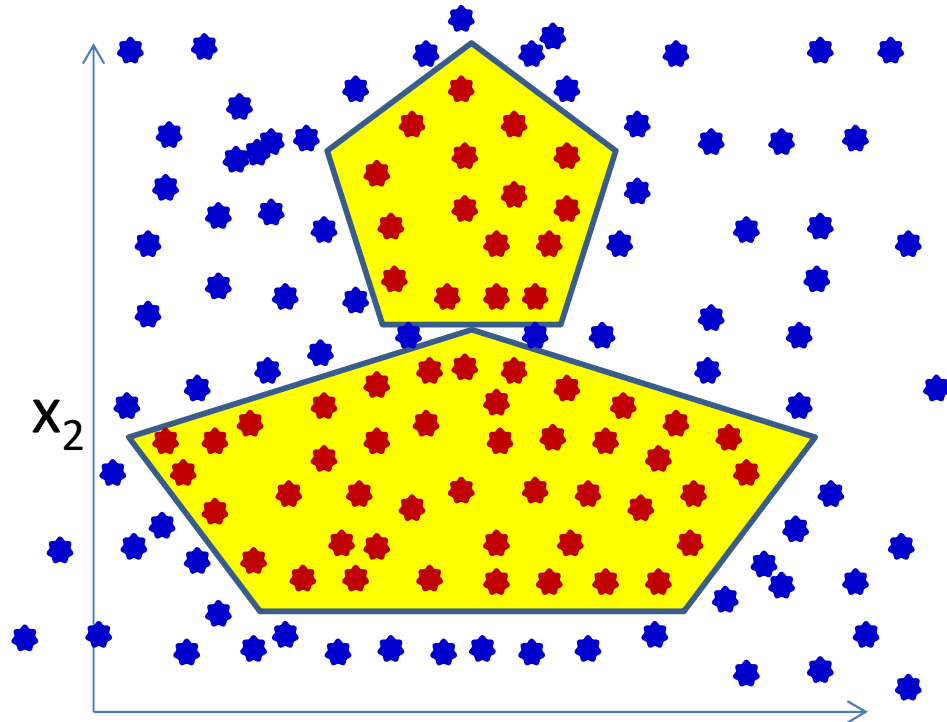
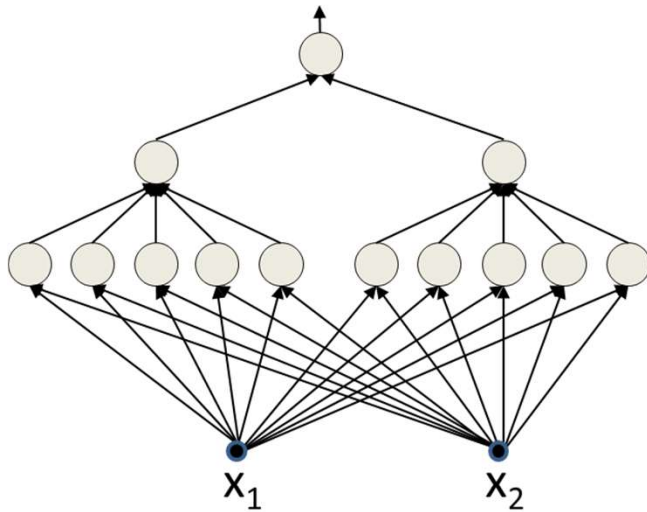
when classes are not linearly separable

Key: Red 1, Blue = -1



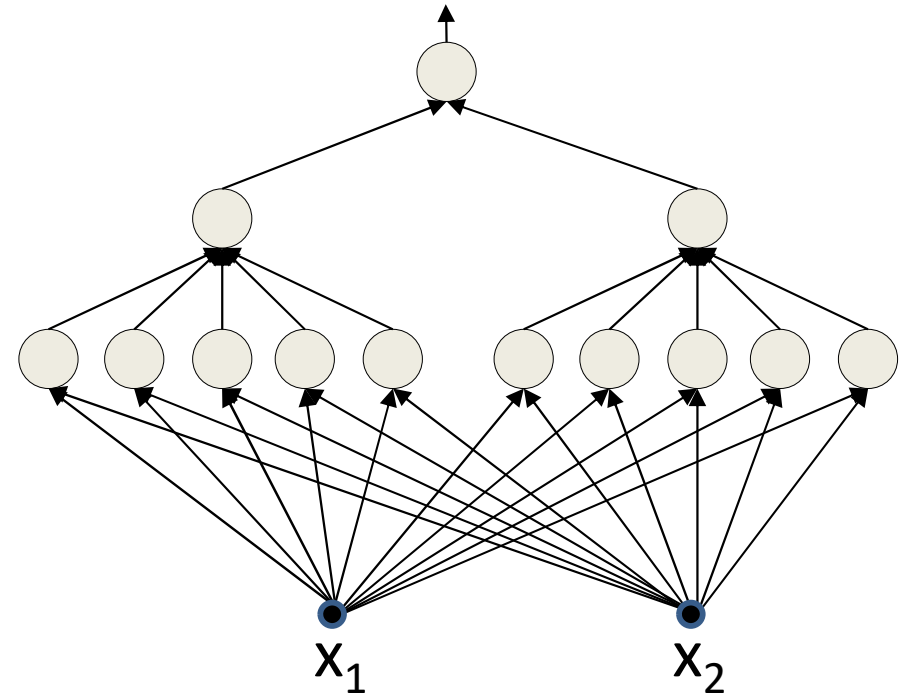
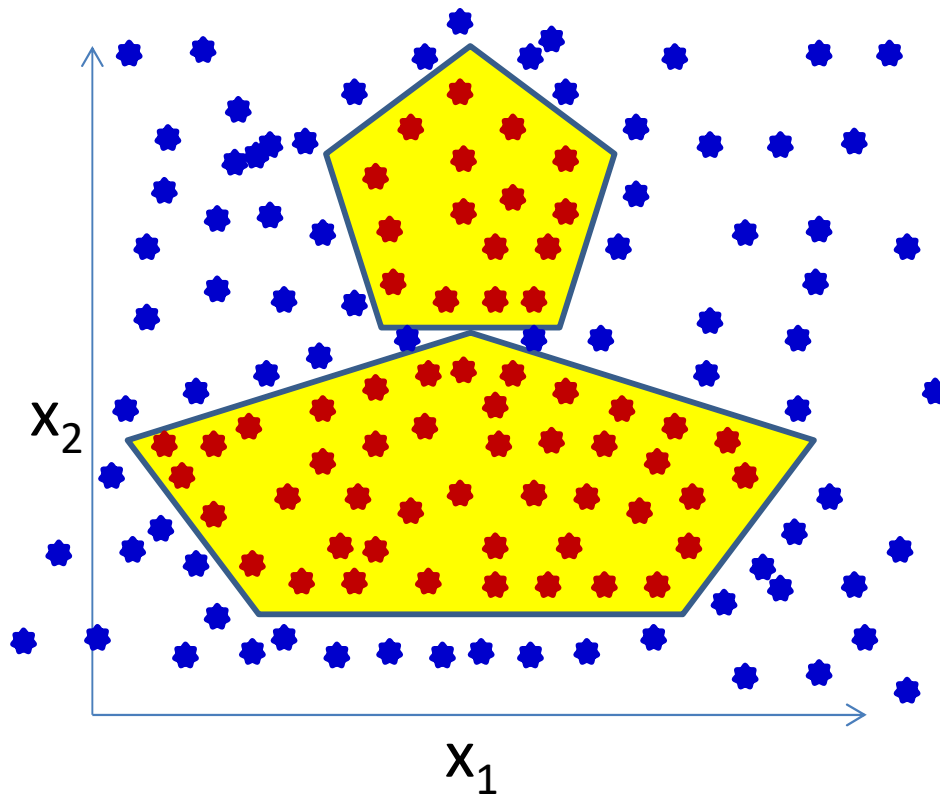
- When classes are not linearly separable, not possible to find a separating hyperplane
 - No “support” plane for reflected data
 - Some points will always lie on the other side
- Model does not support perfect classification of this data

History: A more complex problem



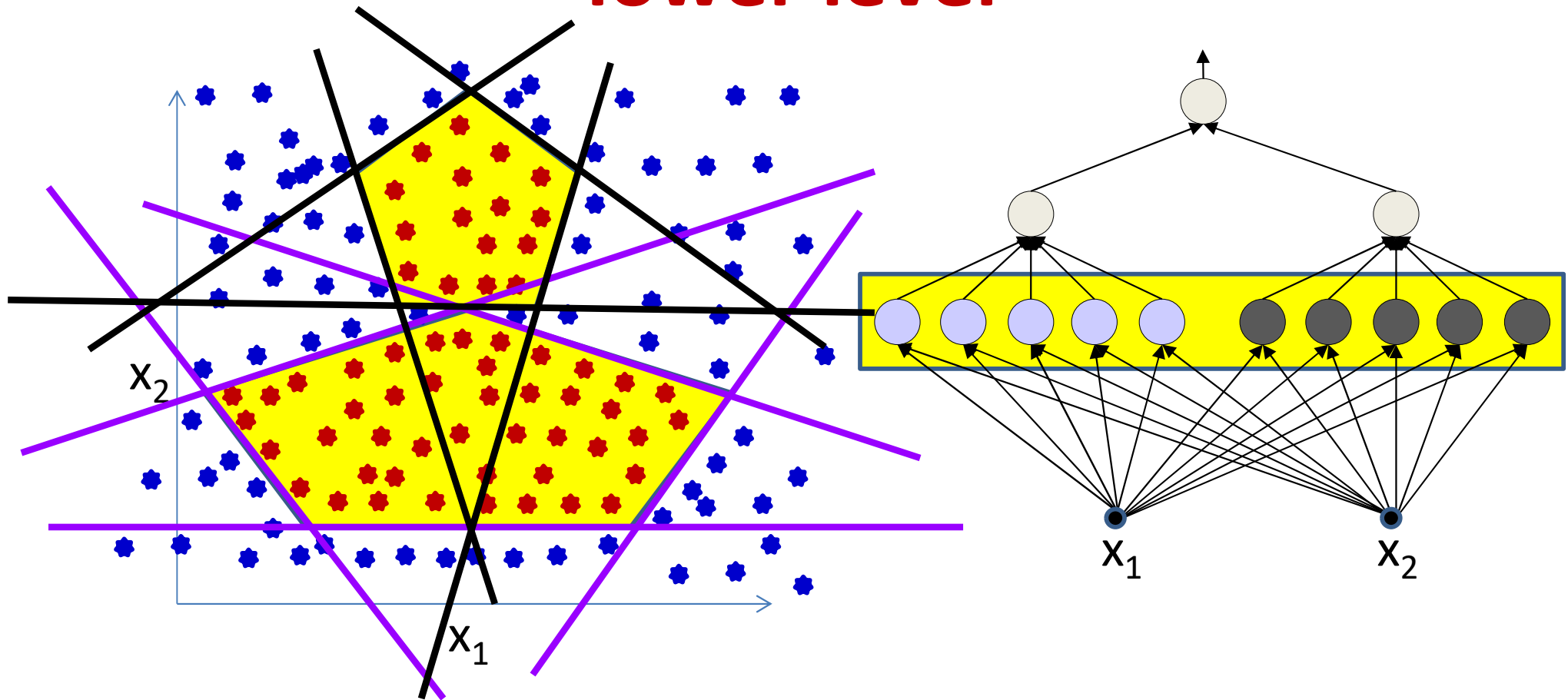
- Learn an *MLP* for this function
 - 1 in the yellow regions, 0 outside
- Using just the samples
- We know this can be perfectly represented using an MLP

More complex decision boundaries



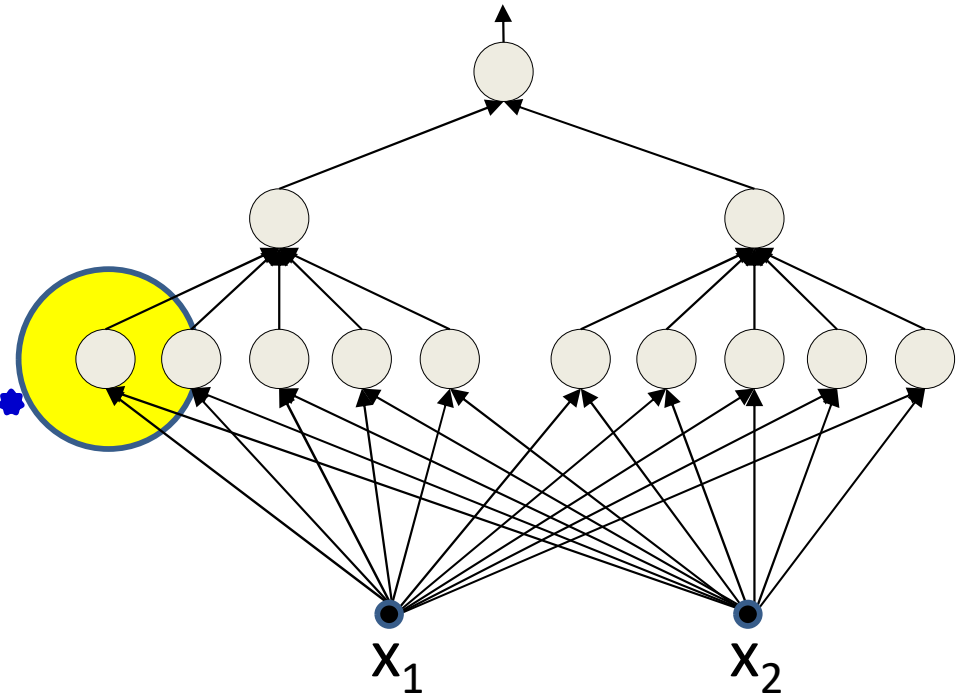
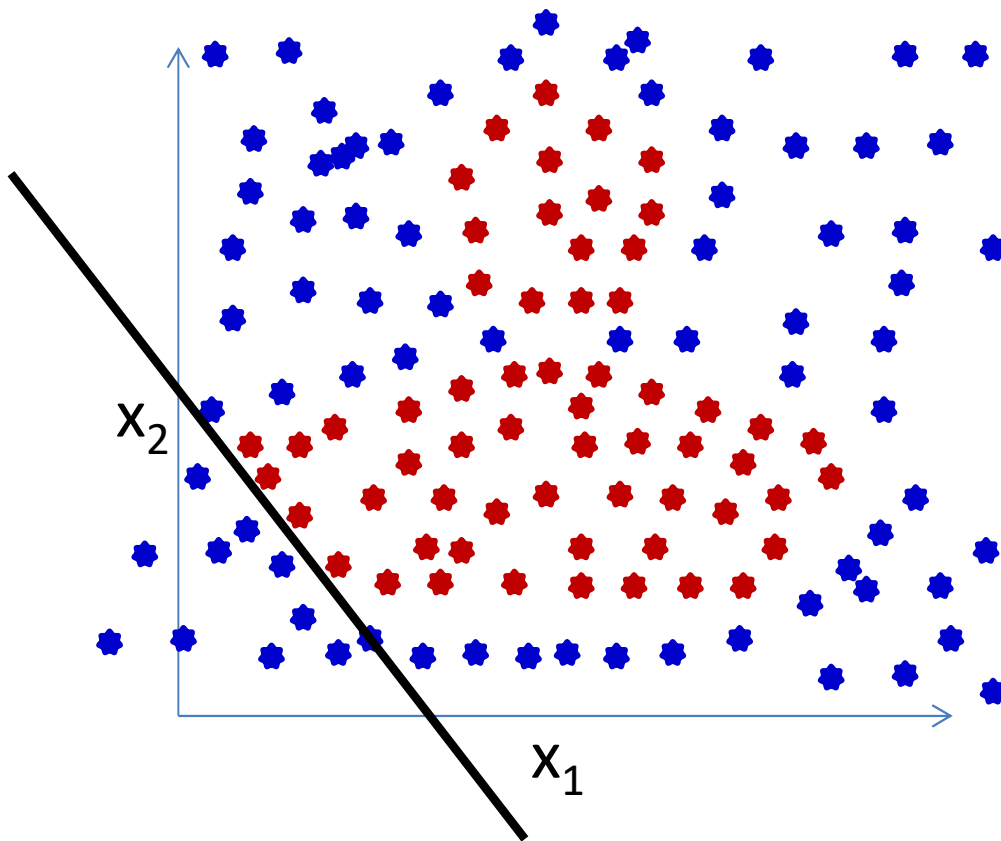
- Even using the perfect architecture...
- ... can we use perceptron learning rules to learn this classification function?

The pattern to be learned at the lower level



- The lower-level neurons are linear classifiers
 - They require linearly separated labels to be learned
 - The actually provided labels are not linearly separated

The pattern to be learned at the lower level



- Consider a single linear classifier that must be learned from the training data
 - Can it be learned from this data?

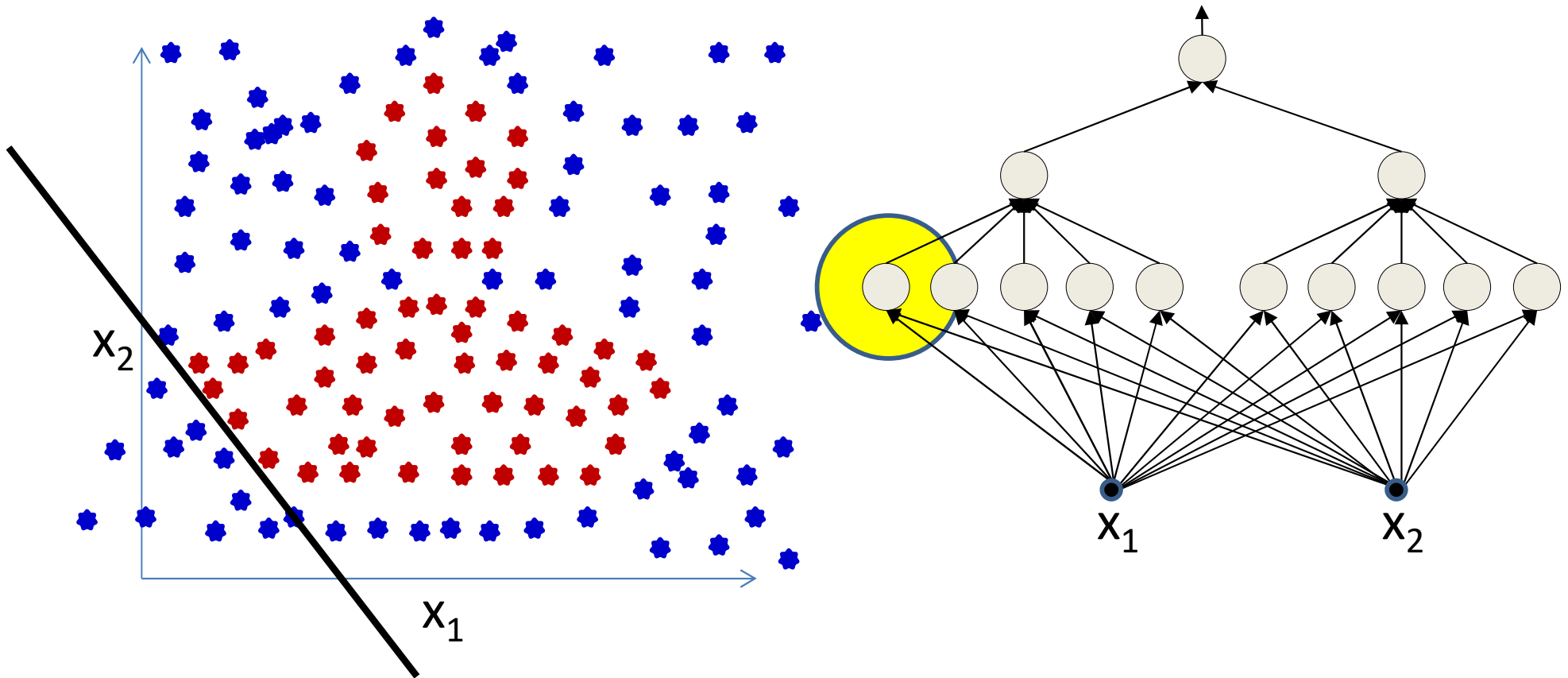
Poll 2

- For the double-pentagon problem, given the data shown on the previous slide, and given that all but the one neuron highlighted in yellow are already correctly learned, can we use the perceptron learning algorithm to learn the one remaining neuron?
 - Yes
 - No
- What problems do you see in using the perceptron rule to learn the remaining perceptron?
 - Perceptron learning will require linearly separable classes to learn the model that classifies the data perfectly, but the data are not linearly separable
 - Perceptron learning will require relabelling the data to make them linearly separable with the correct decision boundary

Poll 2

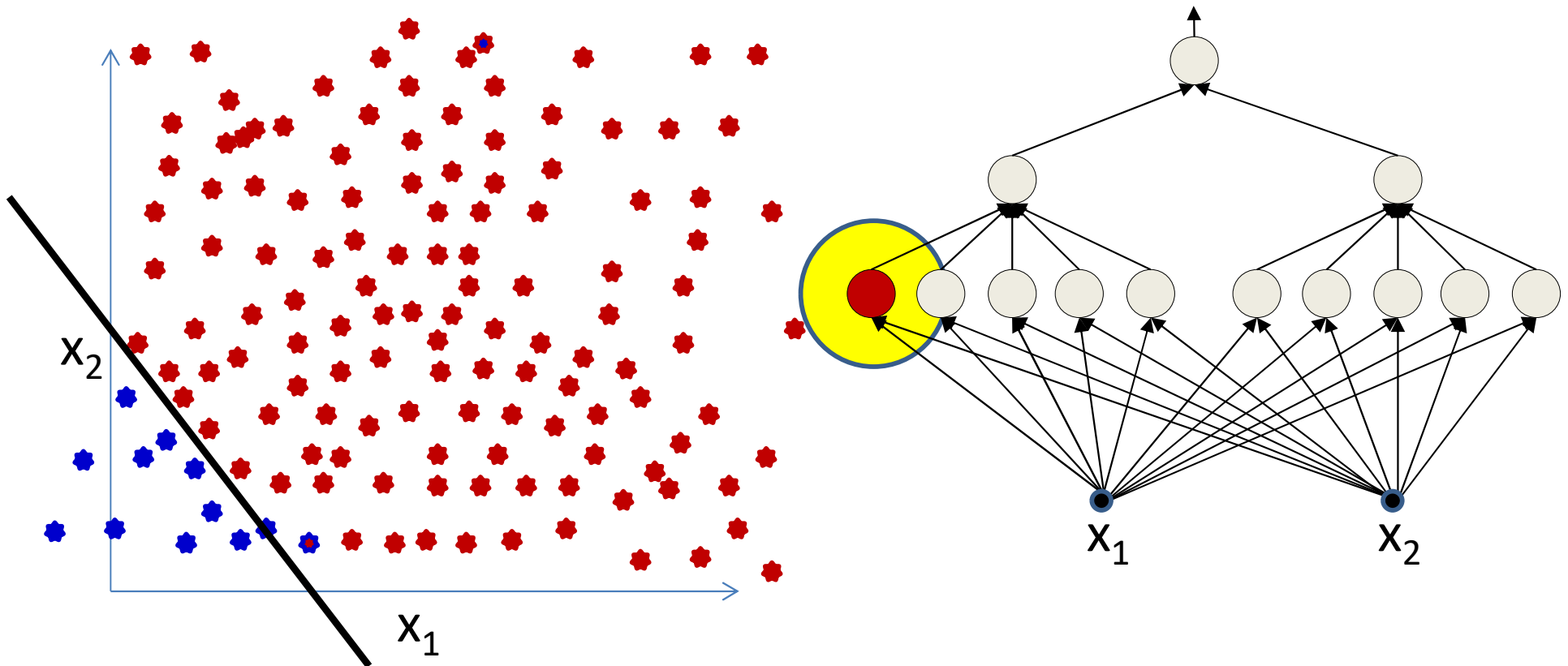
- For the double-pentagon problem, given the data shown on slide 68 and given that all but the one neuron highlighted in yellow are already correctly learned, can we use the perceptron learning algorithm to learn the one remaining neuron?
 - Yes
 - **No**
- What problems do you see in using the perceptron rule to learn the remaining perceptron?
 - **Perceptron learning will require linearly separable classes to learn the model that classifies the data perfectly, but the data are not linearly separable**
 - **Perceptron learning will require relabelling the data to make them linearly separable with the correct decision boundary**

The pattern to be learned at the lower level



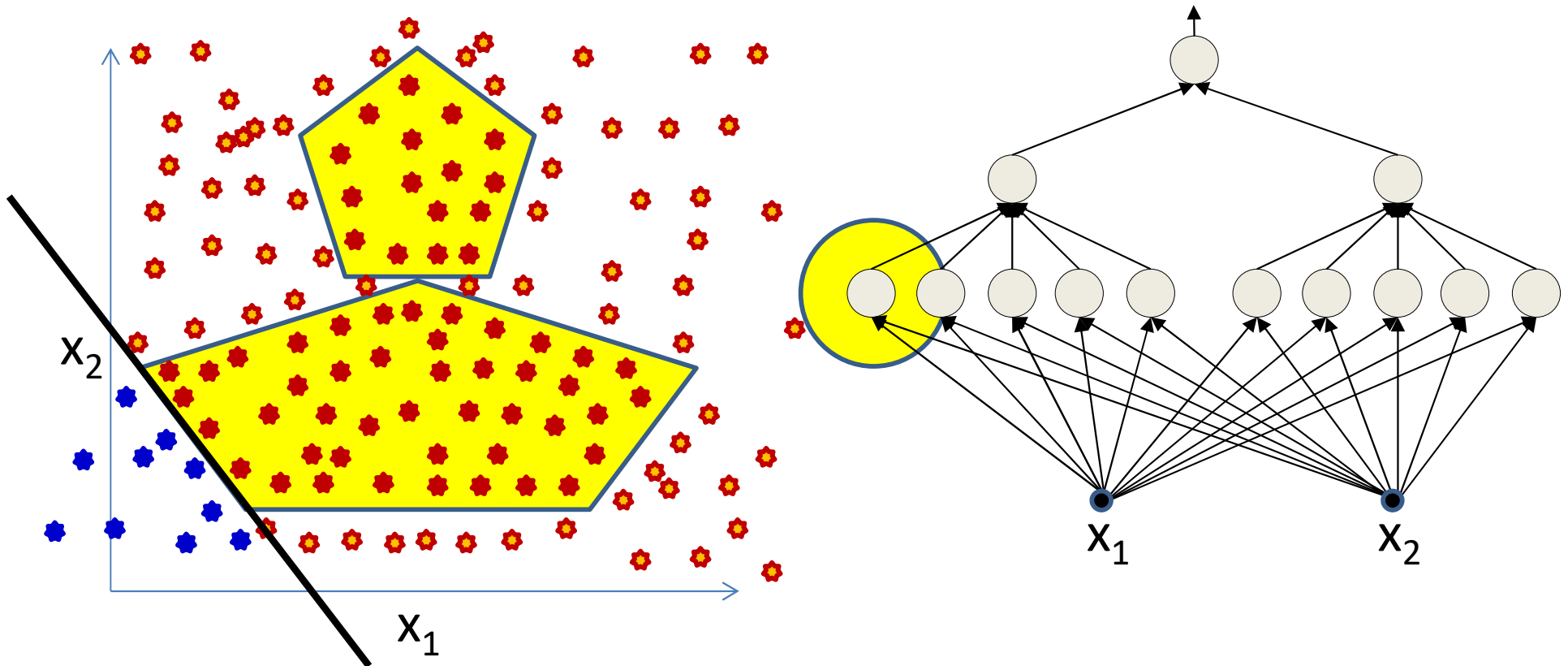
- Consider a single linear classifier that must be learned from the training data
 - Can it be learned from this data?
 - Will require relabelling some instances to learn this line
 - Which ones?

The pattern to be learned at the lower level



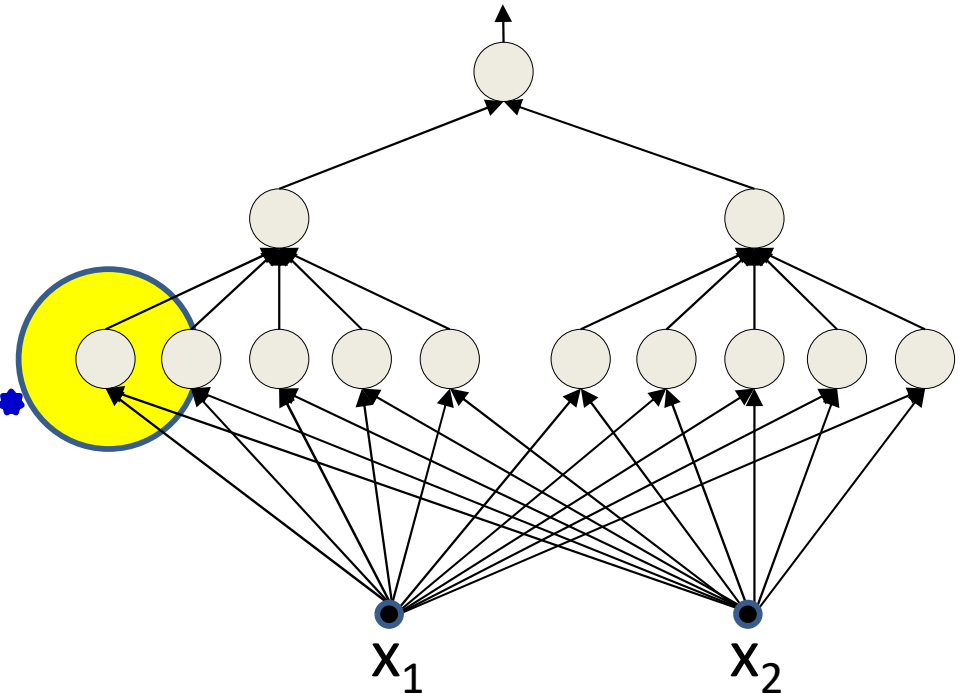
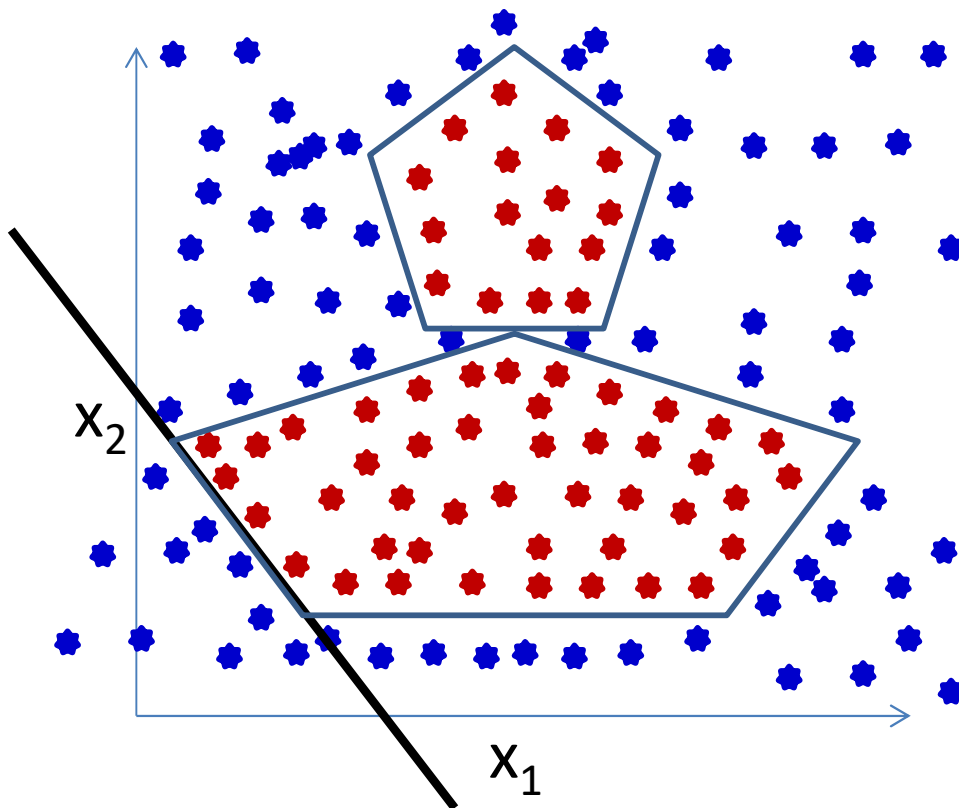
- Consider a single linear classifier that must be learned from the training data
 - Can it be learned from this data?
 - **The individual classifier actually requires the kind of labelling shown here**
 - Which is *not* given!!

The pattern to be learned at the lower level



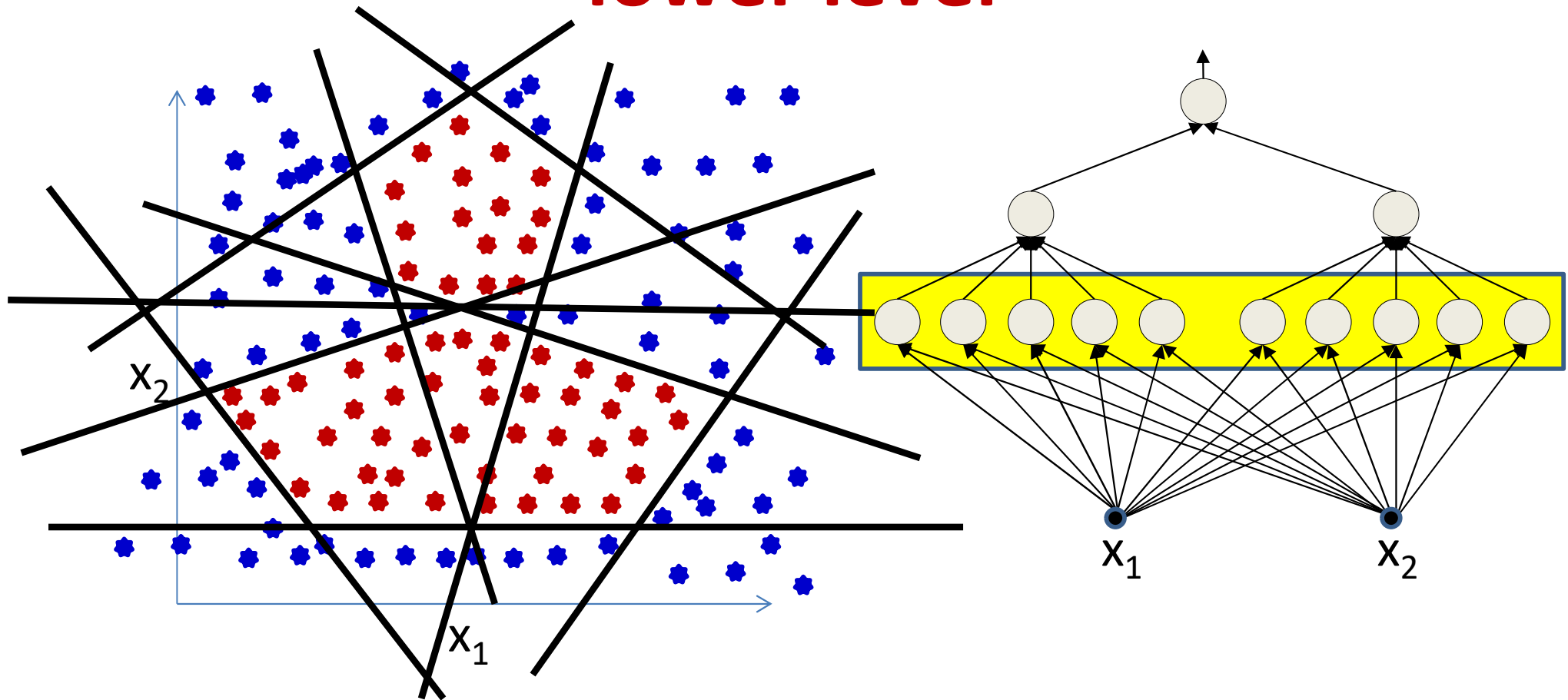
- Try *every possible way* of relabelling the data
 - until we find a linearly-separable relabelling such that...
 - when we learn the line with it, the overall network classifies all training points correctly

The pattern to be learned at the lower level



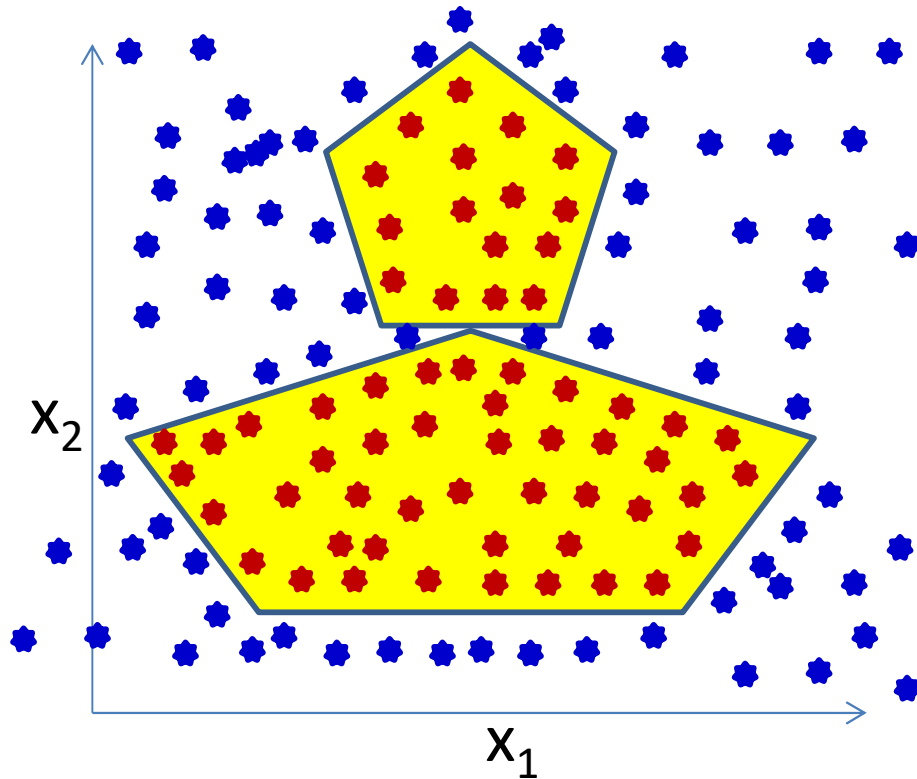
- For a single line:
 - Try out *every possible way of relabeling the red and blue dots such that we can learn a line that correctly completes the classification boundary*

The pattern to be learned at the lower level



- This must be done for *each* of the lines (perceptrons)
- Such that, when all of them are combined by the higher-level perceptrons we get the desired pattern
 - Basically an exponential search over inputs

Individual neurons represent one of the lines that compose the figure (linear classifiers)



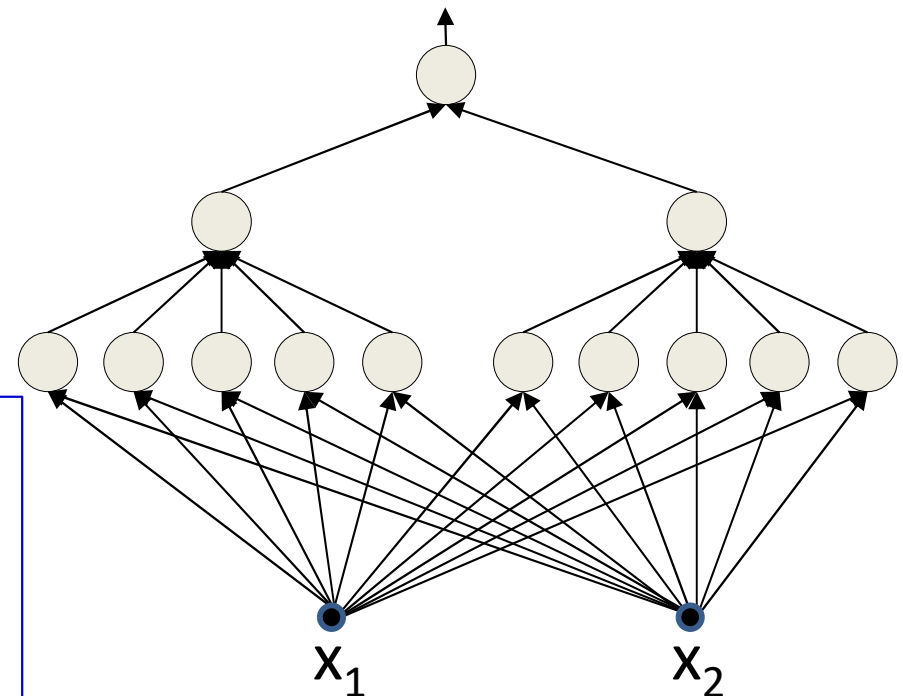
Must know the desired output of every neuron for *every* training instance, in order to learn this neuron

The outputs should be such that the neuron individually has a linearly separable task

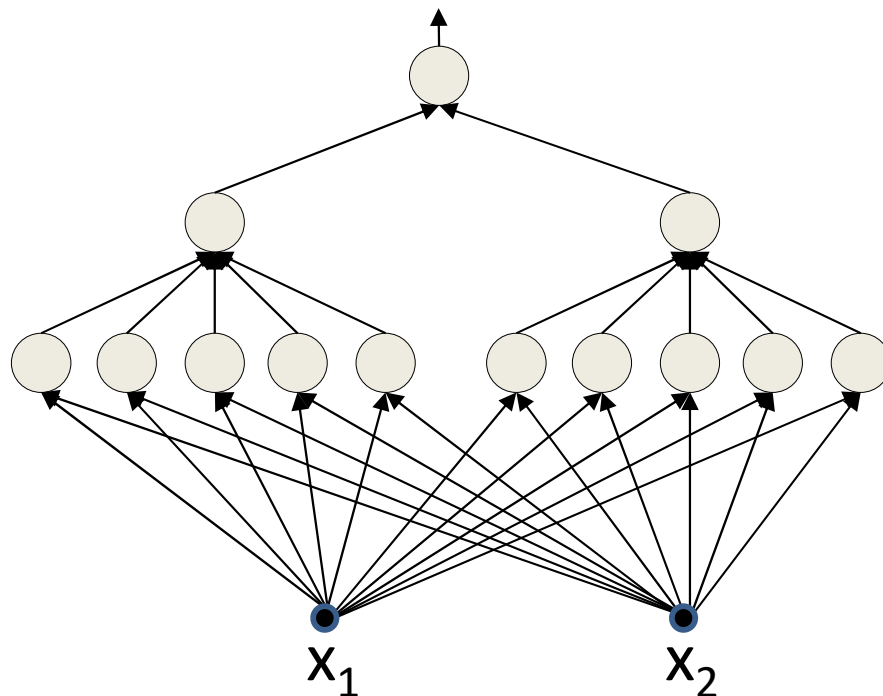
The linear separators must combine to form the desired boundary

This must be done for *every* neuron

Getting any of them wrong will result in incorrect output!



Learning a *multilayer* perceptron



Training data only specifies
input and output of network

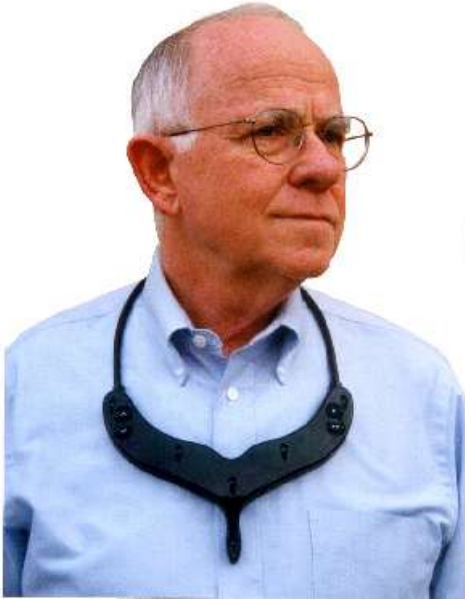
Intermediate outputs (outputs
of individual neurons) are not specified

- Training this network using the perceptron rule is a combinatorial optimization problem
- We don't know the outputs of the individual intermediate neurons in the network for any training input
- **Must also determine the correct output for *each* neuron for *every* training instance**
- **At least exponential (in inputs) time complexity!!!!!!**

Greedy algorithms: Adaline and Madaline

- Perceptron learning rules cannot directly be used to learn an MLP
 - Exponential complexity of assigning intermediate labels
 - Even worse when classes are not actually separable
- Can we use a *greedy* algorithm instead?
 - Adaline / Madaline
 - On slides, will skip in class (check the quiz)

A little bit of History: Widrow



Bernie Widrow

- Scientist, Professor, Entrepreneur
- Inventor of most useful things in signal processing and machine learning!

- First known attempt at an analytical solution to training the perceptron and the MLP
- Now famous as the LMS algorithm
 - Used everywhere
 - Also known as the “delta rule”

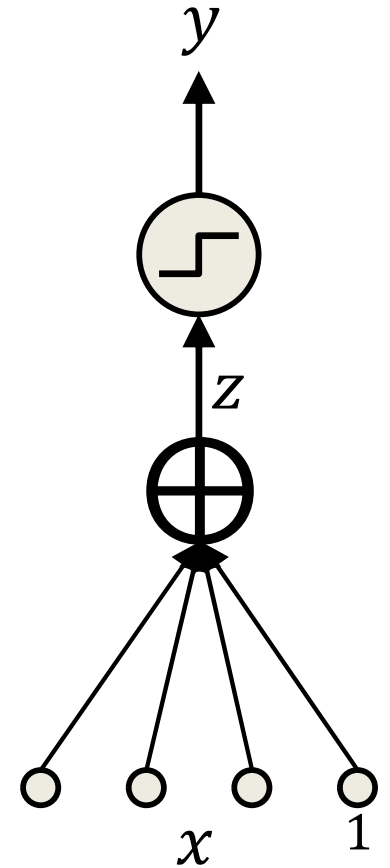
History: ADALINE

$$z = \sum_t w_i x_i$$

Using 1-extended vector notation to account for bias

$$y = \begin{cases} 0, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

- Adaptive *linear* element (Hopf and Widrow, 1960)
- Actually just a regular perceptron
 - Weighted sum on inputs and bias passed through a thresholding function
- ADALINE differs in the *learning rule*



History: Learning in ADALINE

$$z = \sum_i w_i x_i$$

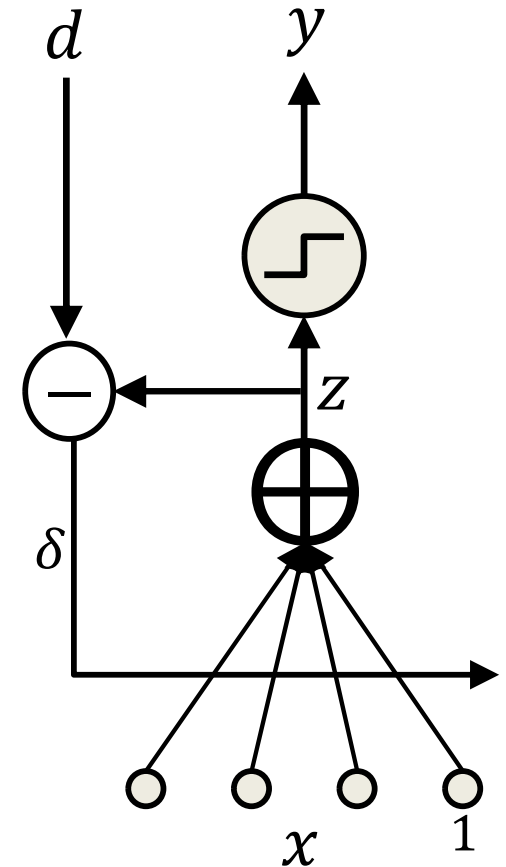
$$out = \begin{cases} 0, & z < 0 \\ 1, & z \geq 0 \end{cases}$$

- During learning, minimize the squared error assuming z to be real output
- The desired output is still binary!

$$Err(x) = \frac{1}{2} (d - z)^2$$

Error for a single input

$$\frac{dErr(x)}{dw_i} = -(d - z)x_i$$



History: Learning in ADALINE

$$z = \sum_t w_i x_i$$

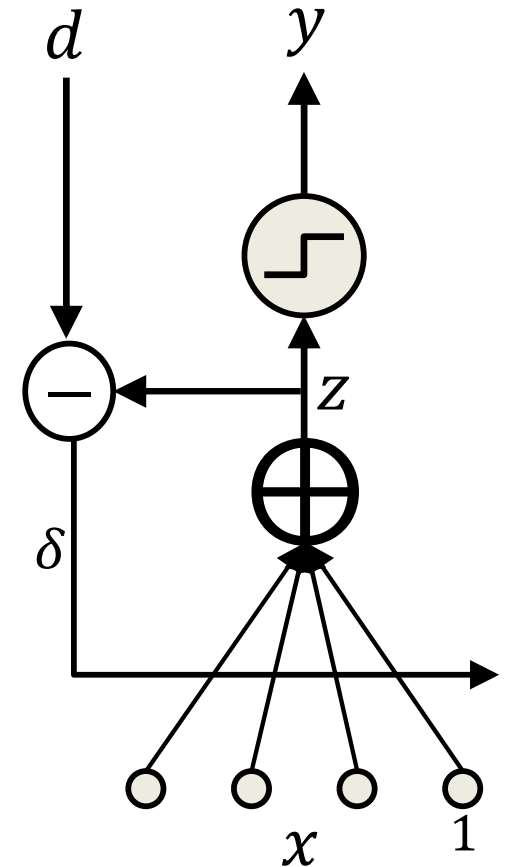
$$Err(x) = \frac{1}{2} (d - z)^2$$

Error for a single input

$$\frac{dErr(x)}{dw_i} = -(d - z)x_i$$

- If we just have a single training input, the *gradient descent* update rule is

$$w_i = w_i + \eta (d - z)x_i$$



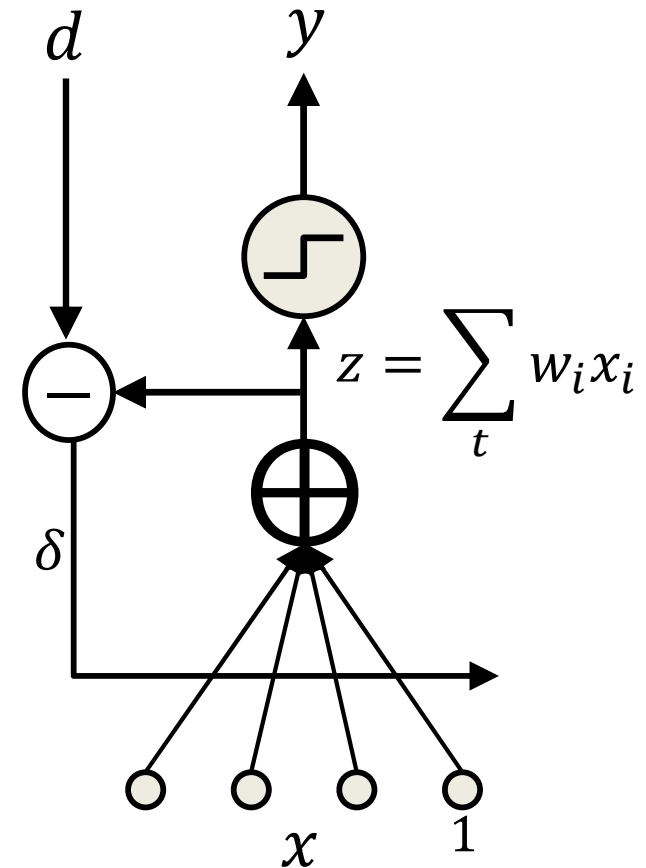
The ADALINE learning rule

- Online learning rule
- **After each input $\mathbf{x}$** , that has target (binary) output d , compute and update:

$$\delta = d - z$$

$$w_i = w_i + \eta \delta x_i$$

- This is the famous *delta rule*
 - Also called the LMS update rule



The Delta Rule

- In fact both the Perceptron and ADALINE use variants of the delta rule!

- Perceptron: Output used in delta rule is y

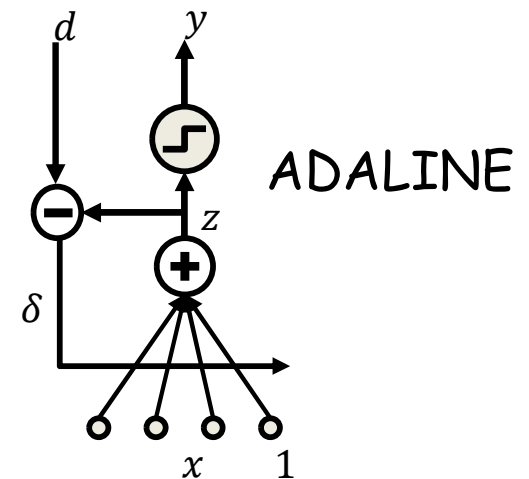
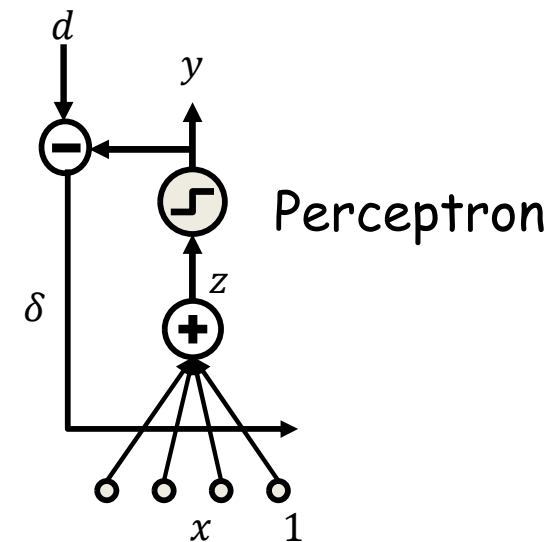
$$\delta = d - y$$

- ADALINE: Output used to estimate weights is z

$$\delta = d - z$$

- For both

$$w_i = w_i + \eta \delta x_i$$



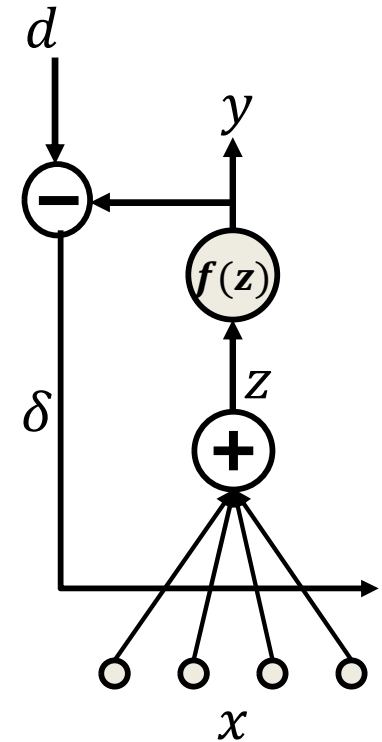
Aside: Generalized delta rule

- For any differentiable activation function the following update rule is used

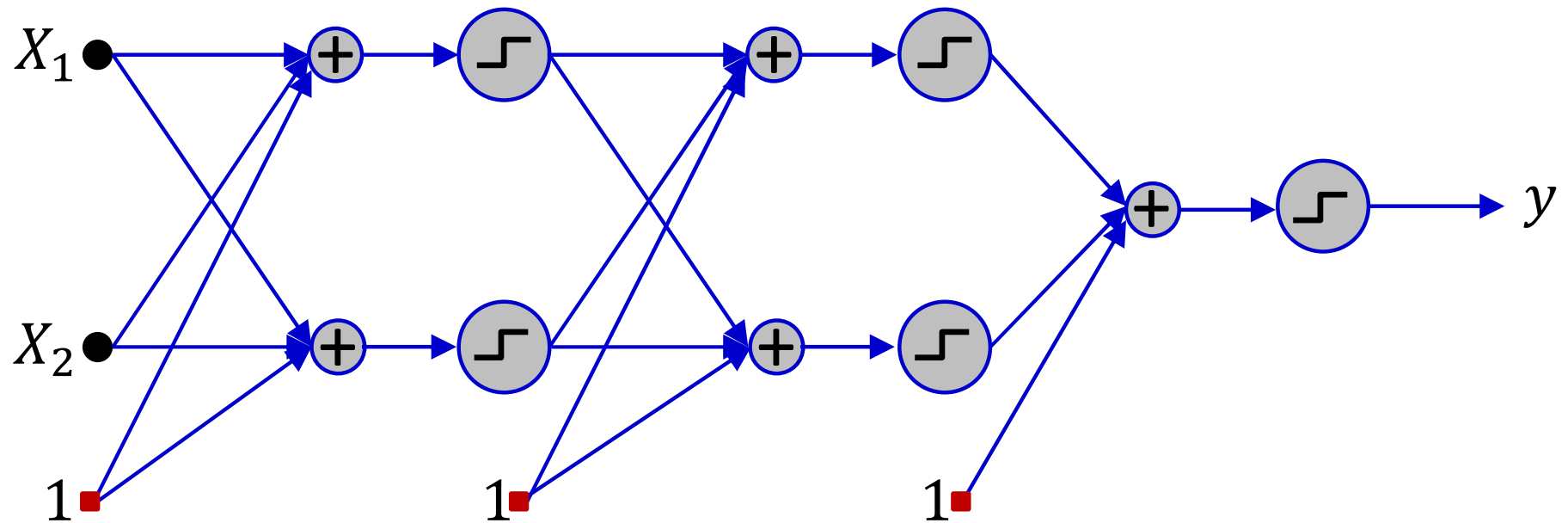
$$\delta = d - y$$

$$w_i = w_i + \eta \delta f'(z) x_i$$

- This is the famous Widrow-Hoff update rule
 - Lookahead: Note that this is *exactly* backpropagation in multilayer nets if we let $f(z)$ represent the entire network between z and y
- It is possibly the most-used update rule in machine learning and signal processing
 - Variants of it appear in almost every problem

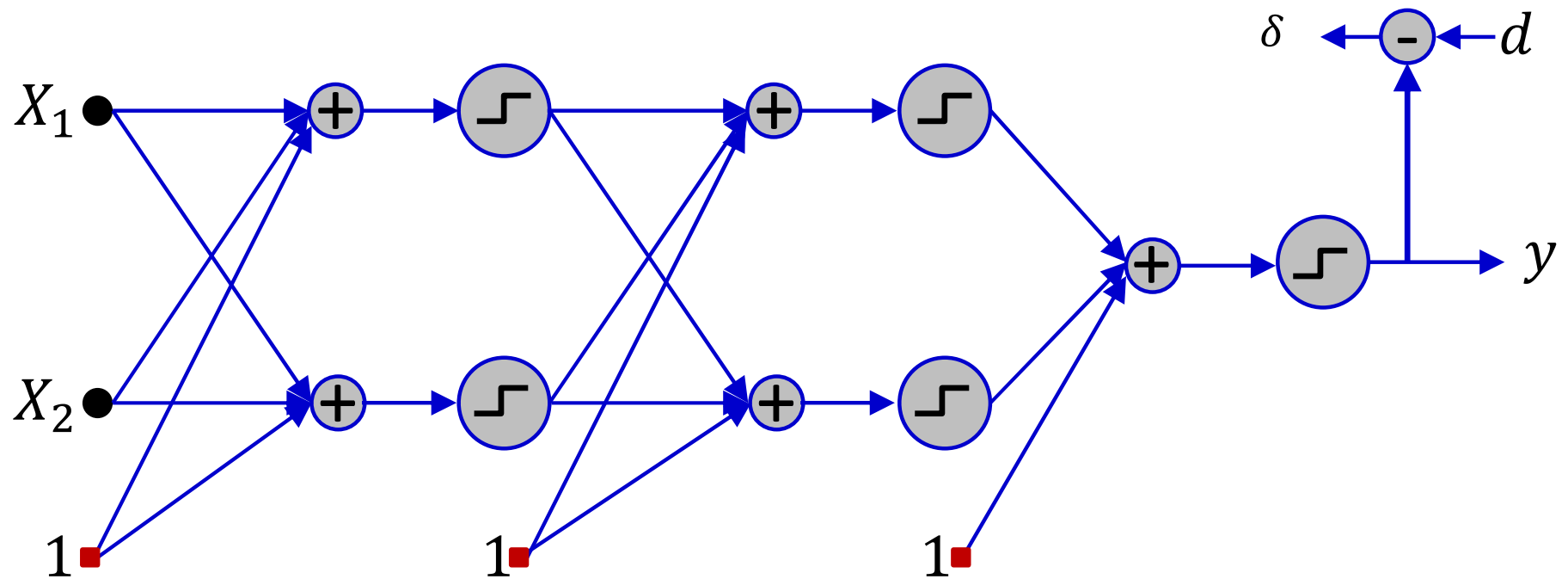


Multilayer perceptron: MADALINE



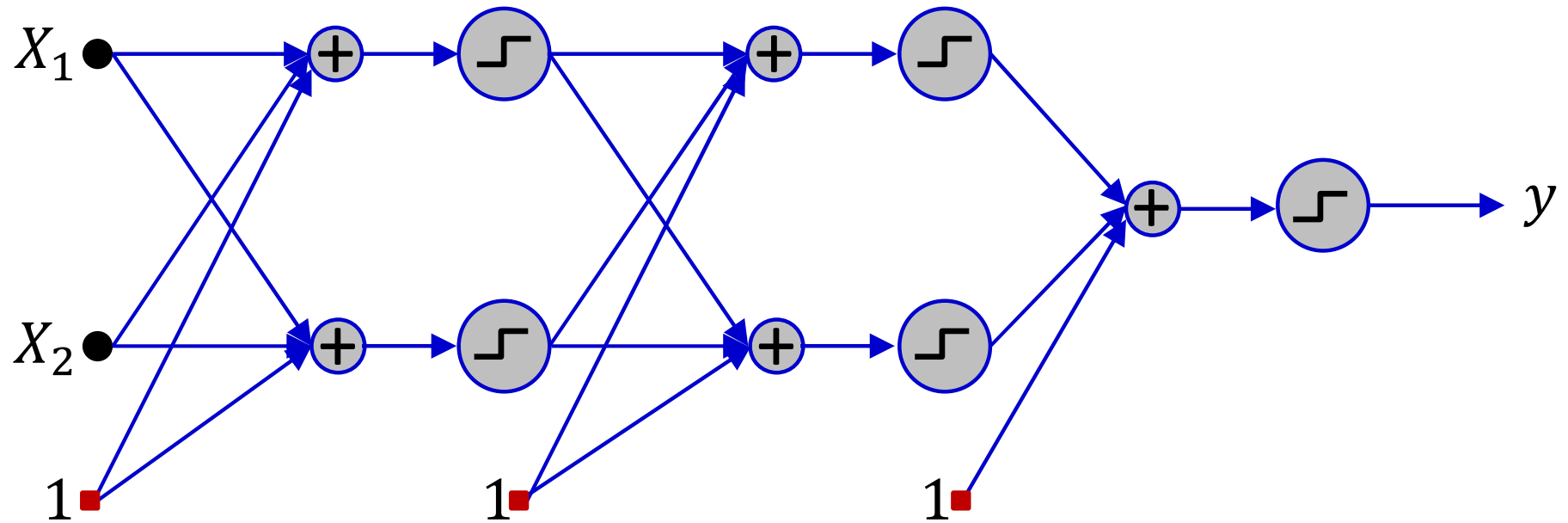
- *Multiple Adaline*
 - A multilayer perceptron with threshold activations
 - The MADALINE

MADALINE Training



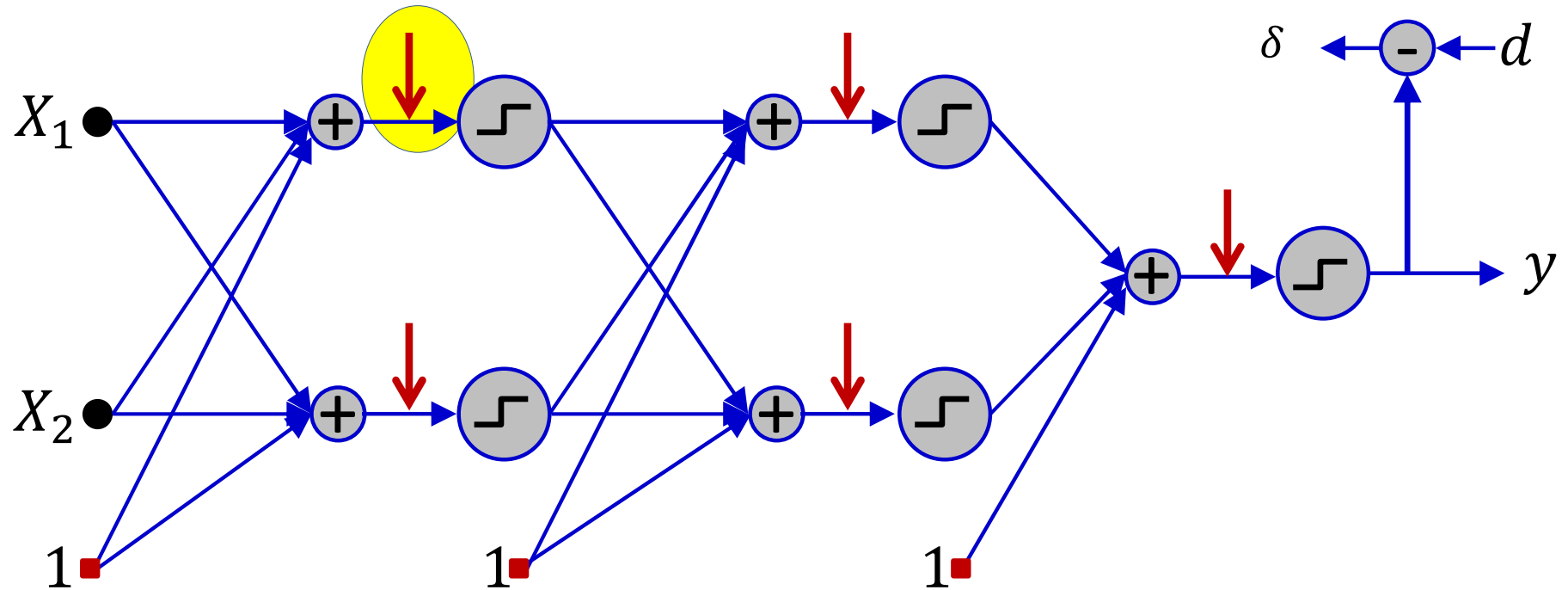
- *Update only on error*
 - $\delta \neq 0$
 - On inputs for which output and target values differ

MADALINE Training



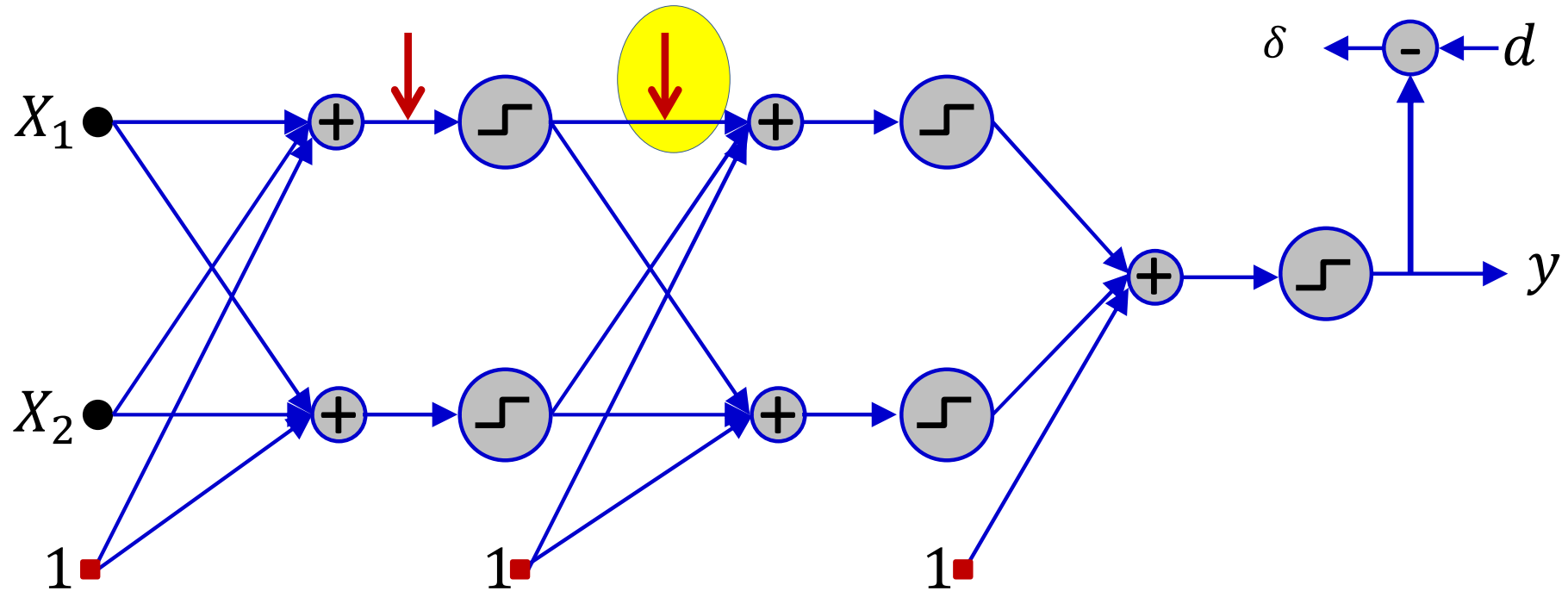
- While stopping criterion not met do:
 - Classify an input

MADALINE Training



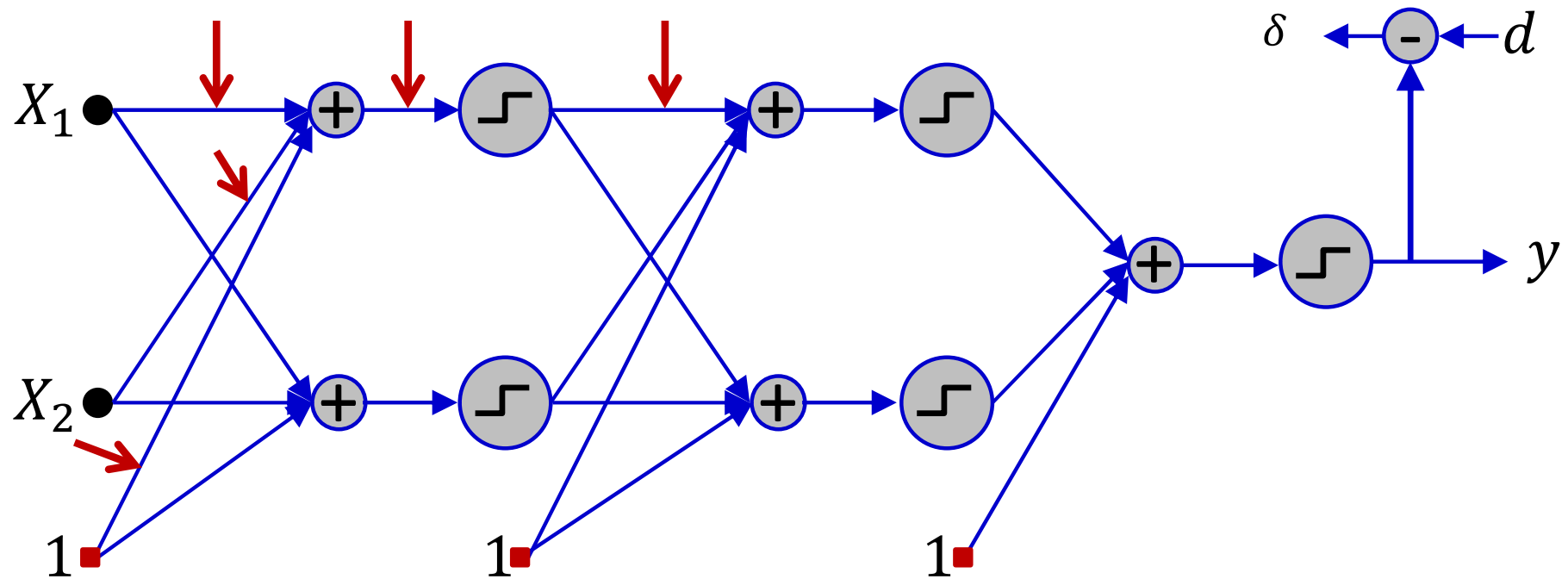
- While stopping criterion not met do:
 - Classify an input
 - If error, find the z that is closest to 0

MADALINE Training



- While stopping criterion not met do:
 - Classify an input
 - If error, find the z that is closest to 0
 - Flip the output of corresponding unit and compute new output

MADALINE Training



- While stopping criterion not met do:
 - Classify an input
 - If error, find the z that is closest to 0
 - Flip the output of corresponding unit and compute new output
 - If error reduces:
 - Set the desired output of the unit to the flipped value
 - Apply ADALINE rule to update weights of the unit

MADALINE

- Greedy algorithm, effective for small networks
- Not very useful for large nets
 - Too expensive
 - Too greedy

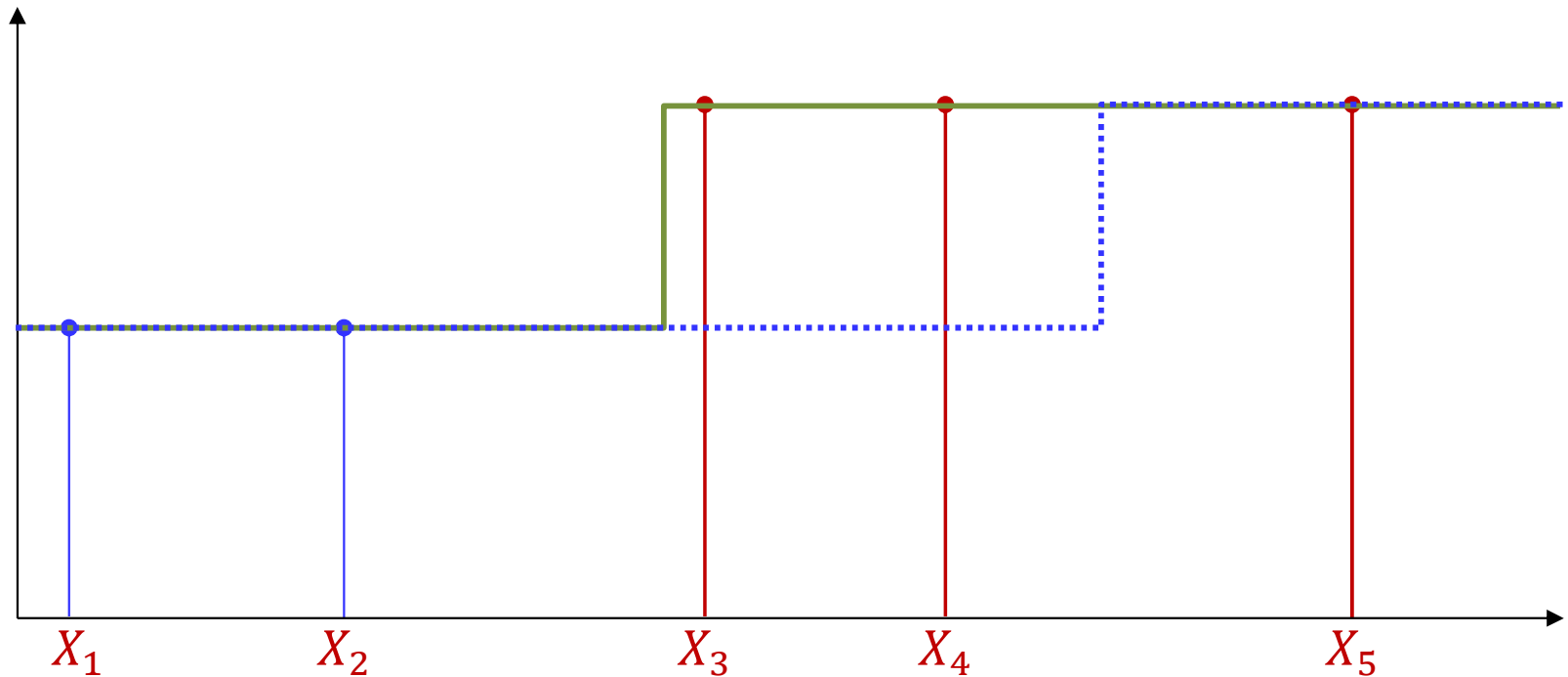
Story so far

- “Learning” a network = learning the weights and biases to compute a target function
 - Will require a network with sufficient “capacity”
- In practice, we learn networks by “fitting” them to match the input-output relation of “training” instances drawn from the target function
- A linear decision boundary can be learned by a single perceptron (with a threshold-function activation) in linear time if classes are linearly separable
- Non-linear decision boundaries require networks of perceptrons
- Training an MLP with threshold-function activation perceptrons will require knowledge of the input-output relation for every training instance, for *every* perceptron in the network
 - These must be determined as part of training
 - For threshold activations, this is an NP-complexity combinatorial optimization problem

History..

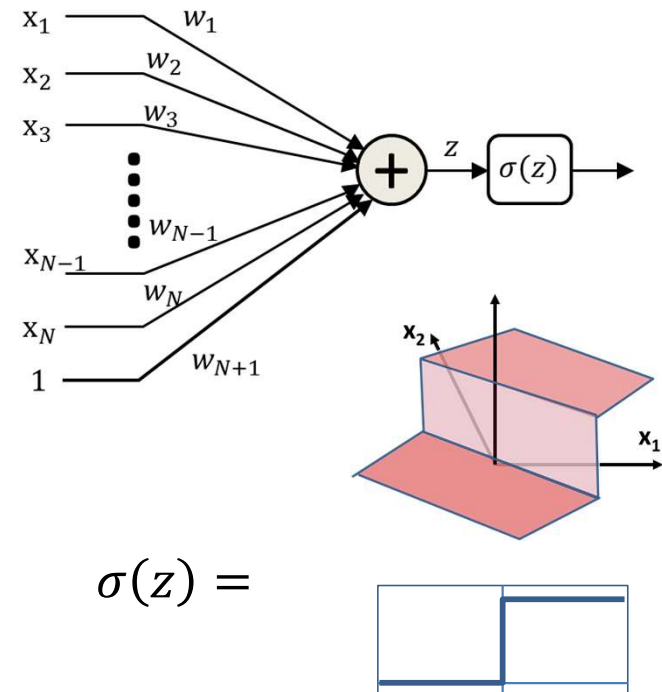
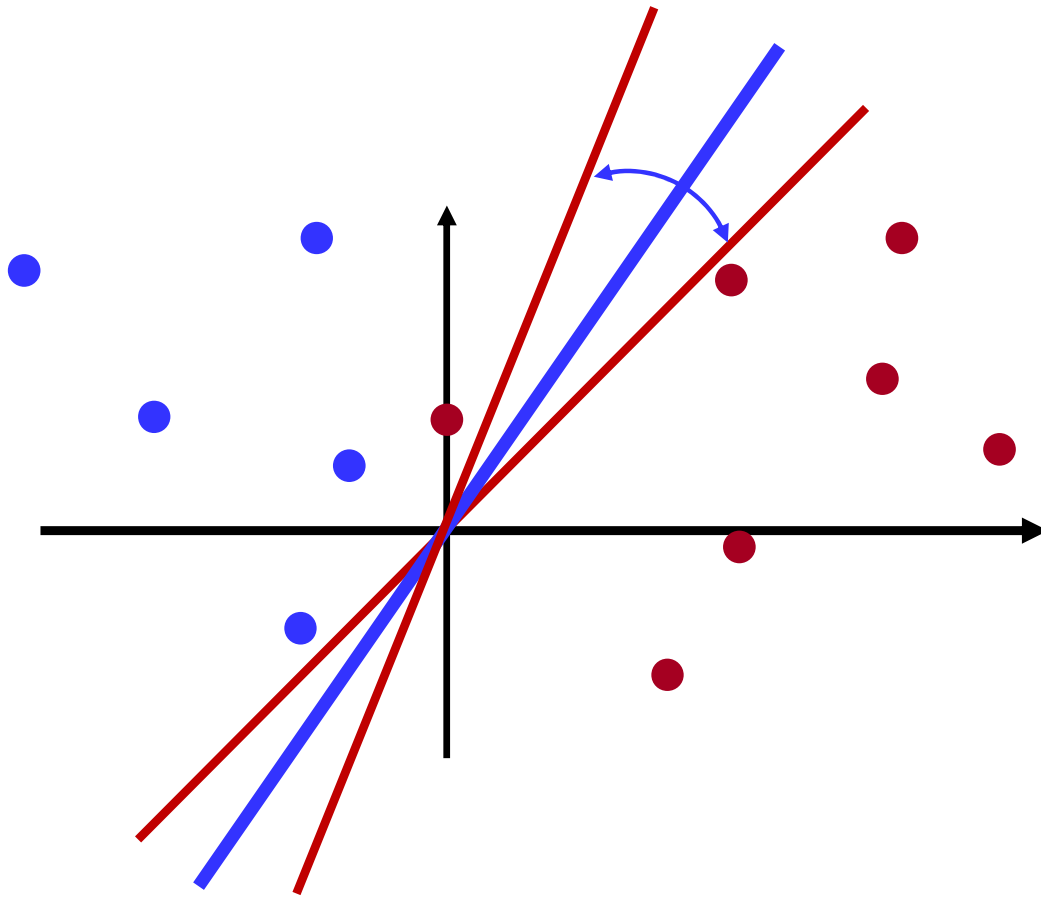
- The realization that training an entire MLP was a combinatorial optimization problem stalled development of neural networks for well over a decade!

The problem



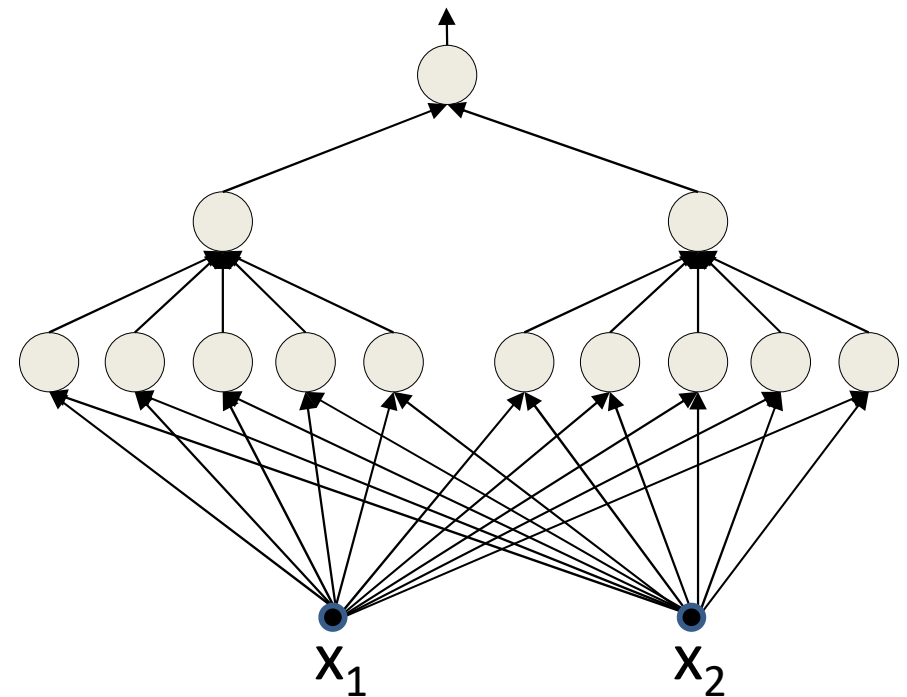
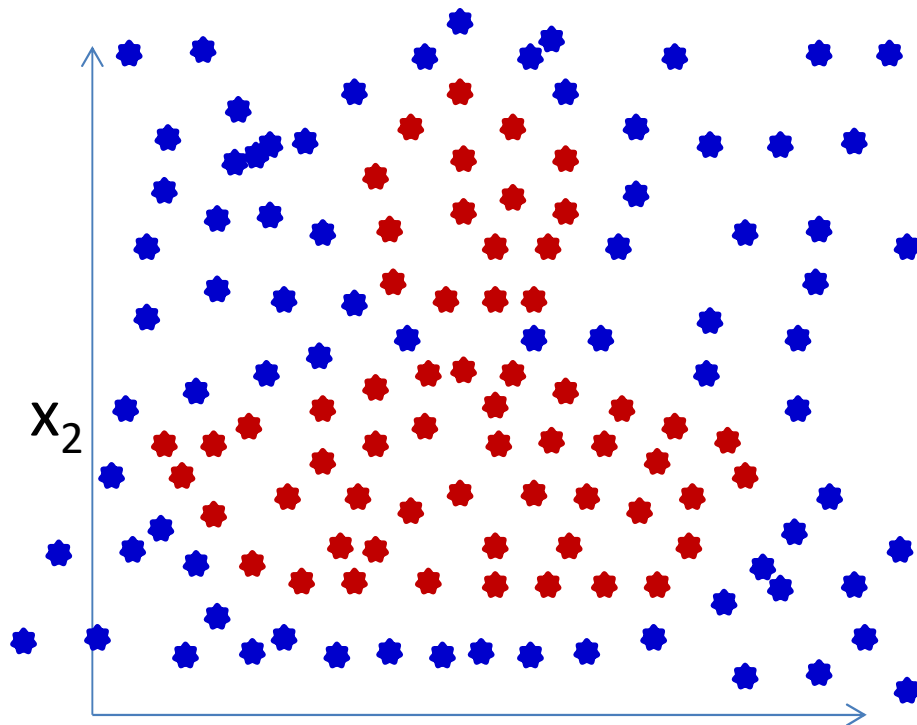
- Our binary error metric is not useful
 - To improve the classifier, we must move the blue dotted line left
 - But if we move it only slightly, moving it either right or left results in no change in error

Why this problem?



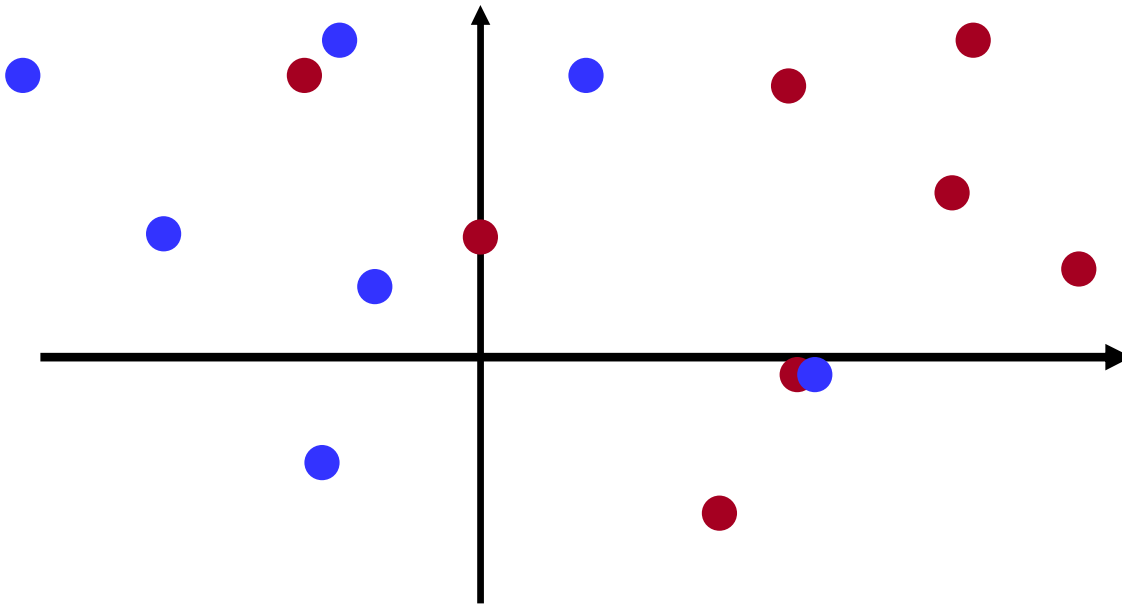
- The perceptron is a flat function with zero derivative everywhere, except at 0 where it is non-differentiable
 - You can vary the weights a *lot* without changing the error
 - There is no indication of which direction to change the weights to reduce error

This only compounds on larger problems



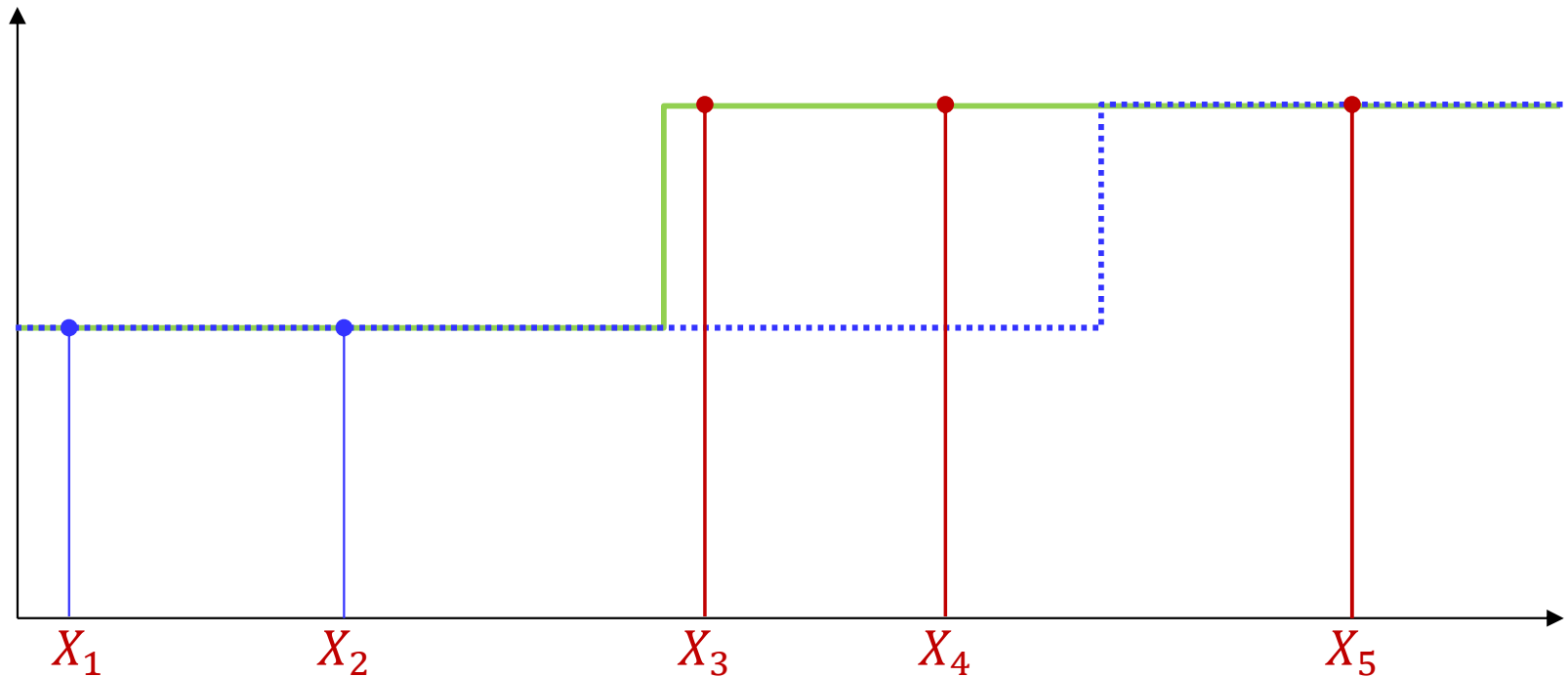
- Individual neurons' weights can change significantly without changing overall error
- The simple MLP is a flat, non-differentiable function
 - Actually, a function with 0 derivative nearly everywhere, and no derivatives at the boundaries

A second problem: What we *actually* model



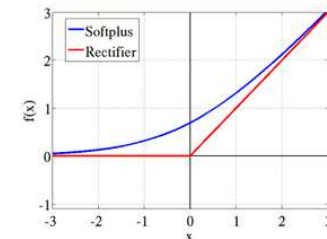
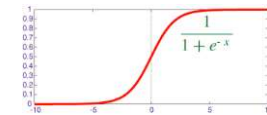
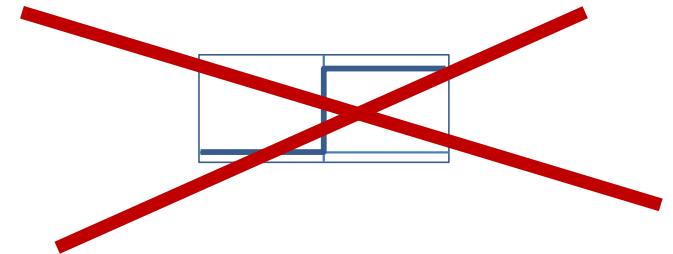
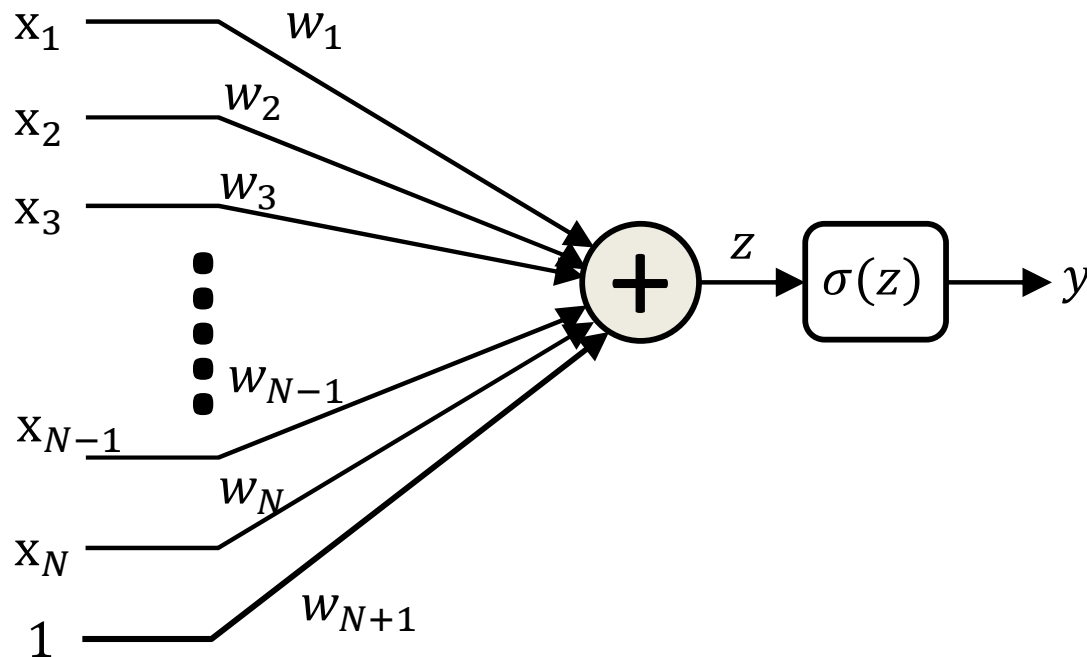
- Real-life data are rarely clean
 - Not linearly separable
 - Rosenblatt's perceptron learning rule wouldn't work in the first place

The solution



- Change our way of computing the mismatch such that modifying the classifier slightly lets us know if we are going the right way or not
 - This requires changing both, our activation functions, and the manner in which we evaluate the mismatch between the classifier output and the target output
 - Our mismatch function will now not actually count errors, but a *proxy* for it

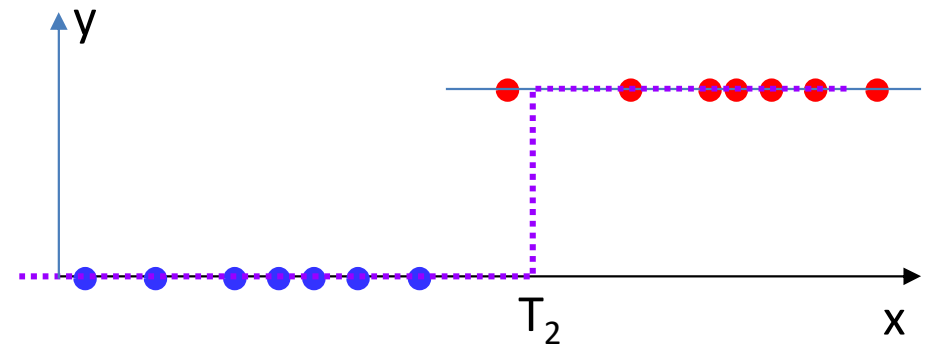
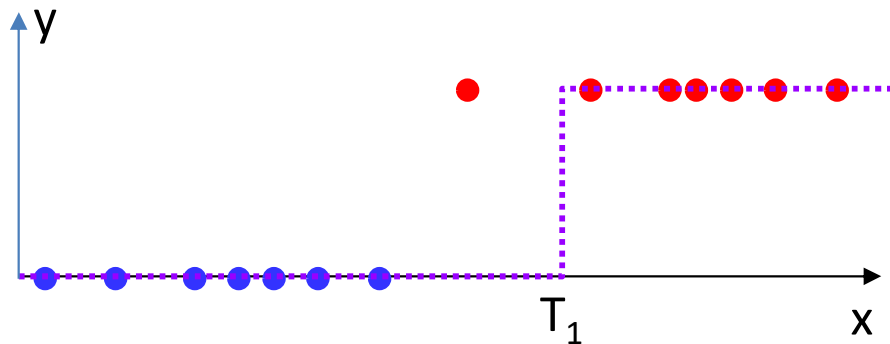
Solution: Differentiable activation



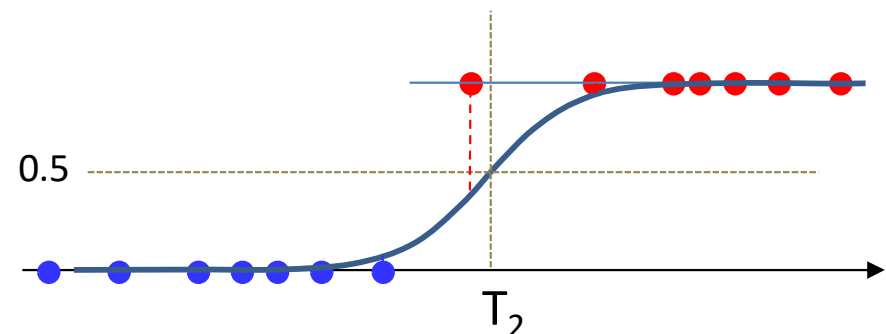
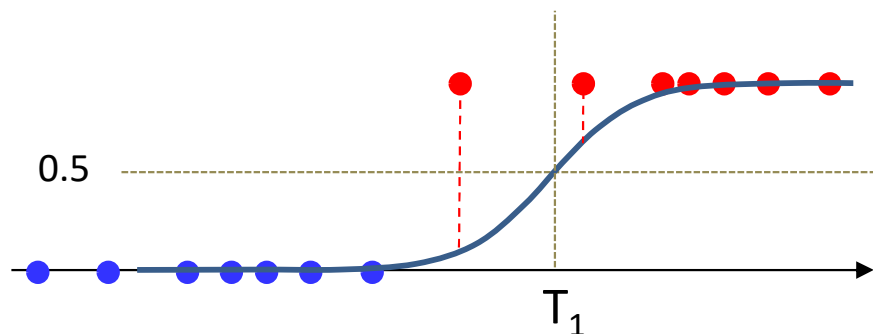
Activation functions $\sigma(z)$

- Let's make the neuron differentiable, *with non-zero derivatives over much of the input space*
 - Small changes in weight can result in non-negligible changes in output
 - This enables us to estimate the parameters using gradient descent techniques..

Differentiable Mismatch function



- Threshold activation: shifting the threshold from T_1 to T_2 does not change classification error
 - Does not indicate if moving the threshold left was good or not



- Smooth, continuously varying activation: Classification based on whether the output is greater than 0.5 or less
 - Quantify *how much* the output differs from the desired target value (0 or 1)
 - Moving the function left or right changes this quantity, even if the classification error itself doesn't change

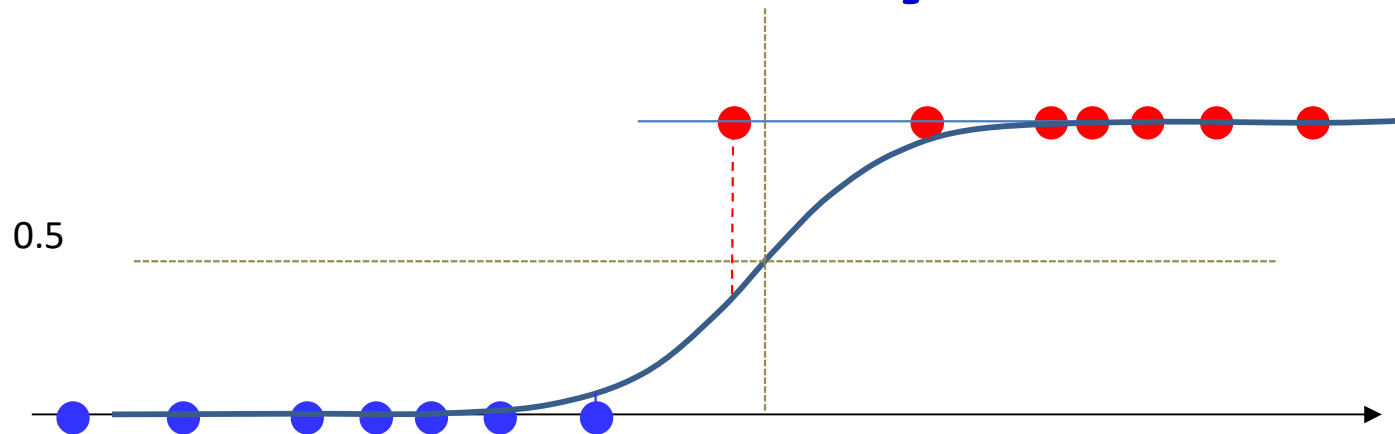
Poll 3

- Which of the following are true of the threshold activation
 - Increasing (or decreasing) the threshold will not change the overall classification error unless the threshold moves past a misclassified training sample
 - We cannot know if a change (increase or decrease) of the threshold moves it in the correct direction that will result in a net decrease in classification error
 - The derivative of the classification error with respect to the threshold gives us an indication of whether to increase or decrease the threshold
- Which of the following are true of the continuous activation (sigmoid)
 - Shifting the function left or right will not change the overall classification error unless the crossover point (where the function crosses 0.5) moves past a misclassified training sample
 - Shifting the function will change the total distance of the value of the function from its target value at the training instances
 - The derivative of the total distance with respect to the shift of the function gives us an indication of which direction to shift the function to improve classification error

Poll 3

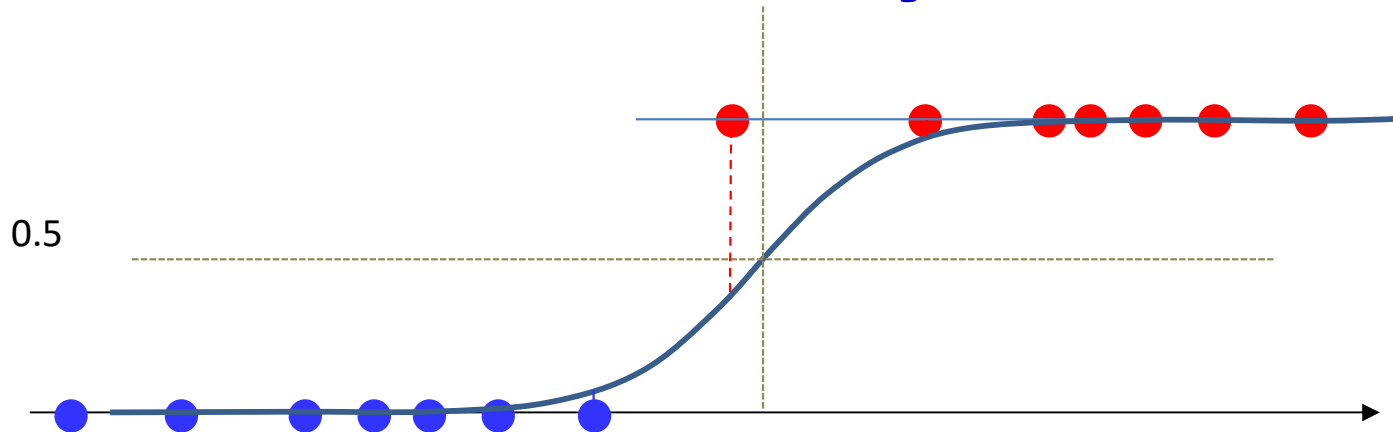
- Which of the following are true of the threshold activation
 - **Increasing (or decreasing) the threshold will not change the overall classification error unless the threshold moves past a misclassified training sample**
 - **We cannot know if a change (increase or decrease) of the threshold moves it in the correct direction that will result in a net decrease in classification error**
 - The derivative of the classification error with respect to the threshold gives us an indication of whether to increase or decrease the threshold
- Which of the following are true of the continuous activation (sigmoid)
 - **Shifting the function left or right will not change the overall classification error unless the crossover point (where the function crosses 0.5) moves past a misclassified training sample**
 - **Shifting the function will change the total distance of the value of the function from its target value at the training instances**
 - **The derivative of the total distance with respect to the shift of the function gives us an indication of which direction to shift the function to improve classification error**

The two key requirements for learnability



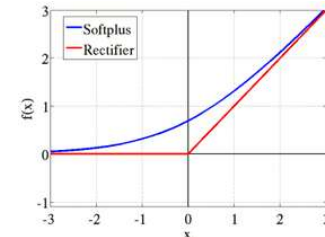
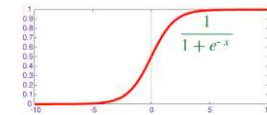
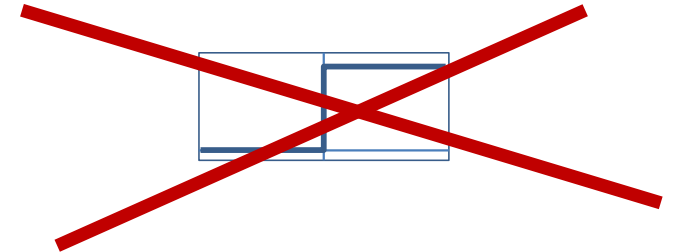
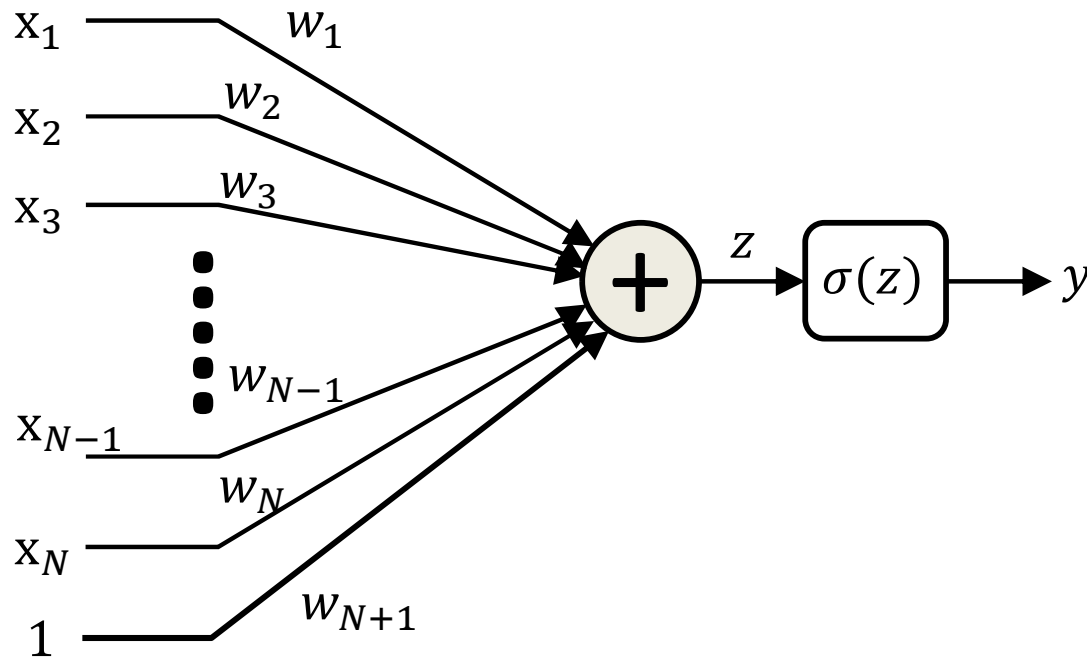
- Continuously varying activation
 - Differentiable
- Continuously varying error function
 - Also differentiable

The two key requirements for learnability



- Continuously varying activation
 - Differentiable
- Continuously varying error function
 - Also differentiable

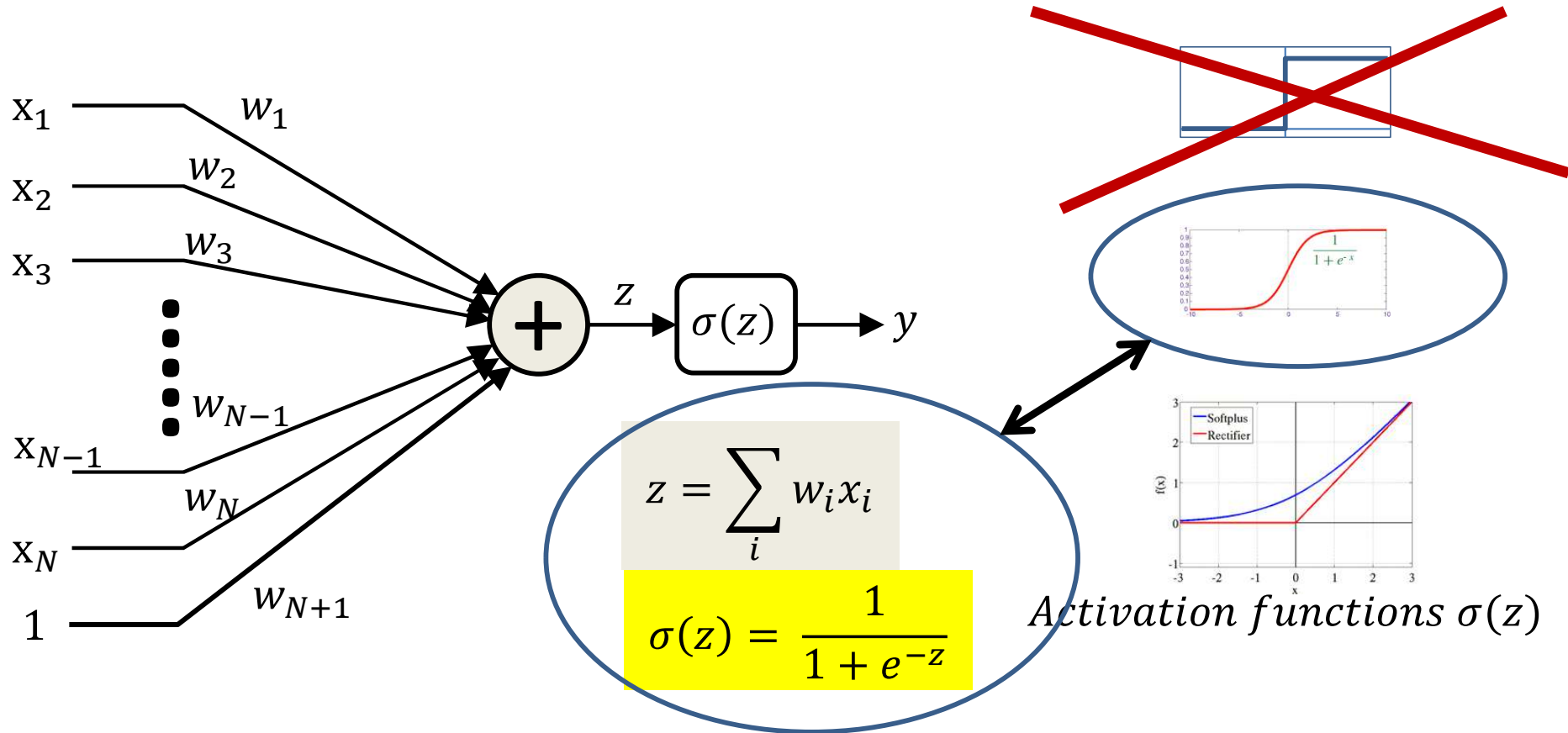
Continuous Activations



Activation functions $\sigma(z)$

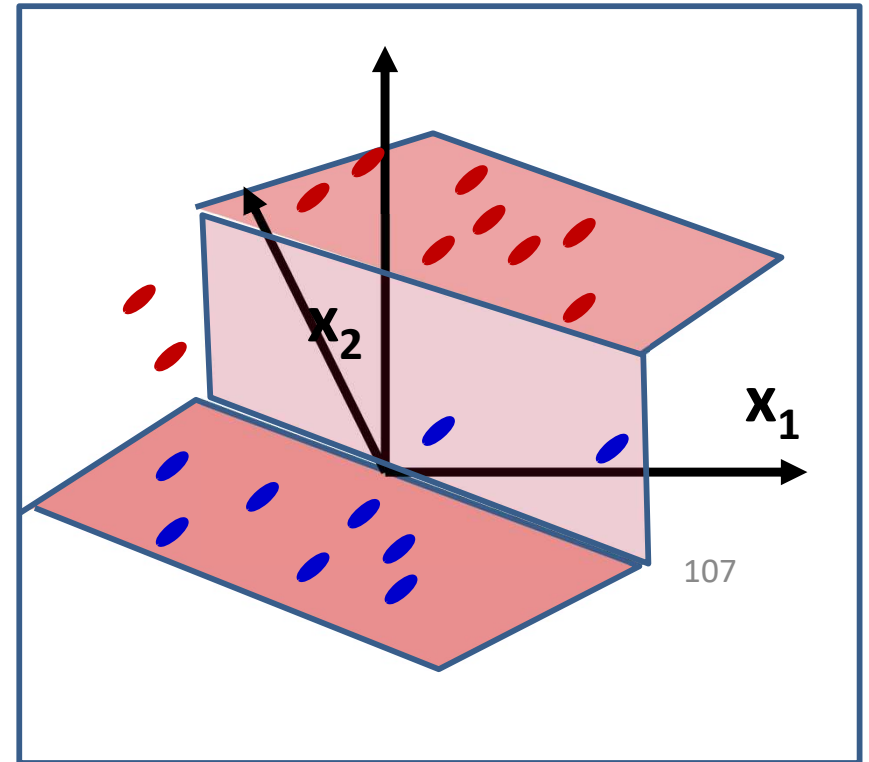
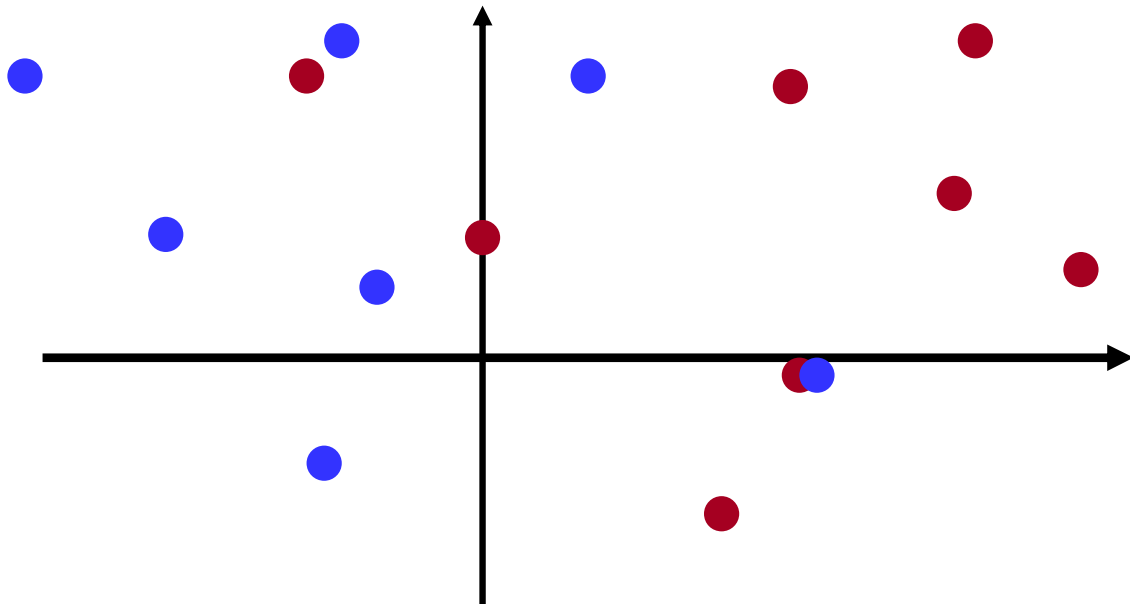
- Replace the threshold activation with continuous graded activations
 - E.g. RELU, softplus, sigmoid etc.
- The activations are *differentiable* almost everywhere
 - Have derivatives almost everywhere
 - And have “subderivatives” at non-differentiable corners
 - Bounds on the derivative that can substitute for derivatives in our setting
 - More on these later

The sigmoid activation is special



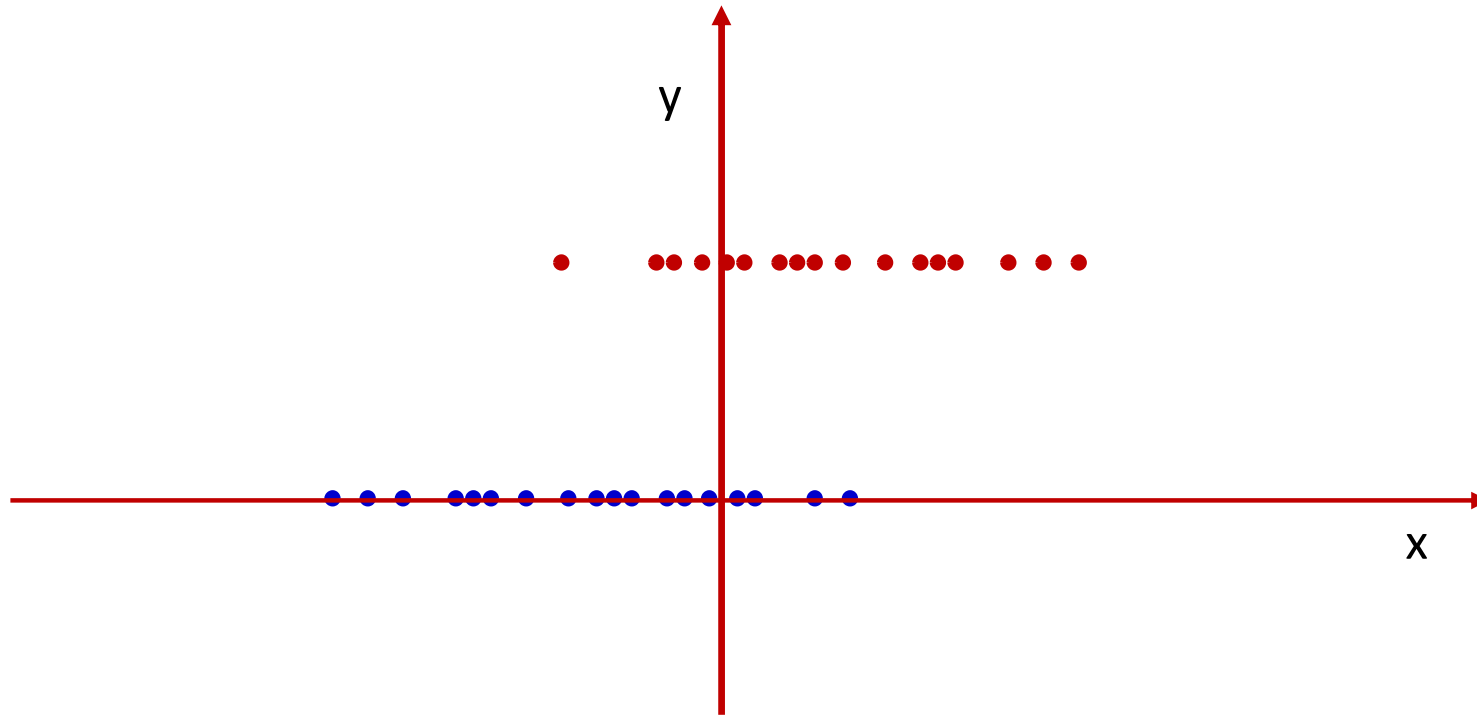
- This particular one has a nice interpretation
- It can be interpreted as $P(y = 1|x)$

Non-linearly separable data



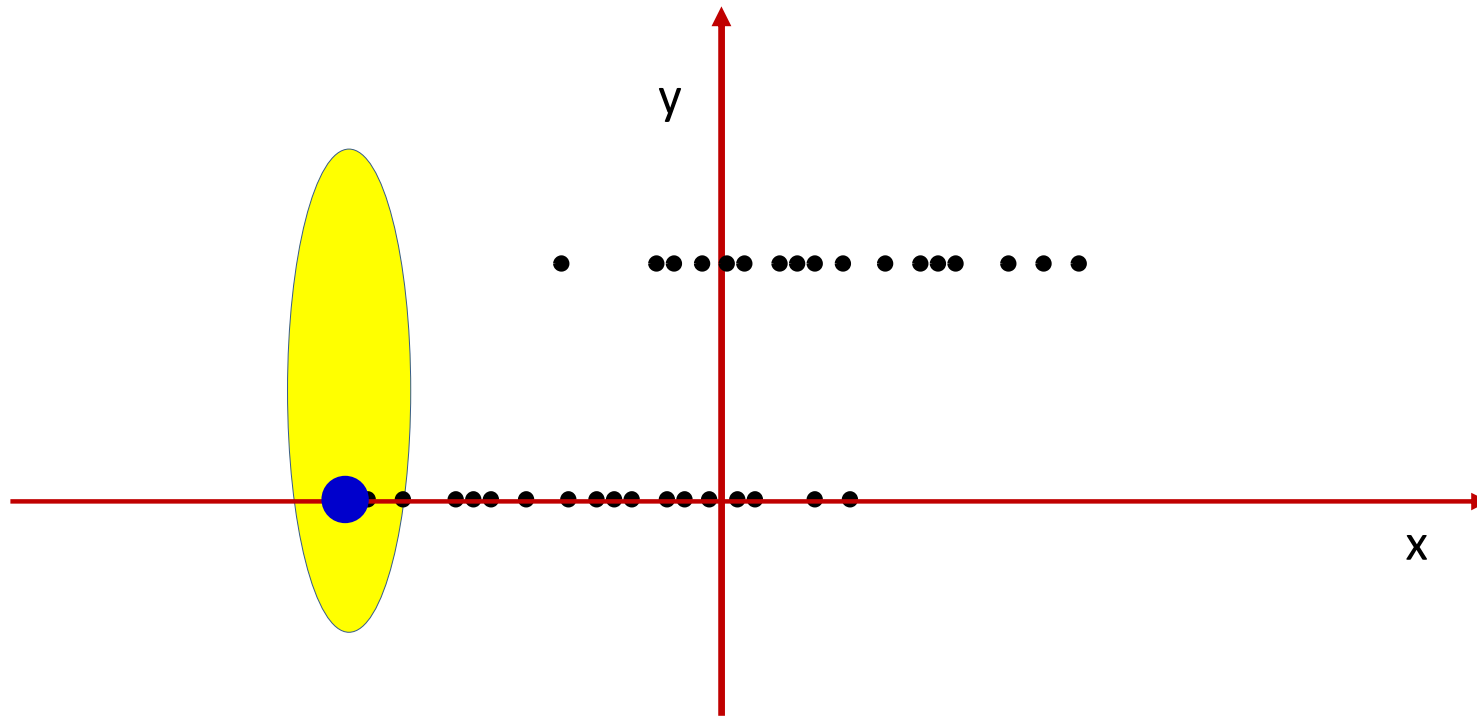
- Two-dimensional example
 - Blue dots (on the floor) on the “red” side
 - Red dots (suspended at $Y=1$) on the “blue” side
 - No line will cleanly separate the two colors

Non-linearly separable data: 1-D example



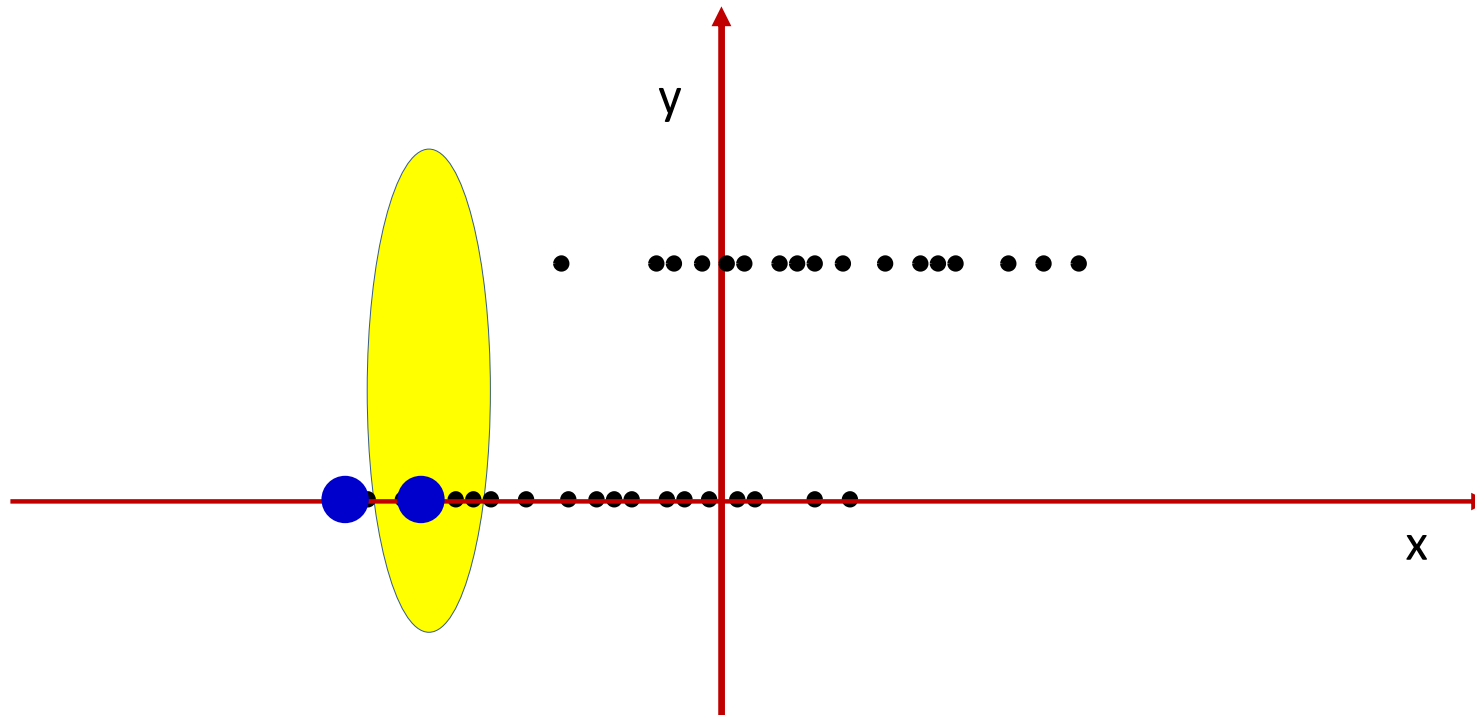
- One-dimensional example for visualization
 - All (red) dots at $Y=1$ represent instances of class $Y=1$
 - All (blue) dots at $Y=0$ are from class $Y=0$
 - The data are not linearly separable
 - In this 1-D example, a linear separator is a threshold
 - No threshold will cleanly separate red and blue dots

The *probability* of $y=1$



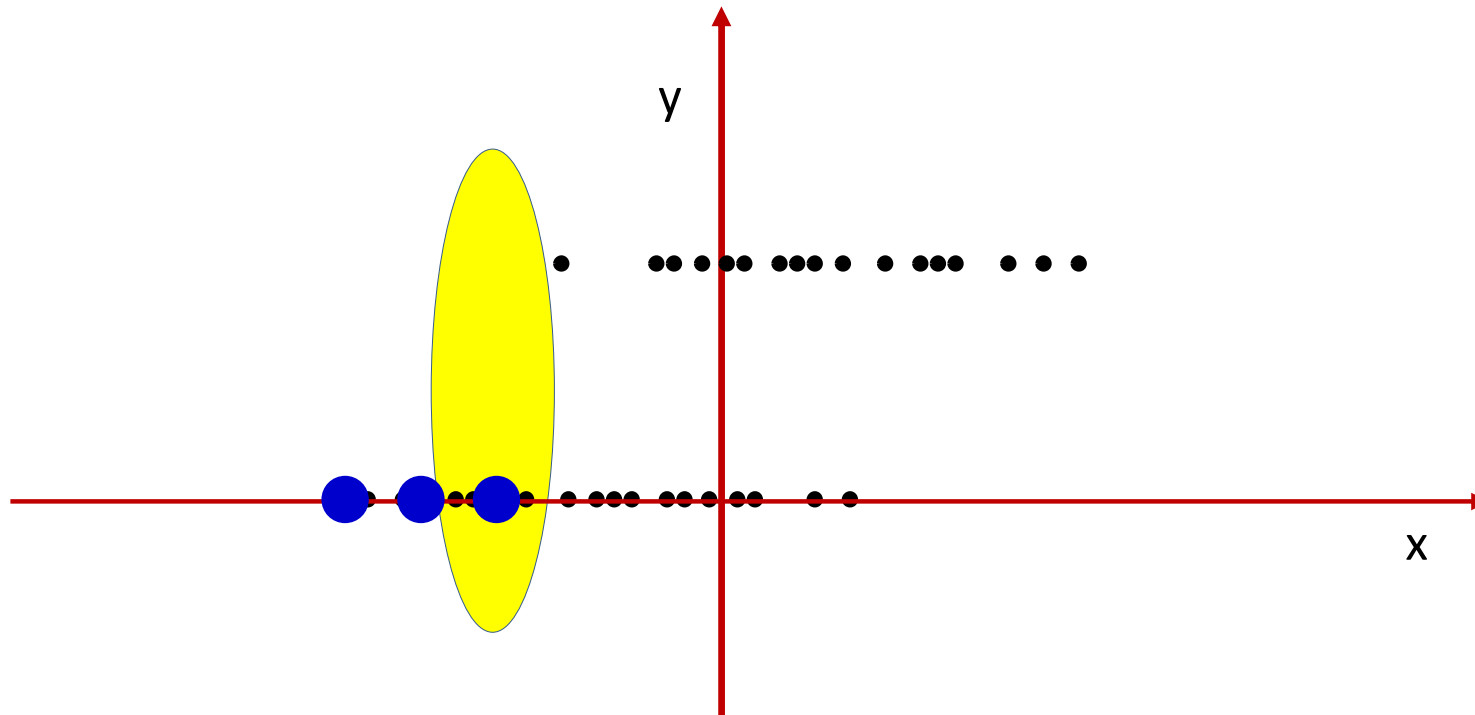
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of $Y=1$ at that point

The *probability* of $y=1$



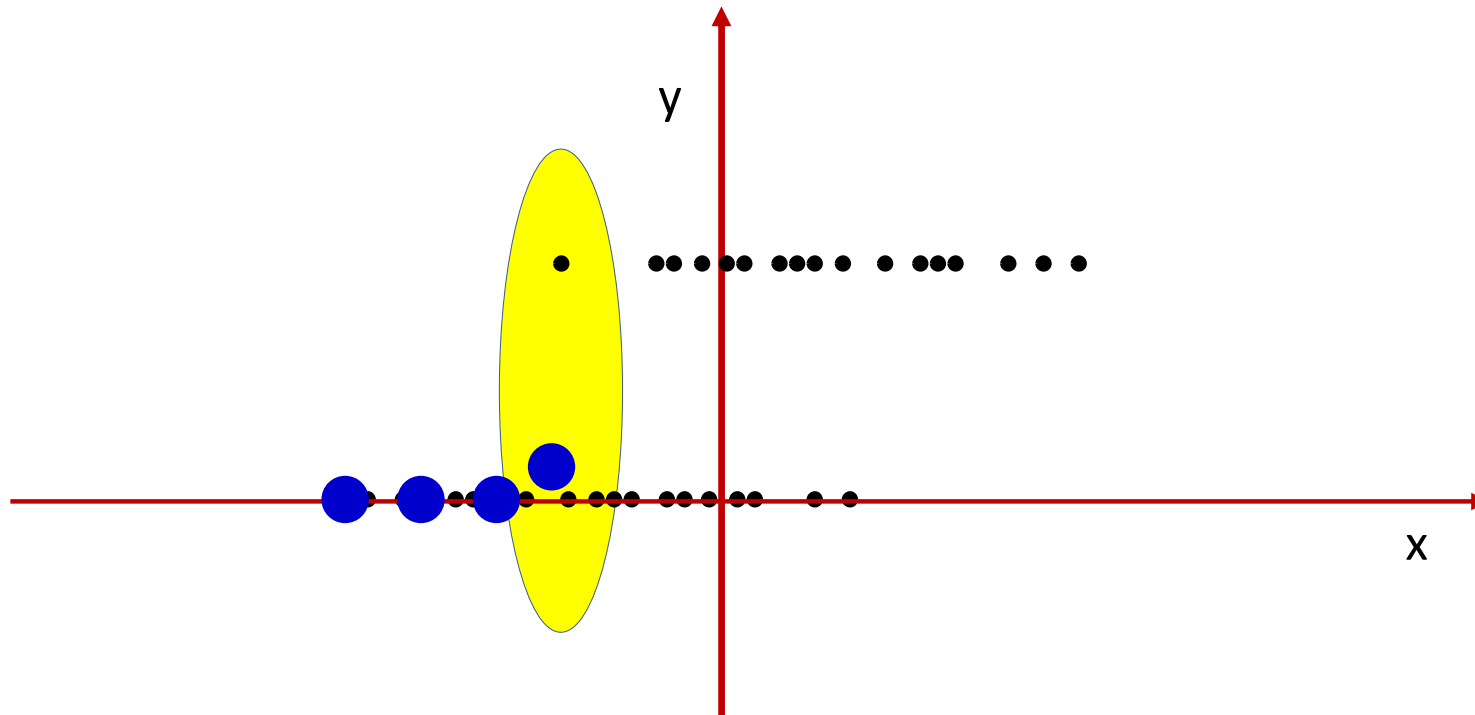
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



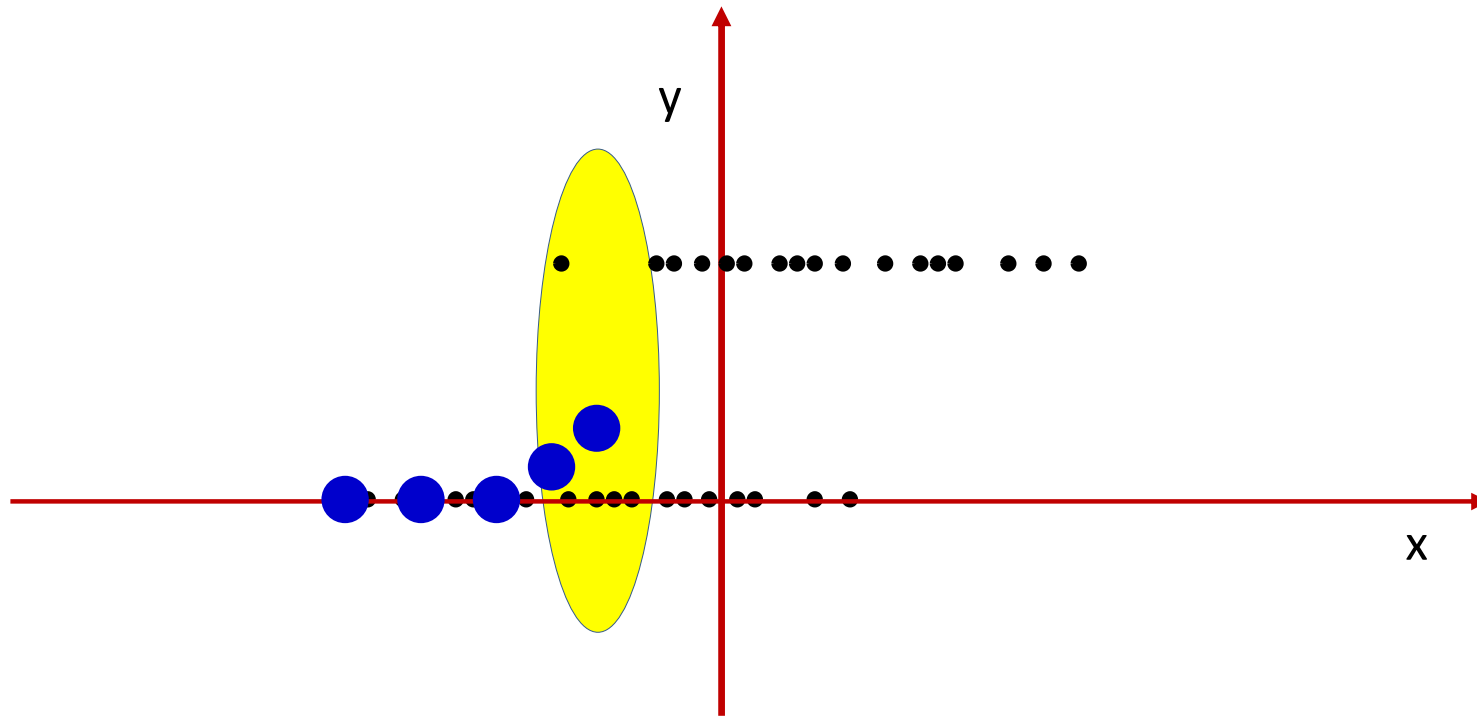
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



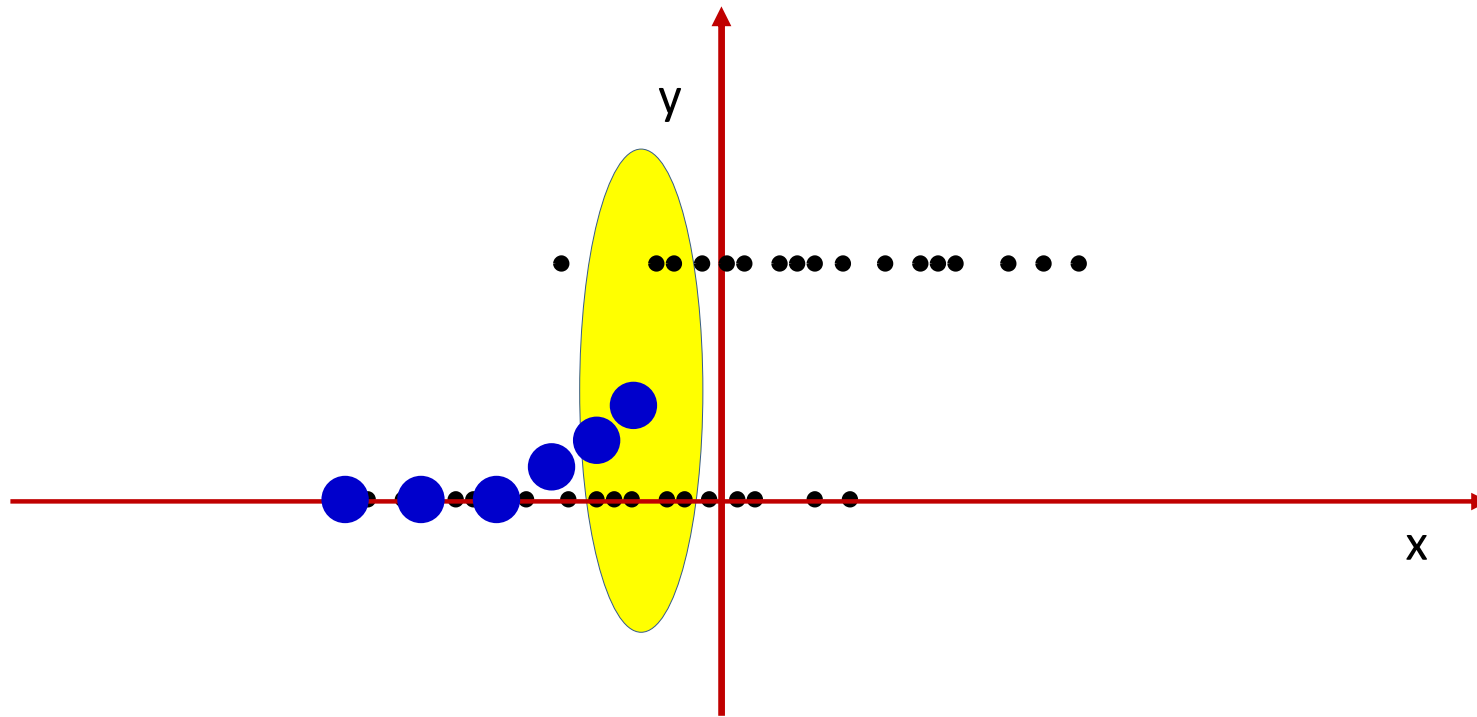
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



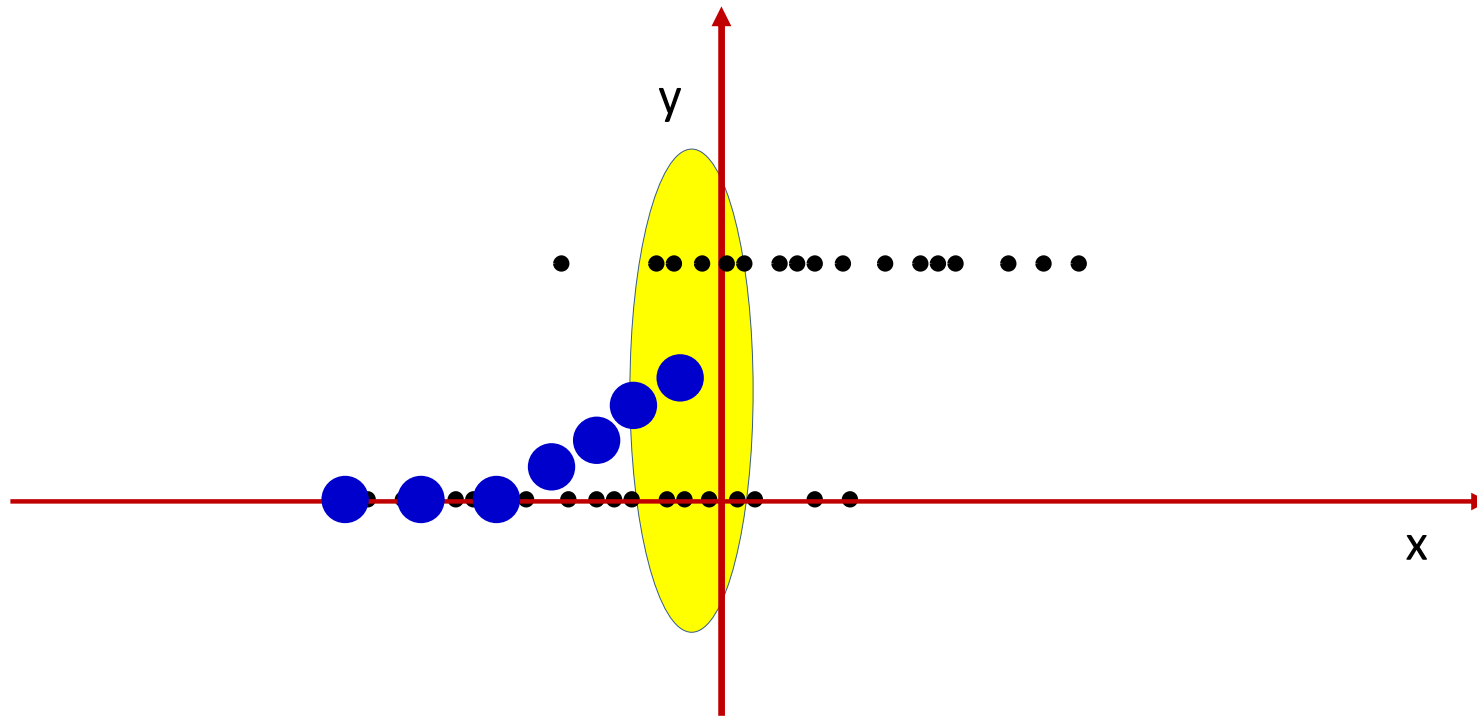
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



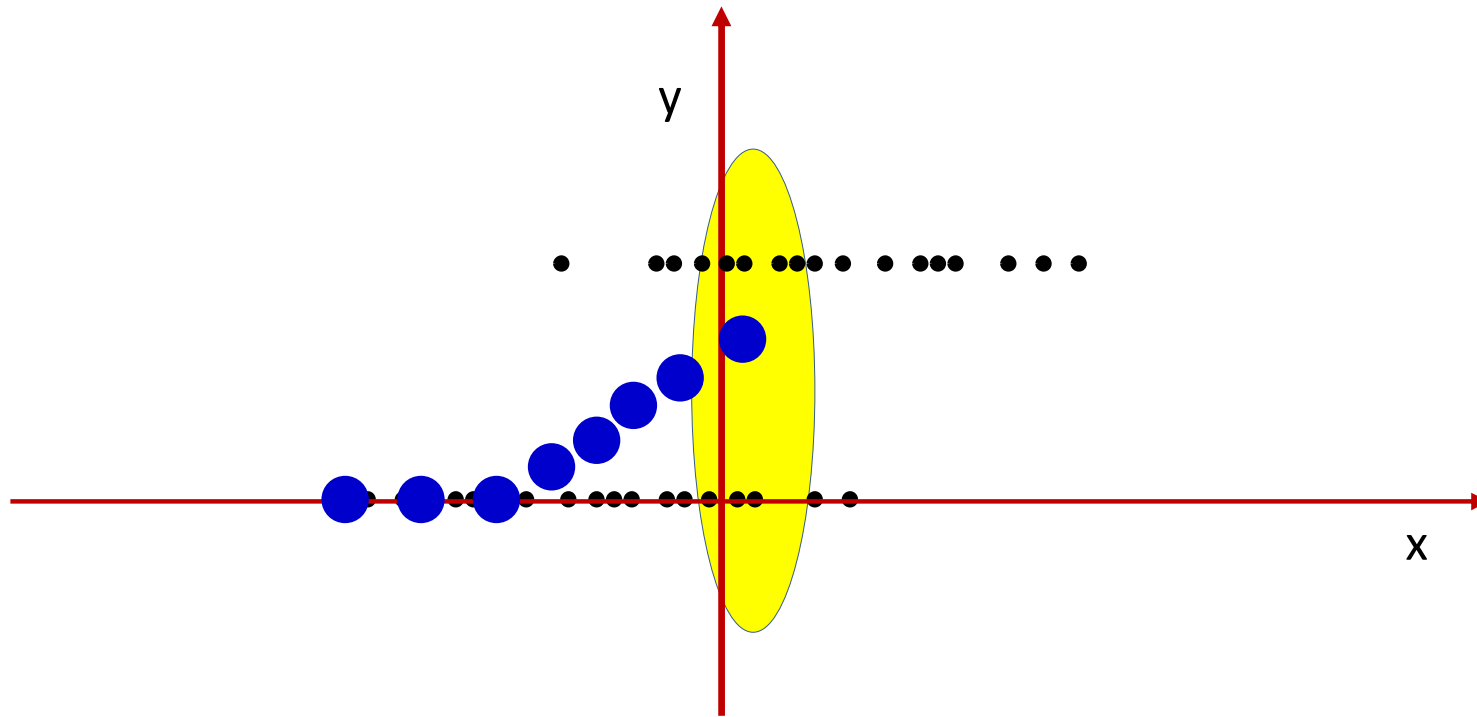
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



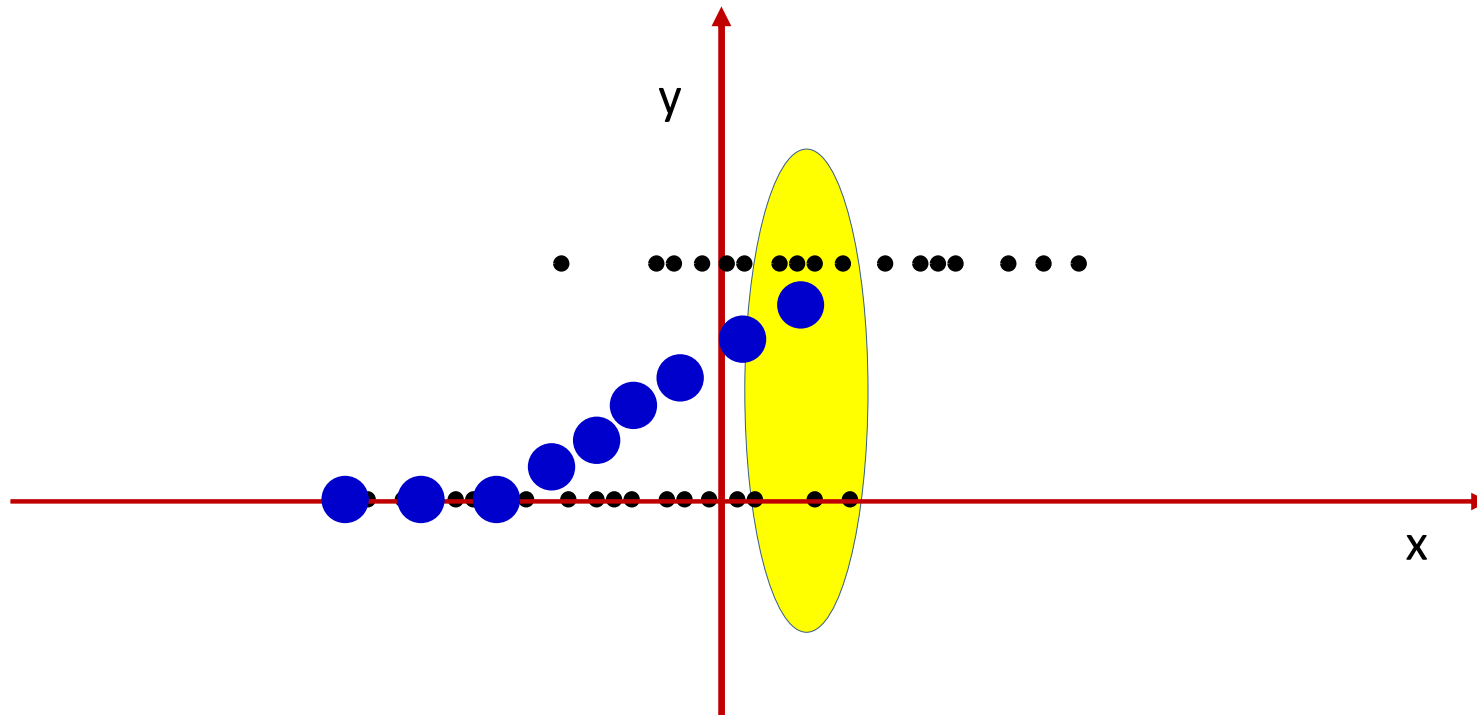
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



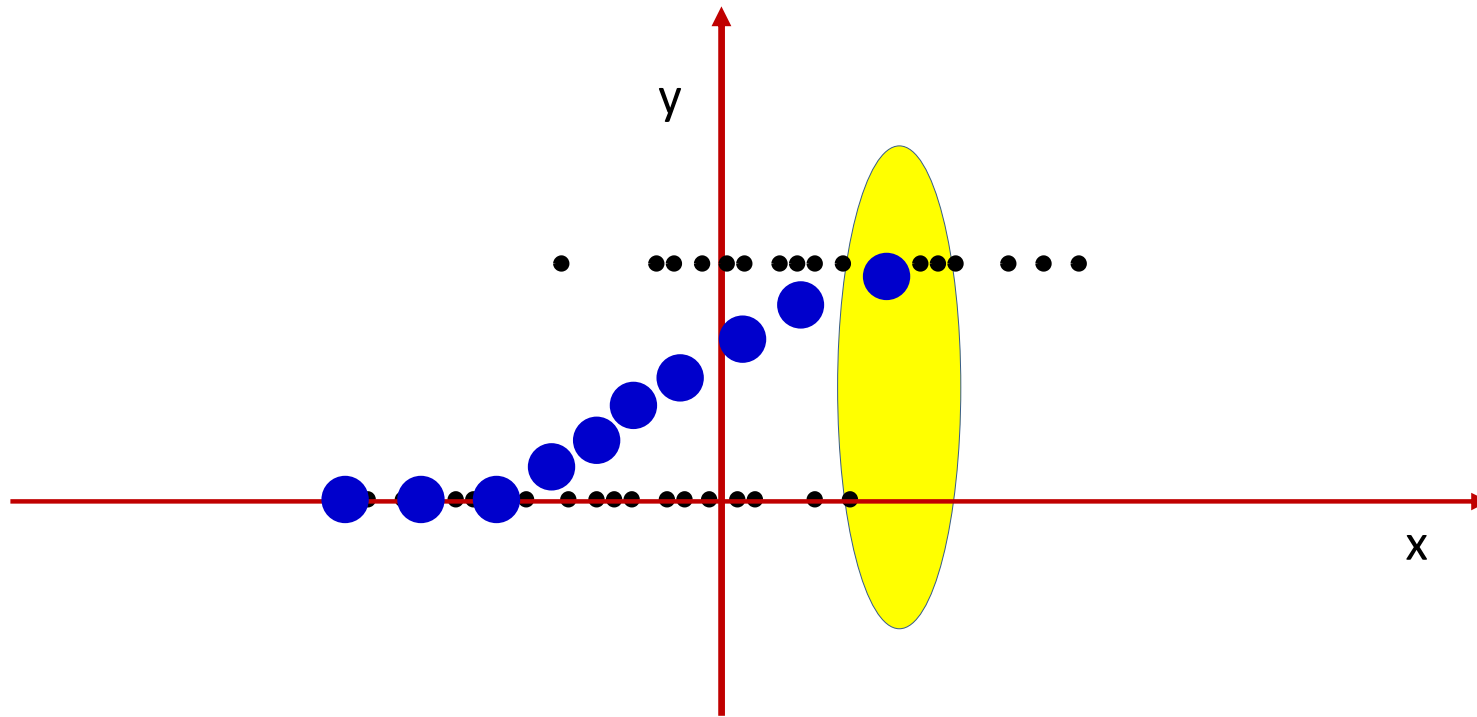
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



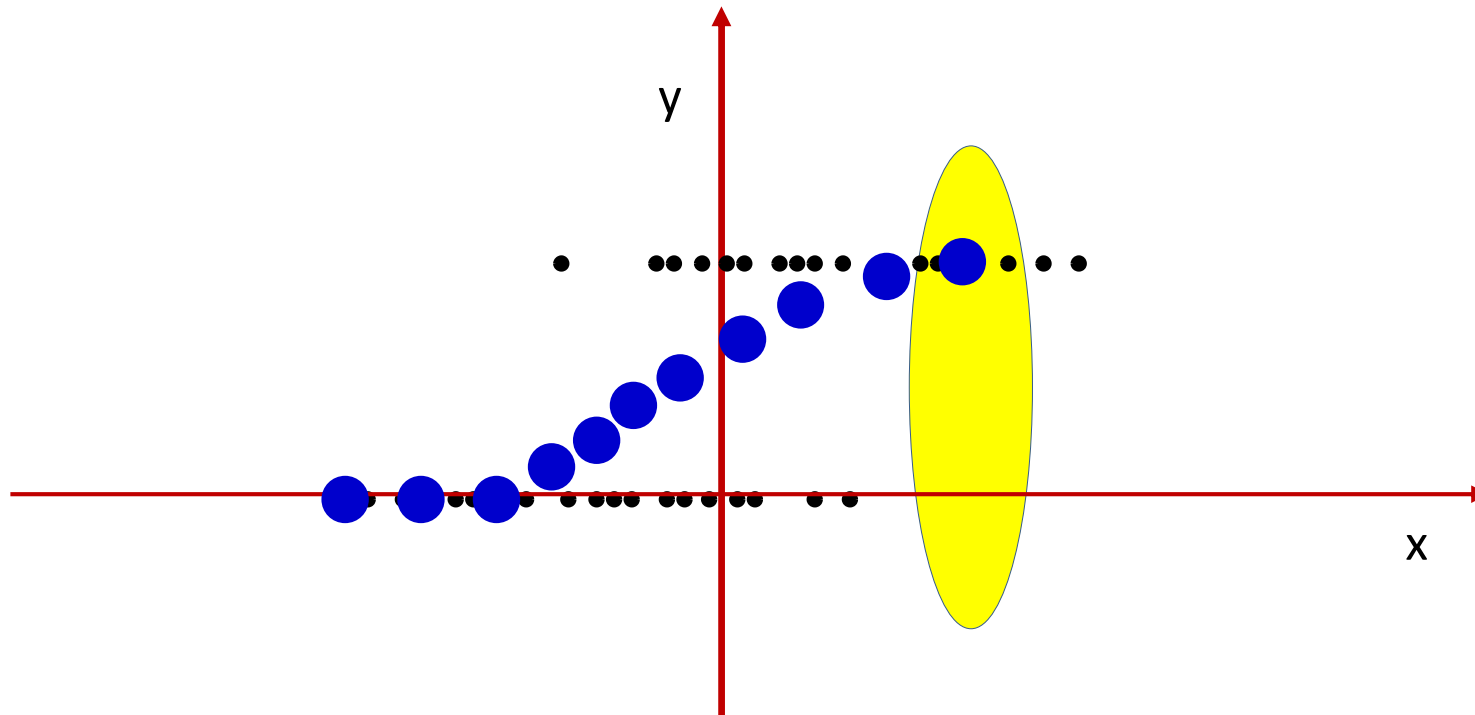
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



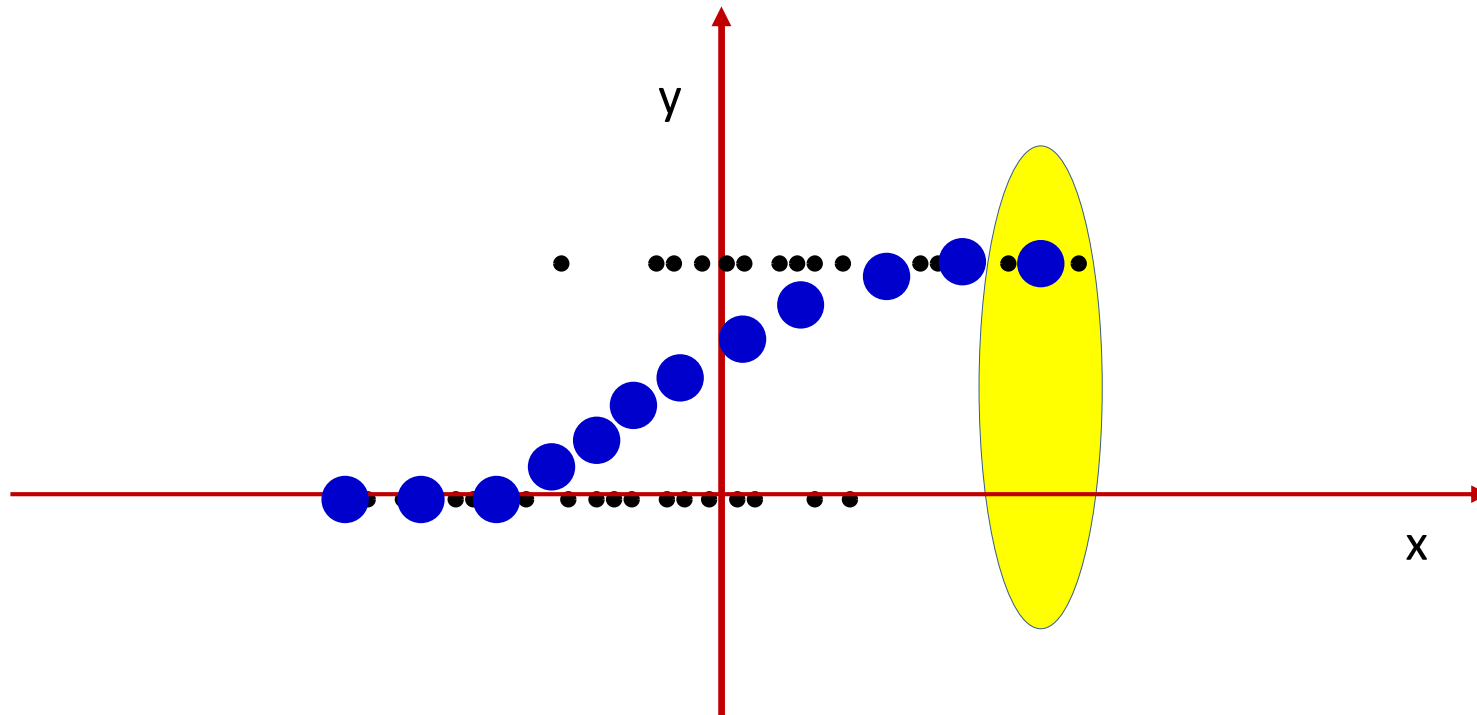
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



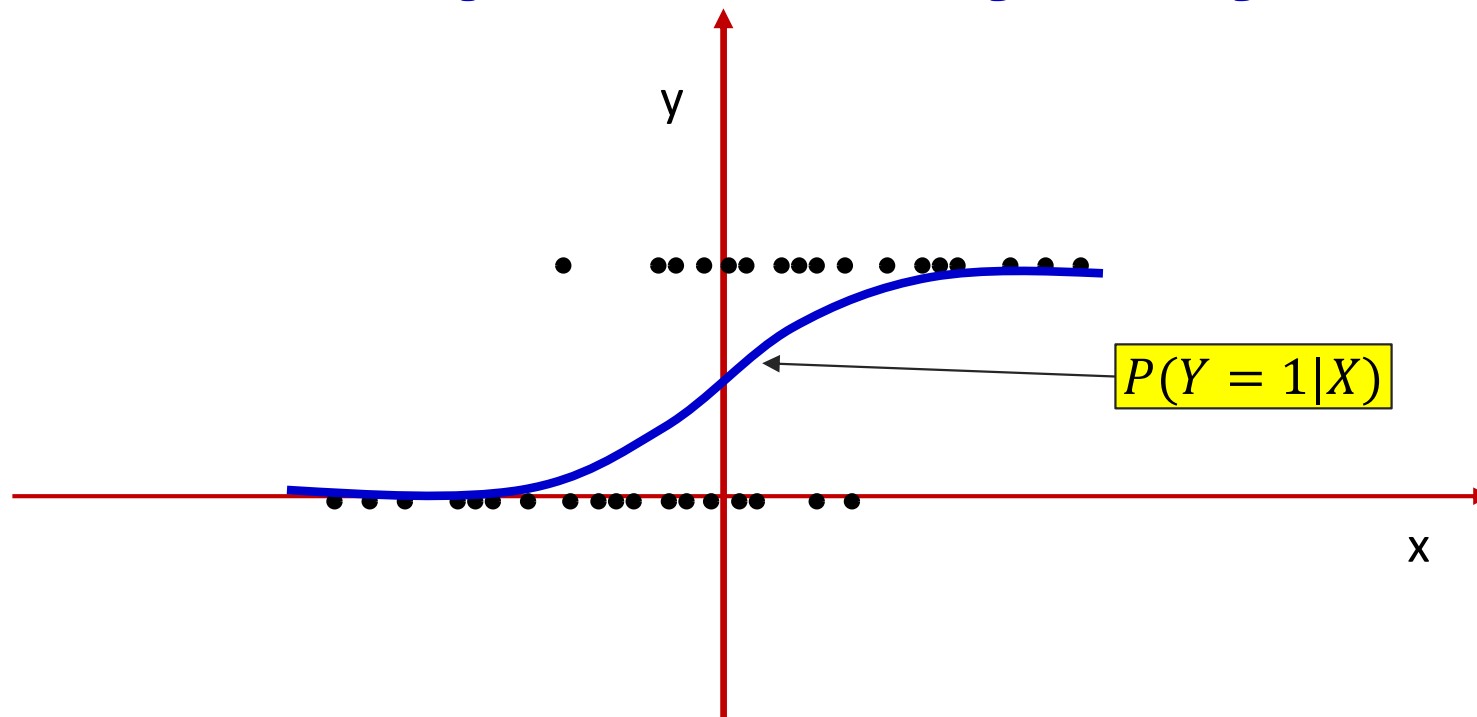
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



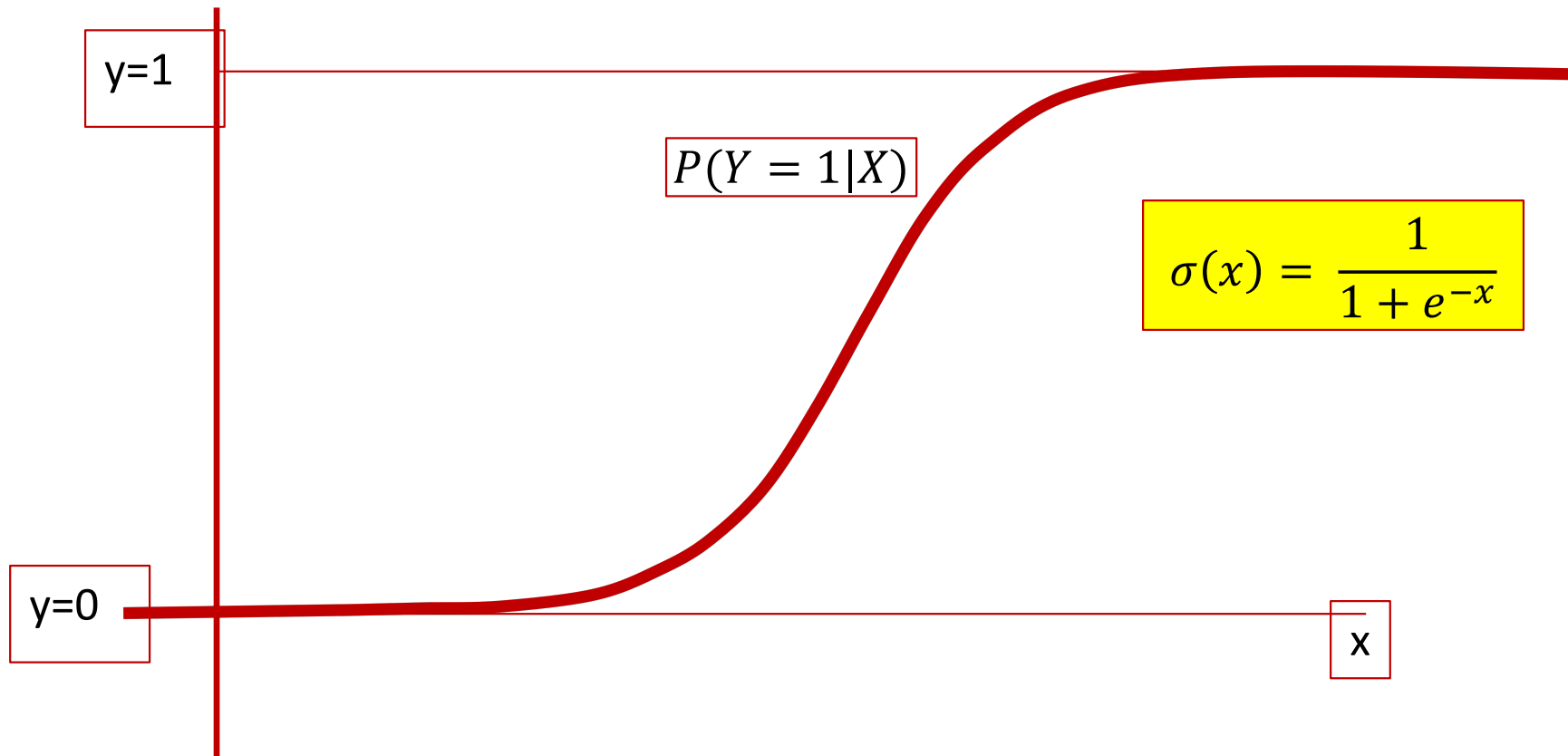
- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

The *probability* of $y=1$



- Consider this differently: at each point look at a small window around that point
- Plot the average value within the window
 - This is an approximation of the *probability* of 1 at that point

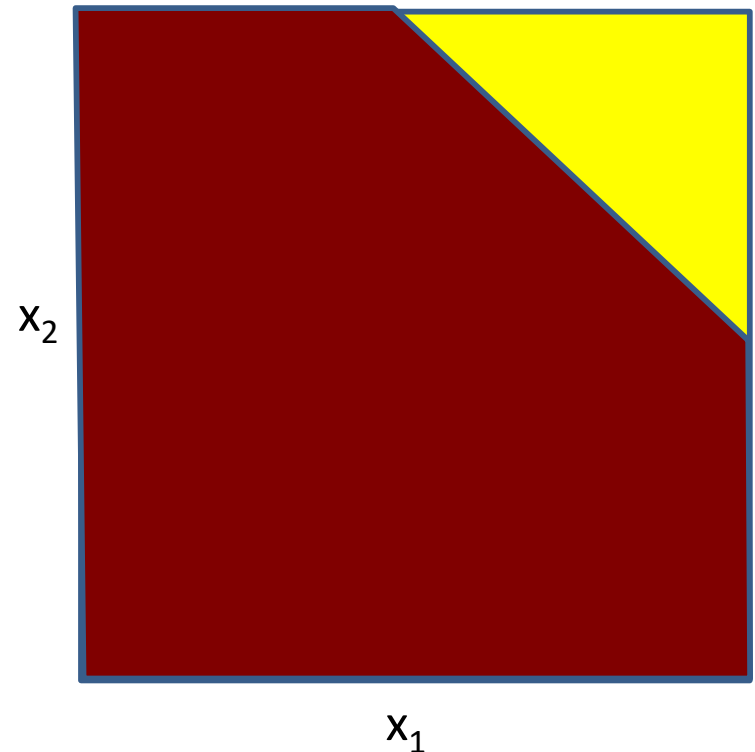
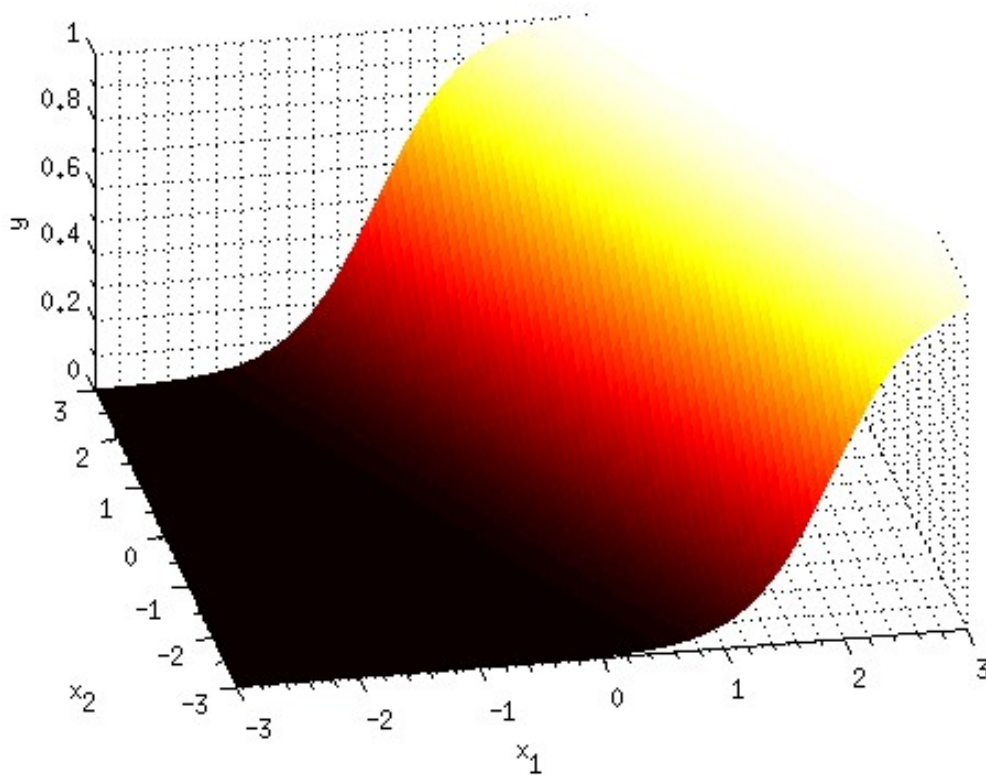
The logistic regression model



- Class 1 becomes increasingly probable going left to right
 - Very typical in many problems

Logistic regression

Decision: $y > 0.5$?



When X is a 2-D variable

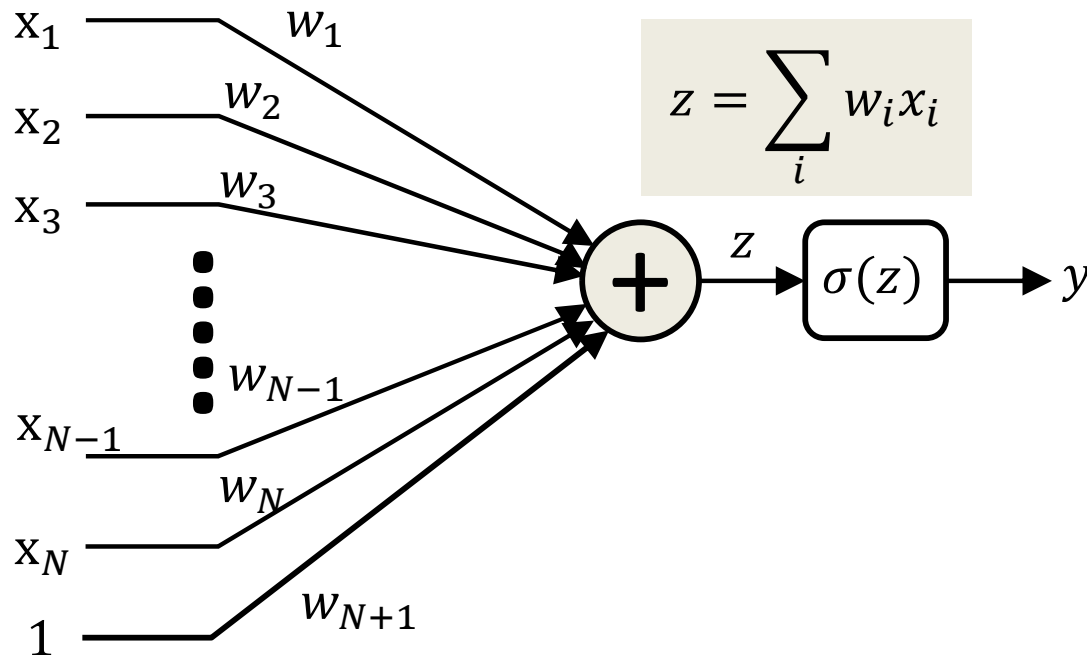
$$P(Y = 1|X) = \frac{1}{1 + \exp(-\sum_i w_i x_i - b)}$$

- This is the perceptron with a sigmoid activation
 - It actually computes the *probability* that the input belongs to class 1

Perceptrons and probabilities

- We will return to the fact that perceptrons with sigmoidal activations actually model class probabilities in a later lecture
- But for now moving on..

Perceptrons with differentiable activation functions



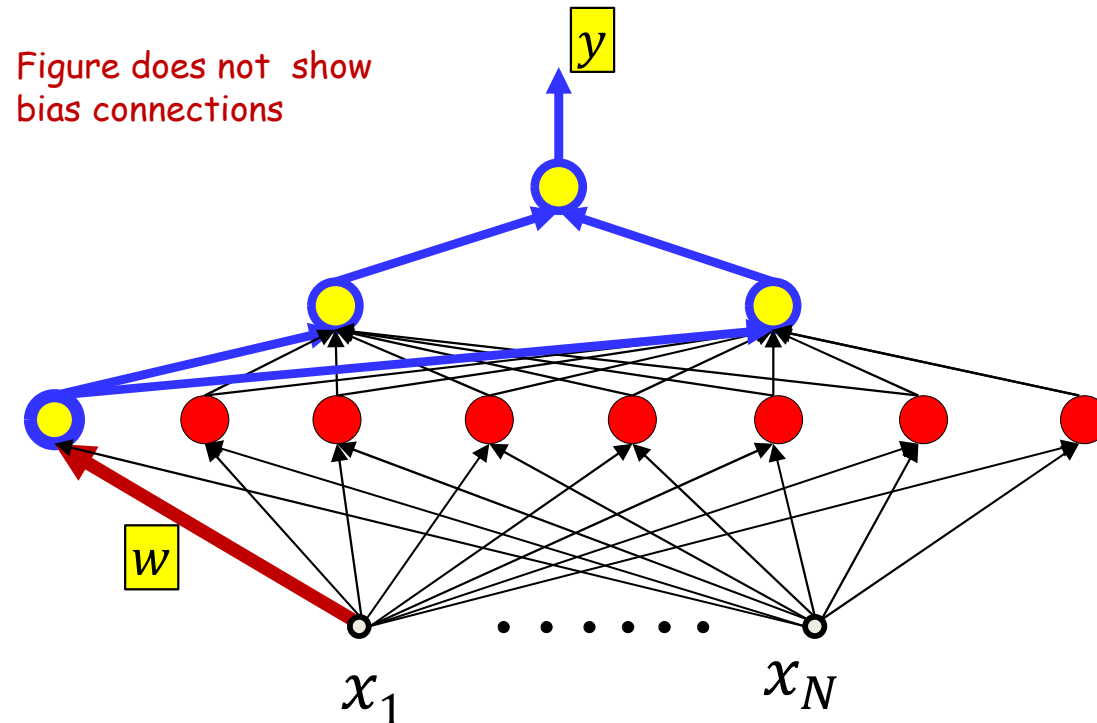
$$\frac{dy}{dz} = \sigma'(z)$$

$$\frac{dy}{dw_i} = \frac{dy}{dz} \frac{dz}{dw_i} = \sigma'(z) x_i$$

$$\frac{dy}{dx_i} = \frac{dy}{dz} \frac{dz}{dx_i} = \sigma'(z) w_i$$

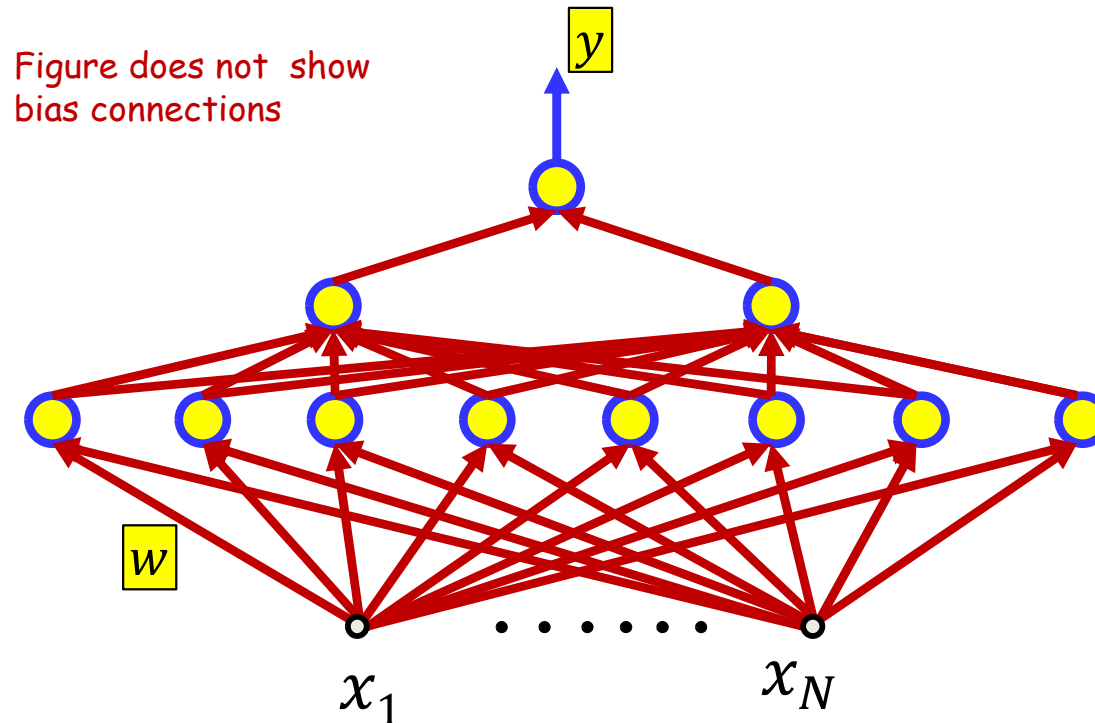
- $\sigma(z)$ is a differentiable function of z
 - $\frac{d\sigma(z)}{dz}$ is well-defined and finite for all z
- Using the chain rule, y is a differentiable function of both inputs x_i and weights w_i
- This means that we can compute the change in the output for *small* changes in either the input or the weights

Overall network is differentiable



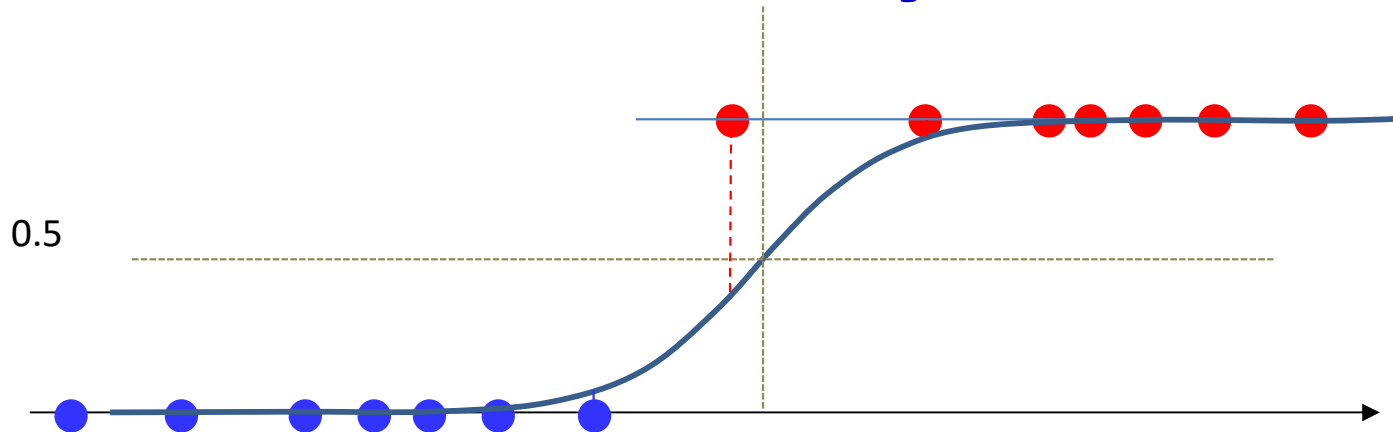
- Every individual perceptron is differentiable w.r.t its inputs and its weights (including “bias” weight)
 - Small changes in the parameters result in measurable changes in output
- Using the chain rule can compute how small perturbations of a parameter change the output of the network
 - The network output is differentiable with respect to the parameter

Overall function is differentiable



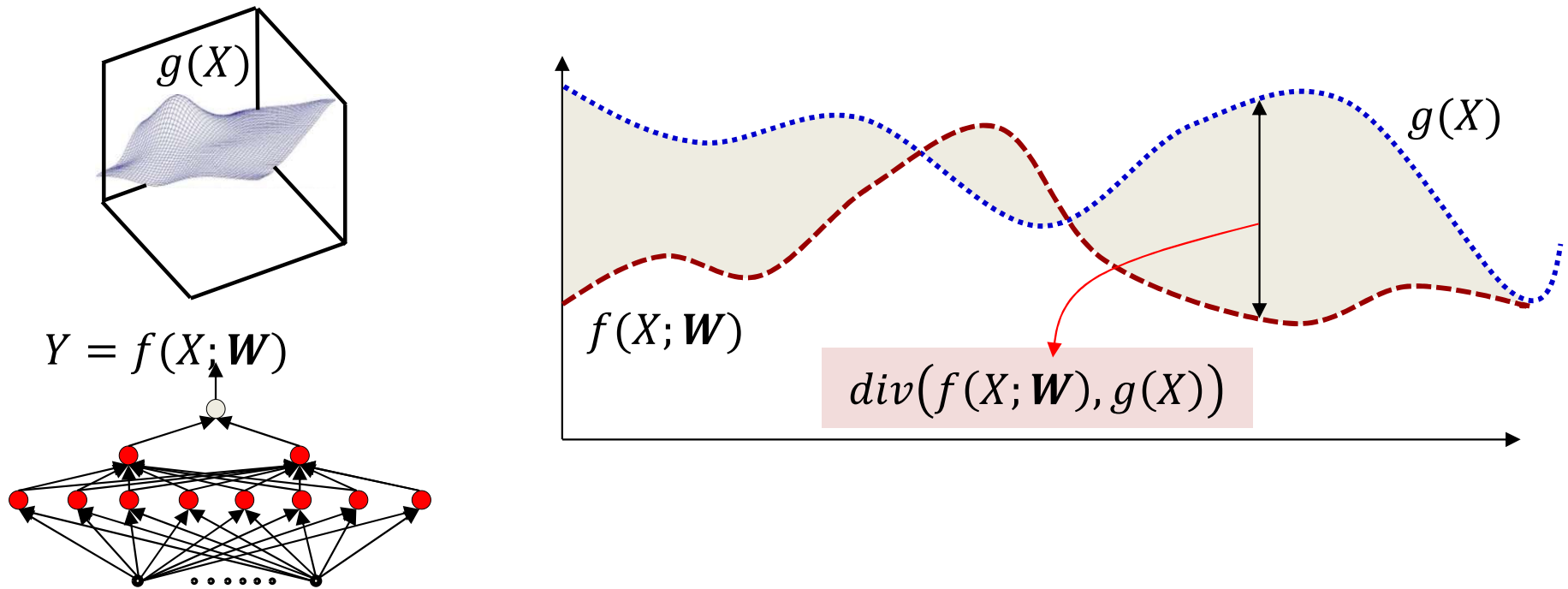
- By extension, the overall function is differentiable w.r.t every parameter in the network
 - We can compute how small changes in the parameters change the output
 - For non-threshold activations the derivative are finite and generally non-zero
- We will derive the actual derivatives using the chain rule later

The two key requirements for learnability



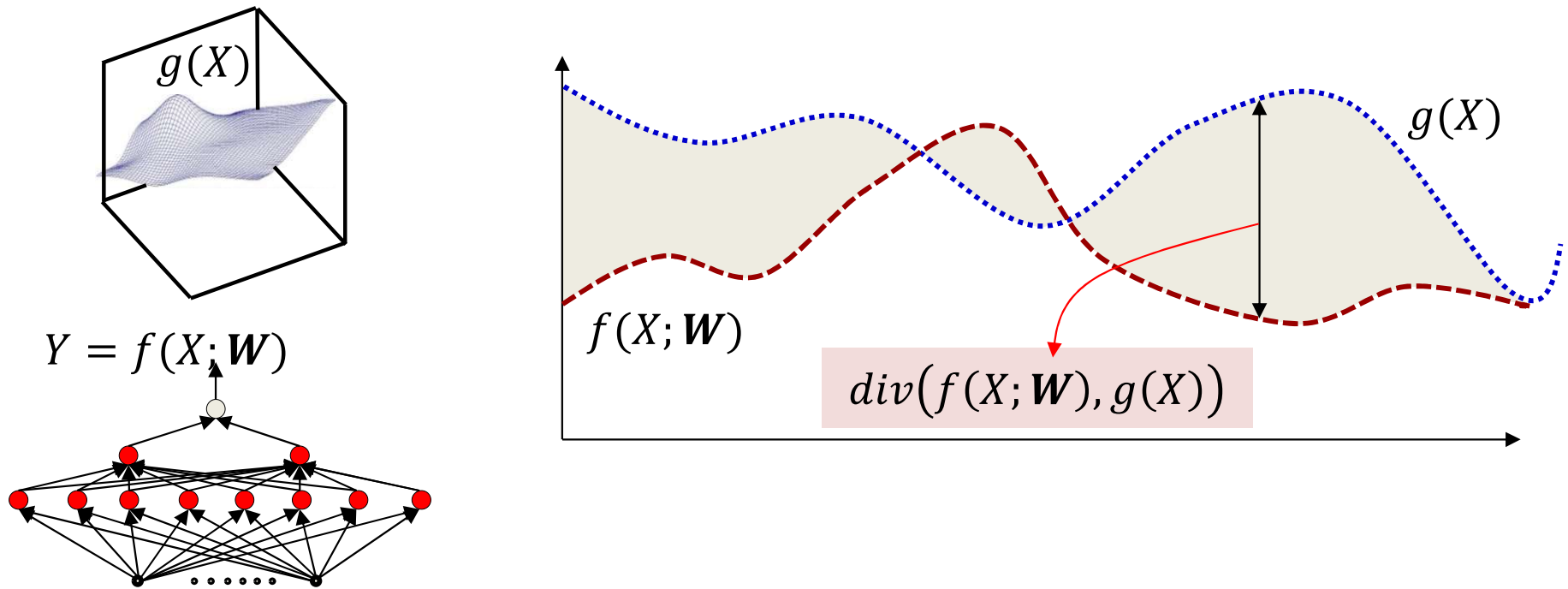
- Continuously varying activation
 - Differentiable
- Continuously varying error function
 - Also differentiable

The error function



- Define a *divergence* function $div(f(X; \mathbf{W}), g(X))$ with the following properties
 - $div(f(X; \mathbf{W}), g(X)) = 0$ if $f(X; \mathbf{W}) = g(X)$
 - $div(f(X; \mathbf{W}), g(X)) > 0$ if $f(X; \mathbf{W}) \neq g(X)$
 - $div(f, g)$ is differentiable with respect to f
- The divergence function quantifies mismatch between the network output and target function
 - For classification, this is usually not the classification error but a proxy to it

How to learn a network

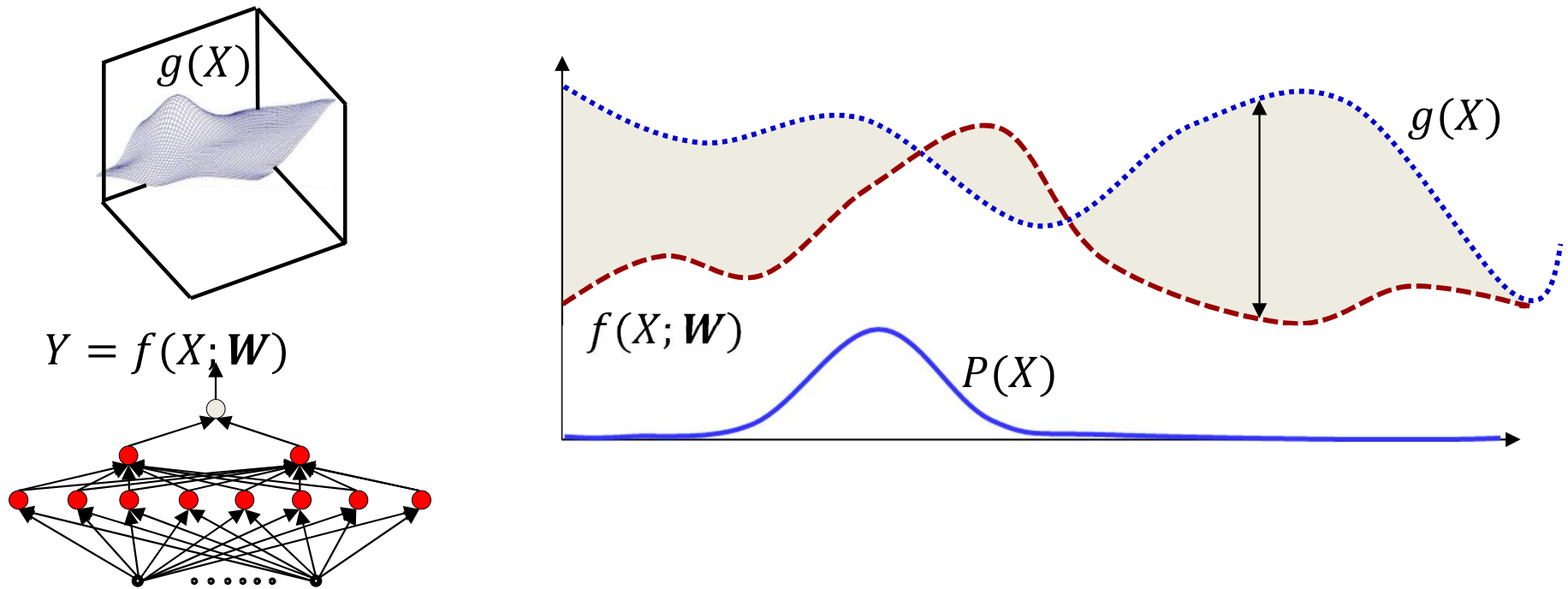


- When $f(X; W)$ has the capacity to exactly represent $g(X)$

$$\widehat{W} = \operatorname{argmin}_W \int_X div(f(X; W), g(X)) dX$$

- $div()$ is a divergence function that goes to zero when $f(X; W) = g(X)$

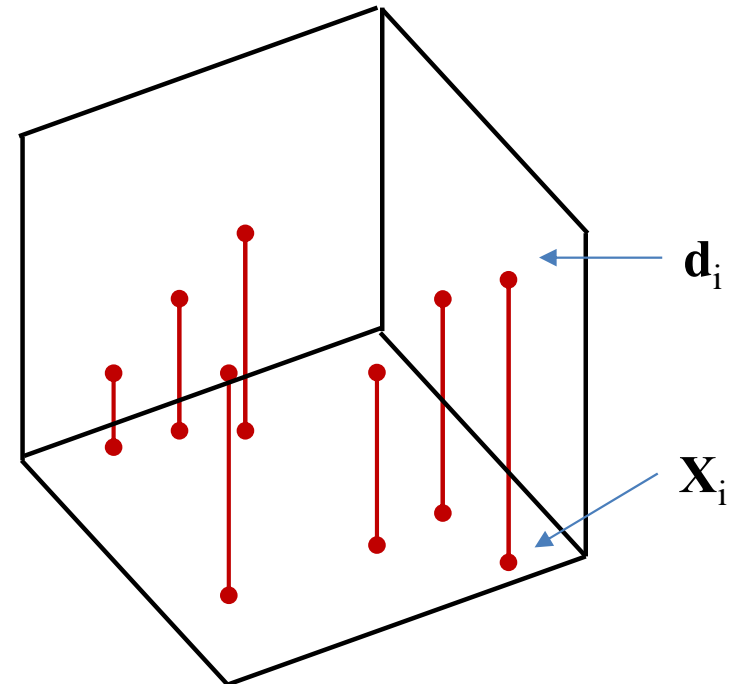
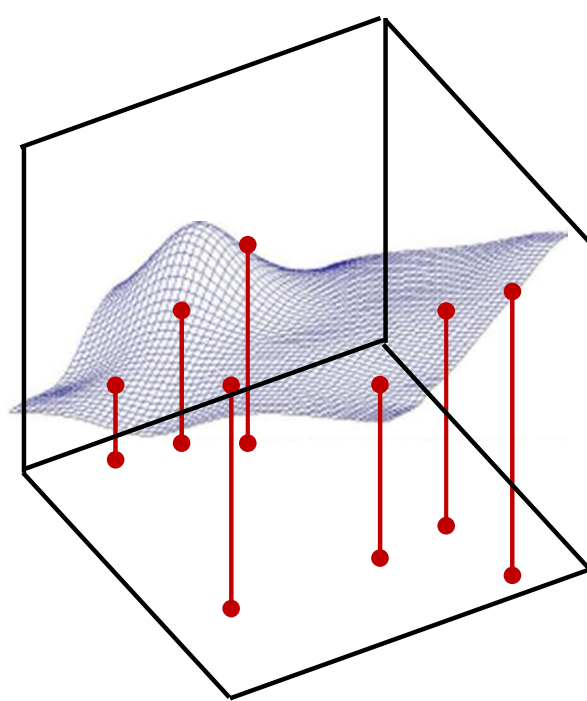
Minimizing *expected* divergence



- More generally, assuming X is a random variable

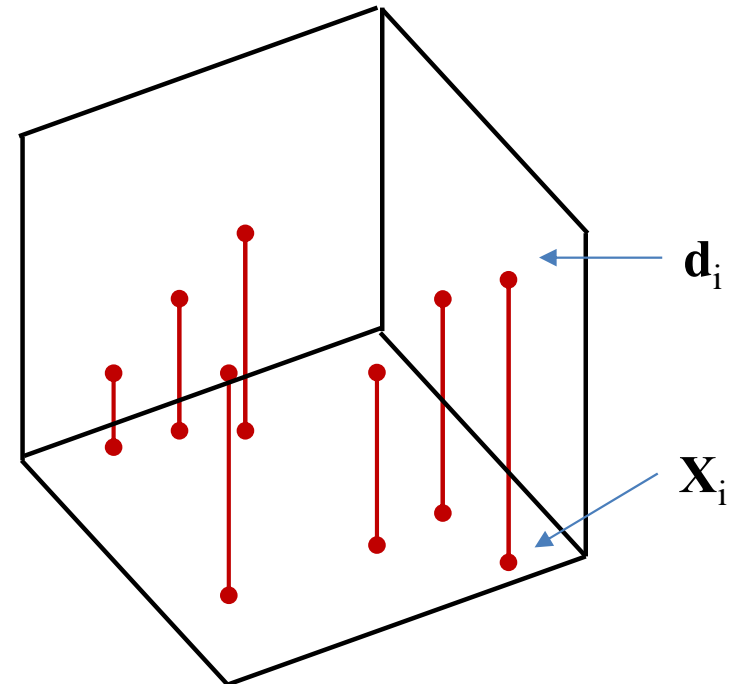
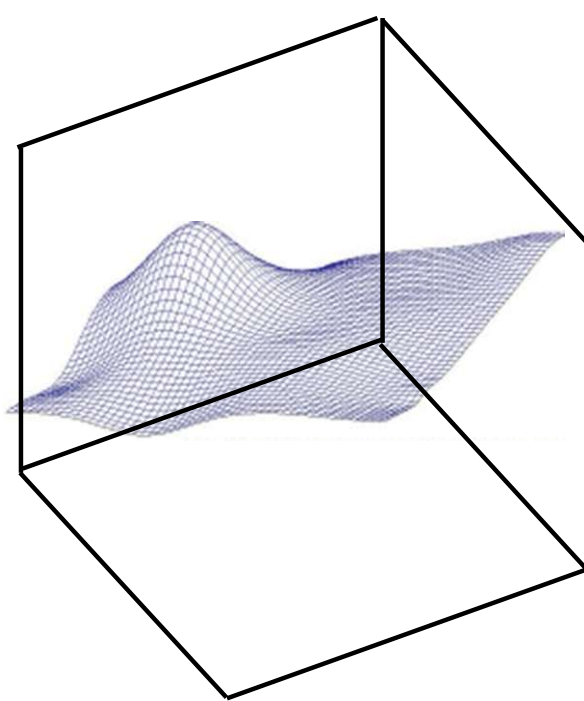
$$\begin{aligned}\widehat{W} &= \operatorname{argmin}_W \int_X \operatorname{div}(f(X; W), g(X)) P(X) dX \\ &= \operatorname{argmin}_W E[\operatorname{div}(f(X; W), g(X))]\end{aligned}$$

Recap: Sampling the function



- *We don't have $g(X)$ so sample $g(X)$*
 - Obtain input-output pairs for a number of samples of input X_i
 - Good sampling: the samples of X will be drawn from $P(X)$
- Estimate function from the samples

The *Empirical* risk



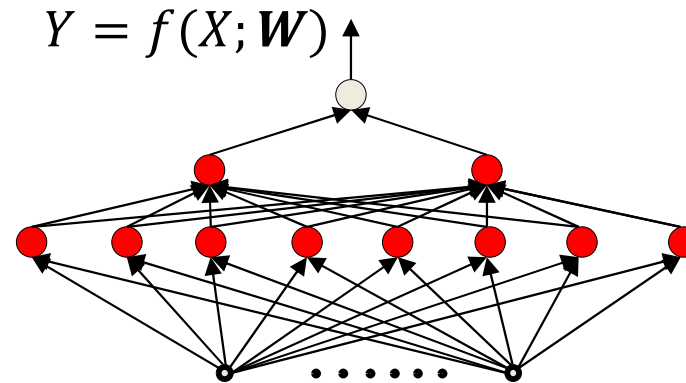
- The *expected* divergence (or risk) is the average divergence over the entire input space

$$E[\text{div}(f(X; W), g(X))] = \int_X \text{div}(f(X; W), g(X)) P(X) dX$$

- The *empirical estimate* of the expected risk is the *average* divergence over the samples

$$E[\text{div}(f(X; W), g(X))] \approx \frac{1}{N} \sum_{i=1}^N \text{div}(f(X_i; W), d_i)$$

Training the network: Empirical Risk Minimization



- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_N, d_N)$
 - Quantification of error on the i^{th} instance: $\text{div}(f(X_i; W), d_i)$
 - Empirical average divergence (**Empirical Risk**) on all training data:

$$\text{Loss}(W) = \frac{1}{N} \sum_i \text{div}(f(X_i; W), d_i)$$

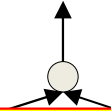
- Estimate the parameters to minimize the empirical estimate of expected divergence (**empirical risk**)

$$\widehat{W} = \underset{W}{\operatorname{argmin}} \text{Loss}(W)$$

- I.e. minimize the *empirical risk* over the drawn samples

Empirical Risk Minimization

$$Y = f(X; W)$$



Note : It's really a measure of error, but using standard terminology, we will call it a "Loss"

Note 2: The empirical risk $Loss(W)$ is only an empirical approximation to the true risk $E[div(f(X; W), g(X))]$ which is our *actual* minimization objective

Note 3: For a *given training set* the loss is only a function of W

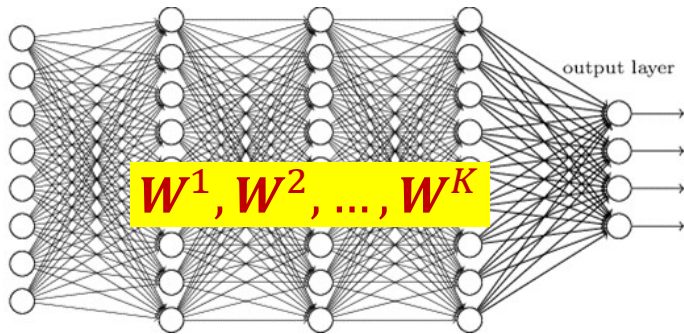
$$Loss(W) = \frac{1}{N} \sum_i div(f(X_i; W), d_i)$$

- Estimate the parameters to minimize the empirical estimate of expected error

$$\widehat{W} = \underset{W}{\operatorname{argmin}} Loss(W)$$

- I.e. minimize the *empirical error* over the drawn samples

ERM for neural networks



Actual output of network:

$$Y_i = \text{net} \left(X_i; \left\{ w_{i,j}^k \forall i, j, k \right\} \right) \\ = \text{net}(X_i; W^1, W^2, \dots, W^K)$$

Desired output of network: d_i

Error on i-th training input: $\text{Div}(Y_i, d_i; W^1, W^2, \dots, W^K)$

Average training error(loss):

$$\text{Loss}(W^1, W^2, \dots, W^K) = \frac{1}{N} \sum_{i=1}^N \text{Div}(Y_i, d_i; W^1, W^2, \dots, W^K)$$

- What is the exact form of $\text{Div}()$? More on this later
- Optimize network parameters to minimize the total error over all training inputs

Problem Statement

- Given a training set of input-output pairs $(X_1, d_1), (X_2, d_2), \dots, (X_N, d_N)$

- Minimize the following function

$$Loss(W) = \frac{1}{N} \sum_i div(f(X_i; W), d_i)$$

w.r.t W

- This is problem of function minimization
 - An instance of optimization

Story so far

- We learn networks by “fitting” them to training instances drawn from a target function
- Learning networks of threshold-activation perceptrons requires solving a hard combinatorial-optimization problem
 - Because we cannot compute the influence of small changes to the parameters on the overall error
- Instead we use continuous activation functions with non-zero derivatives to enable us to estimate network parameters
 - This makes the output of the network differentiable w.r.t every parameter in the network
 - The *logistic* activation perceptron actually computes the *a posteriori* probability of the output given the input
- We define differentiable *divergence* between the output of the network and the desired output for the training instances
 - And a total error, which is the average divergence over all training instances
- We optimize network parameters to minimize this error
 - Empirical risk minimization
- This is an instance of function minimization